

Table of Contents

1. Basic Geometry
2. Segment To Line
3. Lines Intersection
4. Check Segments Intersection
5. Segments Intersection
6. Circle Line Intersection
7. Circle Circle Intersection
8. Tangents To Two Circles
9. Length Of Segments Union
10. Oriented Triangle Area
11. Area Of Simple Polygon
12. Point In Convex Polygon
13. Minkowski
14. Picks Theorem
15. Lattice Points
16. Convex Hull
17. Convex_hull_trick
18. Intersecting_segments
19. Planar
20. Point Location
21. Nearest_points
22. Delaunay
23. Vertical_decomposition
24. Halfplane Intersection
25. Halfplane Intersection Maybe Fixed
26. Manhattan Distance
27. Enclosing Circle

Original

Basic Geometry

In this article we will consider basic operations on points in Euclidean space which maintains the foundation of the whole analytical geometry. We will consider for each point r the vector $\vec{r}$ directed from O to r . Later we will not distinguish between r and $\vec{r}$ and use the term **point** as a synonym for **vector**.

Linear operations

Both 2D and 3D points maintain linear space, which means that for them sum of points and multiplication of point by some number are defined. Here are those basic implementations for 2D:

```
struct point2d {
    ftype x, y;
    point2d() {}
    point2d(ftype x, ftype y): x(x), y(y) {}
    point2d& operator+=(const point2d &t) {
        x += t.x;
        y += t.y;
        return *this;
    }
    point2d& operator-=(const point2d &t) {
        x -= t.x;
        y -= t.y;
        return *this;
    }
    point2d& operator*=(ftype t) {
        x *= t;
        y *= t;
        return *this;
    }
    point2d& operator/=(ftype t) {
        x /= t;
        y /= t;
        return *this;
    }
}
```

```

    point2d operator+(const point2d &t) const {
        return point2d(*this) += t;
    }
    point2d operator-(const point2d &t) const {
        return point2d(*this) -= t;
    }
    point2d operator*(ftype t) const {
        return point2d(*this) *= t;
    }
    point2d operator/(ftype t) const {
        return point2d(*this) /= t;
    }
};
point2d operator*(ftype a, point2d b) {
    return b * a;
}

```

And 3D points:

```

struct point3d {
    ftype x, y, z;
    point3d() {}
    point3d(ftype x, ftype y, ftype z): x(x), y(y), z(z) {}
    point3d& operator+=(const point3d &t) {
        x += t.x;
        y += t.y;
        z += t.z;
        return *this;
    }
    point3d& operator-=(const point3d &t) {
        x -= t.x;
        y -= t.y;
        z -= t.z;
        return *this;
    }
    point3d& operator*=(ftype t) {
        x *= t;
        y *= t;
        z *= t;
        return *this;
    }
    point3d& operator/=(ftype t) {
        x /= t;
        y /= t;
        z /= t;
        return *this;
    }
    point3d operator+(const point3d &t) const {
        return point3d(*this) += t;
    }
}

```

```

    }
    point3d operator-(const point3d &t) const {
        return point3d(*this) -= t;
    }
    point3d operator*(ftype t) const {
        return point3d(*this) *= t;
    }
    point3d operator/(ftype t) const {
        return point3d(*this) /= t;
    }
};
point3d operator*(ftype a, point3d b) {
    return b * a;
}

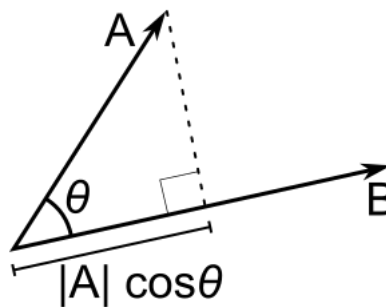
```

Here `ftype` is some type used for coordinates, usually `int`, `double` or `long long`.

Dot product

Definition

The dot (or scalar) product $a \cdot b$ for vectors a and b can be defined in two identical ways. Geometrically it is product of the length of the first vector by the length of the projection of the second vector onto the first one. As you may see from the image below this projection is nothing but $|a| \cos \theta$ where θ is the angle between a and b . Thus $a \cdot b = |a| \cos \theta \cdot |b|$.



The dot product holds some notable properties:

1. $a \cdot b = b \cdot a$
2. $(\alpha \cdot a) \cdot b = \alpha \cdot (a \cdot b)$
3. $(a + b) \cdot c = a \cdot c + b \cdot c$

i.e. it is a commutative function which is linear with respect to both arguments. Let's denote the unit vectors as

$$\mathbf{e}_x = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \mathbf{e}_y = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \mathbf{e}_z = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

With this notation we can write the vector $\mathbf{r} = (x; y; z)$ as $\mathbf{r} = x \cdot \mathbf{e}_x + y \cdot \mathbf{e}_y + z \cdot \mathbf{e}_z$. And since for unit vectors

$$\mathbf{e}_x \cdot \mathbf{e}_x = \mathbf{e}_y \cdot \mathbf{e}_y = \mathbf{e}_z \cdot \mathbf{e}_z = 1, \mathbf{e}_x \cdot \mathbf{e}_y = \mathbf{e}_y \cdot \mathbf{e}_z = \mathbf{e}_z \cdot \mathbf{e}_x = 0$$

we can see that in terms of coordinates for $\mathbf{a} = (x_1; y_1; z_1)$ and $\mathbf{b} = (x_2; y_2; z_2)$ holds

$$\mathbf{a} \cdot \mathbf{b} = (x_1 \cdot \mathbf{e}_x + y_1 \cdot \mathbf{e}_y + z_1 \cdot \mathbf{e}_z) \cdot (x_2 \cdot \mathbf{e}_x + y_2 \cdot \mathbf{e}_y + z_2 \cdot \mathbf{e}_z) = x_1 x_2 + y_1 y_2 + z_1 z_2.$$

That is also the algebraic definition of the dot product. From this we can write functions which calculate it.

```
ftype dot(point2d a, point2d b) {
    return a.x * b.x + a.y * b.y;
}
ftype dot(point3d a, point3d b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
```

When solving problems one should use algebraic definition to calculate dot products, but keep in mind geometric definition and properties to use it.

Properties

We can define many geometrical properties via the dot product. For example

1. Norm of $\mathbf{a}$ (squared length): $|\mathbf{a}|^2 = \mathbf{a} \cdot \mathbf{a}$

2. Length of a : $|a| = \sqrt{a \cdot a}$

3. Projection of a onto b : $\frac{a \cdot b}{|b|}$

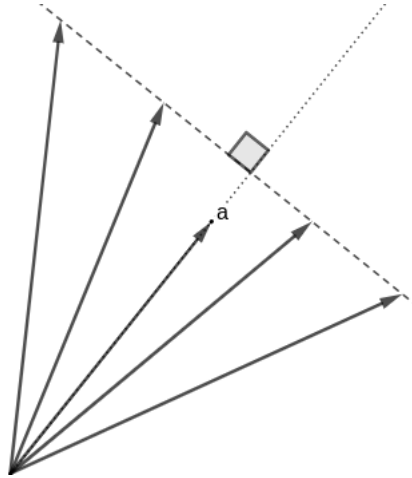
4. Angle between vectors: $\arccos\left(\frac{a \cdot b}{|a| \cdot |b|}\right)$

5. From the previous point we may see that the dot product is positive if the angle between them is acute, negative if it is obtuse and it equals zero if they are orthogonal, i.e. they form a right angle.

Note that all these functions do not depend on the number of dimensions, hence they will be the same for the 2D and 3D case:

```
ftype norm(point2d a) {
    return dot(a, a);
}
double abs(point2d a) {
    return sqrt(norm(a));
}
double proj(point2d a, point2d b) {
    return dot(a, b) / abs(b);
}
double angle(point2d a, point2d b) {
    return acos(dot(a, b) / abs(a) / abs(b));
}
```

To see the next important property we should take a look at the set of points r for which $r \cdot a = C$ for some fixed constant C . You can see that this set of points is exactly the set of points for which the projection onto a is the point $C \cdot \frac{a}{|a|^2}$ and they form a hyperplane orthogonal to a . You can see the vector a alongside with several such vectors having same dot product with it in 2D on the picture below:

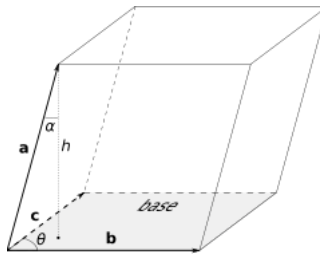


In 2D these vectors will form a line, in 3D they will form a plane. Note that this result allows us to define a line in 2D as $\mathbf{r} \cdot \mathbf{n} = C$ or $(\mathbf{r} - \mathbf{r}_0) \cdot \mathbf{n} = 0$ where $\mathbf{n}$ is vector orthogonal to the line and $\mathbf{r}_0$ is any vector already present on the line and $C = \mathbf{r}_0 \cdot \mathbf{n}$. In the same manner a plane can be defined in 3D.

Cross product

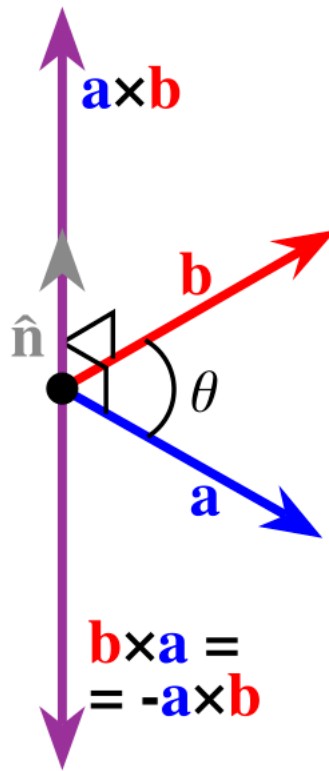
Definition

Assume you have three vectors $\mathbf{a}$, $\mathbf{b}$ and $\mathbf{c}$ in 3D space joined in a parallelepiped as in the picture below:



How would you calculate its volume? From school we know that we should multiply the area of the base with the height, which is projection of $\mathbf{a}$ onto direction orthogonal to base. That means that if we define $\mathbf{b} \times \mathbf{c}$ as the vector which is orthogonal to both $\mathbf{b}$ and $\mathbf{c}$ and which length is equal to the area of the parallelogram formed by $\mathbf{b}$ and $\mathbf{c}$ then $|\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})|$ will be equal to the volume of the parallelepiped. For integrity we will say that $\mathbf{b} \times \mathbf{c}$ will be always directed in such way that the rotation from the vector $\mathbf{b}$

to the vector c from the point of $b \times c$ is always counter-clockwise (see the picture below).



This defines the cross (or vector) product $b \times c$ of the vectors b and c and the triple product $a \cdot (b \times c)$ of the vectors a , b and c .

Some notable properties of cross and triple products:

1. $a \times b = -b \times a$
2. $(\alpha \cdot a) \times b = \alpha \cdot (a \times b)$
3. For any b and c there is exactly one vector r such that $a \cdot (b \times c) = a \cdot r$ for any vector a .
Indeed if there are two such vectors r_1 and r_2 then $a \cdot (r_1 - r_2) = 0$ for all vectors a which is possible only when $r_1 = r_2$.
4. $a \cdot (b \times c) = b \cdot (c \times a) = -a \cdot (c \times b)$
5. $(a + b) \times c = a \times c + b \times c$. Indeed for all vectors r the chain of equations holds:

$$r \cdot ((a + b) \times c) = (a + b) \cdot (c \times r) = a \cdot (c \times r) + b \cdot (c \times r) = r \cdot (a \times c + b \times c)$$

Which proves $(a + b) \times c = a \times c + b \times c$ due to point 3.

6. $|a \times b| = |a| \cdot |b| \sin \theta$ where θ is angle between a and b , since $|a \times b|$ equals to the area of the parallelogram formed by a and b .

Given all this and that the following equation holds for the unit vectors

$$e_x \times e_x = e_y \times e_y = e_z \times e_z = 0, e_x \times e_y = e_z, e_y \times e_z = e_x, e_z \times e_x = e_y$$

we can calculate the cross product of $a = (x_1; y_1; z_1)$ and $b = (x_2; y_2; z_2)$ in coordinate form:

$$a \times b = (x_1 \cdot e_x + y_1 \cdot e_y + z_1 \cdot e_z) \times (x_2 \cdot e_x + y_2 \cdot e_y + z_2 \cdot e_z) = (y_1 z_2 - z_1 y_2) e_x + (z_1 x_2 - x_1 z_2) e_y + (x_1 y_2 - y_1 x_2) e_z$$

Which also can be written in the more elegant form:

$$a \times b = \begin{vmatrix} e_x & e_y & e_z \\ x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \end{vmatrix}, a \cdot (b \times c) = \begin{vmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{vmatrix}$$

Here $|\cdot|$ stands for the determinant of a matrix.

Some kind of cross product (namely the pseudo-scalar product) can also be implemented in the 2D case. If we would like to calculate the area of parallelogram formed by vectors a and b we would compute $|e_z \cdot (a \times b)| = |x_1 y_2 - y_1 x_2|$.

Another way to obtain the same result is to multiply $|a|$ (base of parallelogram) with the height, which is the projection of vector b onto vector a rotated by 90° which in turn is $\hat{a} = (-y_1; x_1)$. That is, to calculate $|\hat{a} \cdot b| = |x_1 y_2 - y_1 x_2|$.

If we will take the sign into consideration then the area will be positive if the rotation from a to b (i.e. from the view of the point of e_z) is performed counter-clockwise and negative otherwise. That defines the pseudo-scalar product. Note that it also equals $|a| \cdot |b| \sin \theta$ where θ is angle from a to b count counter-clockwise (and negative if rotation is clockwise).

Let's implement all this stuff!

```
point3d cross(point3d a, point3d b) {
    return point3d(a.y * b.z - a.z * b.y,
                  a.z * b.x - a.x * b.z,
                  a.x * b.y - a.y * b.x);
}

ftype triple(point3d a, point3d b, point3d c) {
    return dot(a, cross(b, c));
}

ftype cross(point2d a, point2d b) {
    return a.x * b.y - a.y * b.x;
}
```

Properties

As for the cross product, it equals to the zero vector iff the vectors a and b are collinear (they form a common line, i.e. they are parallel). The same thing holds for the triple product, it is equal to zero iff the vectors a , b and c are coplanar (they form a common plane).

From this we can obtain universal equations defining lines and planes. A line can be defined via its direction vector d and an initial point r_0 or by two points a and b . It is defined as $(r - r_0) \times d = 0$ or as $(r - a) \times (b - a) = 0$. As for planes, it can be defined by three points a , b and c as $(r - a) \cdot ((b - a) \times (c - a)) = 0$ or by initial point r_0 and two direction vectors lying in this plane d_1 and d_2 :

$$(r - r_0) \cdot (d_1 \times d_2) = 0.$$

In 2D the pseudo-scalar product also may be used to check the orientation between two vectors because it is positive if the rotation from the first to the second vector is clockwise and negative otherwise. And, of course, it can be used to calculate areas of polygons, which is described in a different article. A triple product can be used for the same purpose in 3D space.

Exercises

Line intersection

There are many possible ways to define a line in 2D and you shouldn't hesitate to combine them. For example we have two lines and we want to find their intersection points. We can say that all points from first line can be parameterized as $r = a_1 + t \cdot d_1$ where a_1 is initial point, d_1 is direction and t is some real parameter. As for second line all its points must satisfy $(r - a_2) \times d_2 = 0$. From this we can easily find parameter t :

$$(a_1 + t \cdot d_1 - a_2) \times d_2 = 0 \Rightarrow t = \frac{(a_2 - a_1) \times d_2}{d_1 \times d_2}$$

Let's implement function to intersect two lines.

```
point2d intersect(point2d a1, point2d d1, point2d a2, point2d d2) {
    return a1 + cross(a2 - a1, d2) / cross(d1, d2) * d1;
}
```

Planes intersection

However sometimes it might be hard to use some geometric insights. For example, you're given three planes defined by initial points a_i and directions d_i and you want to find their intersection point. You may note that you just have to solve the system of equations:

$$\begin{cases} r \cdot n_1 = a_1 \cdot n_1, \\ r \cdot n_2 = a_2 \cdot n_2, \\ r \cdot n_3 = a_3 \cdot n_3 \end{cases}$$

Instead of thinking on geometric approach, you can work out an algebraic one which can be obtained immediately. For example, given that you already implemented a point class, it will be easy for you to solve this system using Cramer's rule because the triple product is simply the determinant of the matrix obtained from the vectors being its columns:

```
point3d intersect(point3d a1, point3d n1, point3d a2, point3d n2,
point3d a3, point3d n3) {
    point3d x(n1.x, n2.x, n3.x);
    point3d y(n1.y, n2.y, n3.y);
    point3d z(n1.z, n2.z, n3.z);
    point3d d(dot(a1, n1), dot(a2, n2), dot(a3, n3));
    return point3d(triple(d, y, z),
                    triple(x, d, z),
                    triple(x, y, d)) / triple(n1, n2, n3);
}
```

Now you may try to find out approaches for common geometric operations yourself to get used to all this stuff.

Contributors:

[Alexander Kulkov](#) (78.59%) [Jakob Kogler](#) (15.9%) [Michael Hayter](#) (3.67%)
[Oleksandr Kulkov](#) (1.53%) [Syed Umair](#) (0.31%)

Translated

From: e-maxx.ru

Finding the equation of a line for a segment

The task is: given the coordinates of the ends of a segment, construct a line passing through it.

We assume that the segment is non-degenerate, i.e. has a length greater than zero (otherwise, of course, infinitely many different lines pass through it).

Two-dimensional case

Let the given segment be PQ i.e. the known coordinates of its ends P_x, P_y, Q_x, Q_y .

It is necessary to construct **the equation of a line in the plane** passing through this segment, i.e. find the coefficients A, B, C in the equation of a line:

$$Ax + By + C = 0.$$

Note that for the required triples (A, B, C) there are **infinitely many** solutions which describe the given segment: you can multiply all three coefficients by an arbitrary non-zero number and get the same straight line. Therefore, our task is to find one of these triples.

It is easy to verify (by substitution of these expressions and the coordinates of the points P and Q into the equation of a straight line) that the following set of coefficients fits:

$$\begin{aligned} A &= P_y - Q_y, \\ B &= Q_x - P_x, \\ C &= -AP_x - BP_y. \end{aligned}$$

Integer case

An important advantage of this method of constructing a straight line is that if the coordinates of the ends were integer, then the coefficients obtained will also be **integer**. In some cases, this allows one to perform geometric operations without resorting to real numbers at all.

However, there is a small drawback: for the same straight line different triples of coefficients can be obtained. To avoid this, but do not go away from the integer coefficients, you can apply the following technique, often called **rationing**. Find the [greatest common divisor](#) of numbers $|A|, |B|, |C|$, we divide all three coefficients by it, and then we make the normalization of the sign: if $A < 0$ or $A = 0, B < 0$ then multiply all three coefficients by -1 . As a result, we will come to the conclusion that for identical straight lines, identical triples of coefficients will be obtained, which makes it easy to check straight lines for equality.

Real case

When working with real numbers, you should always be aware of errors.

The coefficients A and B will have the order of the original coordinates, the coefficient C is of the order of the square of them. This may already be quite large numbers, and, for example, when we [intersect straight lines](#), they will become even larger, which can lead to large rounding errors already when the coordinates of the end points are of order 10^3 .

Therefore, when working with real numbers, it is desirable to produce the so-called **normalization**, this is straightforward: namely, to make the coefficients such that $A^2 + B^2 = 1$. To do this, calculate the number Z :

$$Z = \sqrt{A^2 + B^2},$$

and divide all three coefficients A, B, C by it.

Thus, the order of the coefficients A and B will not depend on the order of the input coordinates, and the coefficient C will be of the same order as the input coordinates. In practice, this leads to a significant improvement in the accuracy of calculations.

Finally, we mention the **comparison** of straight lines - in fact, after such a normalization, for the same straight line, only two triples of coefficients can be obtained: up to multiplication by -1 . Accordingly, if we make an additional normalization taking into account the sign (if $A < -\varepsilon$ or $|A| < \varepsilon$, $B < -\varepsilon$ then multiply by -1), the resulting coefficients will be unique.

Three-dimensional and multidimensional case

Already in the three-dimensional case there is **no simple equation** describing a straight line (it can be defined as the intersection of two planes, that is, a system of two equations, but this is an inconvenient method).

Consequently, in the three-dimensional and multidimensional cases we must use the **parametric method of defining a straight line**, i.e. as a point p and a vector v :

$$p + vt, \quad t \in \mathbb{R}.$$

Those. a straight line is all points that can be obtained from a point p adding a vector v with an arbitrary coefficient.

The **construction** of a straight line in a parametric form along the coordinates of the ends of a segment is trivial, we just take one end of the segment for the point p , and the vector from the first to the second end — for the vector v .

Contributors:

[David Castelblanco](#) (80.88%) [Jakob Kogler](#) (10.29%) [Oleksandr Kulkov](#) (8.82%)

Translated

From: e-maxx.ru

Intersection Point of Lines

You are given two lines, described via the equations $a_1x + b_1y + c_1 = 0$ and $a_2x + b_2y + c_2 = 0$. We have to find the intersection point of the lines, or determine that the lines are parallel.

Solution

If two lines are not parallel, they intersect. To find their intersection point, we need to solve the following system of linear equations:

$$\begin{cases} a_1x + b_1y + c_1 = 0 \\ a_2x + b_2y + c_2 = 0 \end{cases}$$

Using Cramer's rule, we can immediately write down the solution for the system, which will give us the required intersection point of the lines:

$$x = -\frac{\begin{vmatrix} c_1 & b_1 \\ c_2 & b_2 \end{vmatrix}}{\begin{vmatrix} a_1 & b_1 \\ a_2 & b_2 \end{vmatrix}} = -\frac{c_1b_2 - c_2b_1}{a_1b_2 - a_2b_1},$$

$$y = -\frac{\begin{vmatrix} a_1 & c_1 \\ a_2 & c_2 \end{vmatrix}}{\begin{vmatrix} a_1 & b_1 \\ a_2 & b_2 \end{vmatrix}} = -\frac{a_1c_2 - a_2c_1}{a_1b_2 - a_2b_1}.$$

If the denominator equals 0, i.e.

$$\begin{vmatrix} a_1 & b_1 \\ a_2 & b_2 \end{vmatrix} = a_1 b_2 - a_2 b_1 = 0$$

then either the system has no solutions (the lines are parallel and distinct) or there are infinitely many solutions (the lines overlap). If we need to distinguish these two cases, we have to check if coefficients c are proportional with the same ratio as the coefficients a and b . To do that we only have calculate the following determinants, and if they both equal 0, the lines overlap:

$$\begin{vmatrix} a_1 & c_1 \\ a_2 & c_2 \end{vmatrix}, \begin{vmatrix} b_1 & c_1 \\ b_2 & c_2 \end{vmatrix}$$

Notice, a different approach for computing the intersection point is explained in the article [Basic Geometry](#).

Implementation

```
struct pt {
    double x, y;
};

struct line {
    double a, b, c;
};

const double EPS = 1e-9;

double det(double a, double b, double c, double d) {
    return a*d - b*c;
}

bool intersect(line m, line n, pt & res) {
    double zn = det(m.a, m.b, n.a, n.b);
    if (abs(zn) < EPS)
        return false;
    res.x = -det(m.c, m.b, n.c, n.b) / zn;
    res.y = -det(m.a, m.c, n.a, n.c) / zn;
}
```

```
        return true;
    }

    bool parallel(line m, line n) {
        return abs(det(m.a, m.b, n.a, n.b)) < EPS;
    }

    bool equivalent(line m, line n) {
        return abs(det(m.a, m.b, n.a, n.b)) < EPS
            && abs(det(m.a, m.c, n.a, n.c)) < EPS
            && abs(det(m.b, m.c, n.b, n.c)) < EPS;
    }
}
```

Contributors:

[Silap Aliyev](#) (64.86%) [Jakob Kogler](#) (25.68%) [Oleksandr Kulkov](#) (8.11%)
[Mariia Mykhailova](#) (1.35%)

Translated

From: e-maxx.ru

Check if two segments intersect

You are given two segments (a, b) and (c, d) . You have to check if they intersect. Of course, you may find their intersection and check if it isn't empty, but this can't be done in integers for segments with integer coordinates. The approach described here can work in integers.

Algorithm

Firstly, consider the case when the segments are part of the same line. In this case it is sufficient to check if their projections on Ox and Oy intersect. In the other case a and b must not lie on the same side of line (c, d) , and c and d must not lie on the same side of line (a, b) . It can be checked with a couple of cross products.

Implementation

The given algorithm is implemented for integer points. Of course, it can be easily modified to work with doubles.

```
struct pt {
    long long x, y;
    pt() {}
    pt(long long _x, long long _y) : x(_x), y(_y) {}
    pt operator-(const pt& p) const { return pt(x - p.x, y - p.y); }
    long long cross(const pt& p) const { return x * p.y - y * p.x; }
    long long cross(const pt& a, const pt& b) const { return (a -
*this).cross(b - *this); }
};

int sgn(const long long& x) { return x >= 0 ? x ? 1 : 0 : -1; }
```

```
bool inter1(long long a, long long b, long long c, long long d) {  
    if (a > b)  
        swap(a, b);  
    if (c > d)  
        swap(c, d);  
    return max(a, c) <= min(b, d);  
}  
  
bool check_inter(const pt& a, const pt& b, const pt& c, const pt& d) {  
    if (c.cross(a, d) == 0 && c.cross(b, d) == 0)  
        return inter1(a.x, b.x, c.x, d.x) && inter1(a.y, b.y, c.y, d.y);  
    return sgn(a.cross(b, c)) != sgn(a.cross(b, d)) &&  
        sgn(c.cross(d, a)) != sgn(c.cross(d, b));  
}
```

Contributors:

[Yury Semenov](#) (86.27%) [Oleksandr Kulkov](#) (11.76%) [Jakob Kogler](#) (1.96%)

Translated

From: e-maxx.ru

Finding intersection of two segments

You are given two segments AB and CD, described as pairs of their endpoints. Each segment can be a single point if its endpoints are the same. You have to find the intersection of these segments, which can be empty (if the segments don't intersect), a single point or a segment (if the given segments overlap).

Solution

We can find the intersection point of segments in the same way as [the intersection of lines](#): reconstruct line equations from the segments' endpoints and check whether they are parallel.

If the lines are not parallel, we need to find their point of intersection and check whether it belongs to both segments (to do this it's sufficient to verify that the intersection point belongs to each segment projected on X and Y axes). In this case the answer will be either "no intersection" or the single point of lines' intersection.

The case of parallel lines is slightly more complicated (the case of one or more segments being a single point also belongs here). In this case we need to check that both segments belong to the same line. If they don't, the answer is "no intersection". If they do, the answer is the intersection of the segments belonging to the same line, which is obtained by ordering the endpoints of both segments in the increasing order of certain coordinate and taking the rightmost of left endpoints and the leftmost of right endpoints.

If both segments are single points, these points have to be identical, and it makes sense to perform this check separately.

In the beginning of the algorithm let's add a bounding box check - it is necessary for the case when the segments belong to the same line, and (being a lightweight check) it

allows the algorithm to work faster on average on random tests.

Implementation

Here is the implementation, including all helper functions for lines and segments processing.

The main function `intersect` returns true if the segments have a non-empty intersection, and stores endpoints of the intersection segment in arguments `left` and `right`. If the answer is a single point, the values written to `left` and `right` will be the same.

```
const double EPS = 1E-9;

struct pt {
    double x, y;

    bool operator<(const pt& p) const
    {
        return x < p.x - EPS || (abs(x - p.x) < EPS && y < p.y - EPS);
    }
};

struct line {
    double a, b, c;

    line() {}
    line(pt p, pt q)
    {
        a = p.y - q.y;
        b = q.x - p.x;
        c = -a * p.x - b * p.y;
        norm();
    }

    void norm()
    {
        double z = sqrt(a * a + b * b);
        if (abs(z) > EPS)
            a /= z, b /= z, c /= z;
    }

    double dist(pt p) const { return a * p.x + b * p.y + c; }
```

```

};

double det(double a, double b, double c, double d)
{
    return a * d - b * c;
}

inline bool betw(double l, double r, double x)
{
    return min(l, r) <= x + EPS && x <= max(l, r) + EPS;
}

inline bool intersect_1d(double a, double b, double c, double d)
{
    if (a > b)
        swap(a, b);
    if (c > d)
        swap(c, d);
    return max(a, c) <= min(b, d) + EPS;
}

bool intersect(pt a, pt b, pt c, pt d, pt& left, pt& right)
{
    if (!intersect_1d(a.x, b.x, c.x, d.x) || !intersect_1d(a.y, b.y,
c.y, d.y))
        return false;
    line m(a, b);
    line n(c, d);
    double zn = det(m.a, m.b, n.a, n.b);
    if (abs(zn) < EPS) {
        if (abs(m.dist(c)) > EPS || abs(n.dist(a)) > EPS)
            return false;
        if (b < a)
            swap(a, b);
        if (d < c)
            swap(c, d);
        left = max(a, c);
        right = min(b, d);
        return true;
    } else {
        left.x = right.x = -det(m.c, m.b, n.c, n.b) / zn;
        left.y = right.y = -det(m.a, m.c, n.a, n.c) / zn;
        return betw(a.x, b.x, left.x) && betw(a.y, b.y, left.y) &&
            betw(c.x, d.x, left.x) && betw(c.y, d.y, left.y);
    }
}

```

Contributors:

[Mariia Mykhailova](#) (92.37%) [Oleksandr Kulkov](#) (5.08%) [Jakob Kogler](#) (2.54%)

Translated

From: e-maxx.ru

Circle-Line Intersection

Given the coordinates of the center of a circle and its radius, and the equation of a line, you're required to find the points of intersection.

Solution

Instead of solving the system of two equations, we will approach the problem geometrically. This way we get a more accurate solution from the point of view of numerical stability.

We assume without loss of generality that the circle is centered at the origin. If it's not, we translate it there and correct the C constant in the line equation. So we have a circle centered at $(0, 0)$ of radius r and a line with equation $Ax + By + C = 0$.

Let's start by find the point on the line which is closest to the origin (x_0, y_0) . First, it has to be at a distance

$$d_0 = \frac{|C|}{\sqrt{A^2 + B^2}}$$

Second, since the vector (A, B) is perpendicular to the line, the coordinates of the point must be proportional to the coordinates of this vector. Since we know the distance of the point to the origin, we just need to scale the vector (A, B) to this length, and we'll get:

$$x_0 = -\frac{AC}{A^2 + B^2}$$

$$y_0 = -\frac{BC}{A^2 + B^2}$$

The minus signs are not obvious, but they can be easily verified by substituting x_0 and y_0 in the equation of the line.

At this stage we can determine the number of intersection points, and even find the solution when there is one or zero points. Indeed, if the distance from (x_0, y_0) to the origin d_0 is greater than the radius r , the answer is **zero points**. If $d_0 = r$, the answer is **one point** (x_0, y_0) . If $d_0 < r$, there are two points of intersection, and now we have to find their coordinates.

So, we know that the point (x_0, y_0) is inside the circle. The two points of intersection, (a_x, a_y) and (b_x, b_y) , must belong to the line $Ax + By + C = 0$ and must be at the same distance d from (x_0, y_0) , and this distance is easy to find:

$$d = \sqrt{r^2 - \frac{C^2}{A^2 + B^2}}$$

Note that the vector $(-B, A)$ is collinear to the line, and thus we can find the points in question by adding and subtracting vector $(-B, A)$, scaled to the length d , to the point (x_0, y_0) .

Finally, the equations of the two points of intersection are:

$$m = \sqrt{\frac{d^2}{A^2 + B^2}}$$

$$a_x = x_0 + B \cdot m, a_y = y_0 - A \cdot m$$

$$b_x = x_0 - B \cdot m, b_y = y_0 + A \cdot m$$

Had we solved the original system of equations using algebraic methods, we would likely get an answer in a different form with a larger error. The geometric method described here is more graphic and more accurate.

Implementation

As indicated at the outset, we assume that the circle is centered at the origin, and therefore the input to the program is the radius r of the circle and the parameters A , B and C of the equation of the line.

```
double r, a, b, c; // given as input
double x0 = -a*c/(a*a+b*b), y0 = -b*c/(a*a+b*b);
if (c*c > r*r*(a*a+b*b)+EPS)
    puts ("no points");
else if (abs (c*c - r*r*(a*a+b*b)) < EPS) {
    puts ("1 point");
    cout << x0 << ' ' << y0 << '\n';
}
else {
    double d = r*r - c*c/(a*a+b*b);
    double mult = sqrt (d / (a*a+b*b));
    double ax, ay, bx, by;
    ax = x0 + b * mult;
    bx = x0 - b * mult;
    ay = y0 - a * mult;
    by = y0 + a * mult;
    puts ("2 points");
    cout << ax << ' ' << ay << '\n' << bx << ' ' << by << '\n';
}
```

Translated

From: e-maxx.ru

Circle-Circle Intersection

You are given two circles on a 2D plane, each one described as coordinates of its center and its radius. Find the points of their intersection (possible cases: one or two points, no intersection or circles coincide).

Solution

Let's reduce this problem to the [circle-line intersection problem](#).

Assume without loss of generality that the first circle is centered at the origin (if this is not true, we can move the origin to the center of the first circle and adjust the coordinates of intersection points accordingly at output time). We have a system of two equations:

$$x^2 + y^2 = r_1^2$$

$$(x - x_2)^2 + (y - y_2)^2 = r_2^2$$

Subtract the first equation from the second one to get rid of the second powers of variables:

$$x^2 + y^2 = r_1^2$$

$$x \cdot (-2x_2) + y \cdot (-2y_2) + (x_2^2 + y_2^2 + r_1^2 - r_2^2) = 0$$

Thus, we've reduced the original problem to the problem of finding intersections of the first circle and a line:

$$Ax + By + C = 0$$

$$A = -2x_2$$

$$B = -2y_2$$

$$C = x_2^2 + y_2^2 + r_1^2 - r_2^2$$

And this problem can be solved as described in the [corresponding article](#).

The only degenerate case we need to consider separately is when the centers of the circles coincide. In this case $x_2 = y_2 = 0$, and the line equation will be $C = r_1^2 - r_2^2 = 0$. If the radii of the circles are the same, there are infinitely many intersection points, if they differ, there are no intersections.

Translated

From: e-maxx.ru

Finding common tangents to two circles

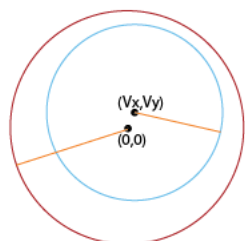
Given two circles. It is required to find all their common tangents, i.e. all such lines that touch both circles simultaneously.

The described algorithm will also work in the case when one (or both) circles degenerate into points. Thus, this algorithm can also be used to find tangents to a circle passing through a given point.

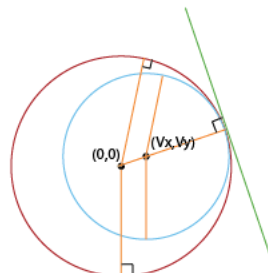
The number of common tangents

The number of common tangents to two circles can be **0,1,2,3,4** and **infinite**. Look at the images for different cases.

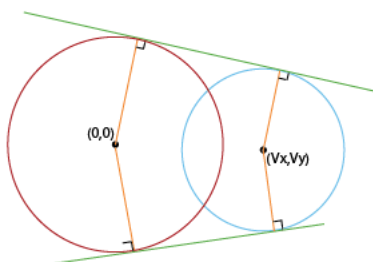
Case : No tangent



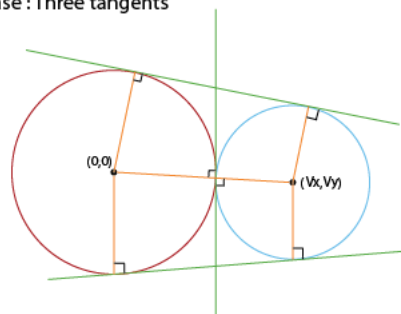
Case: One tangent



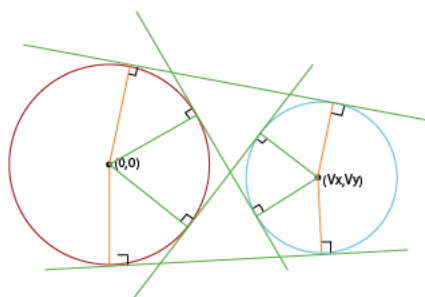
Case : Two tangents



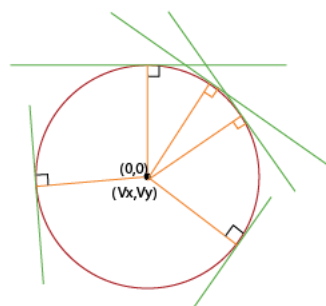
Case : Three tangents



Case : Four tangents



Case : Infinite



Here, we won't be considering **degenerate** cases, i.e when the circles coincide (in this case they have infinitely many common tangents), or one circle lies inside the other (in this case they have no common tangents, or if the circles are tangent, there is one common tangent).

In most cases, two circles have **four** common tangents.

If the circles **are tangent**, then they will have three common tangents, but this can be understood as a degenerate case: as if the two tangents coincided.

Moreover, the algorithm described below will work in the case when one or both circles have zero radius: in this case there will be, respectively, two or one common tangent.

Summing up, we will always look for **four tangents** for all cases except infinite tangents case (The infinite tangents case needs to be handled separately and it is not discussed here). In degenerate cases, some of tangents will coincide, but nevertheless, these cases will also fit into the big picture.

Algorithm

For the sake of simplicity of the algorithm, we will assume, without losing generality, that the center of the first circle has coordinates $(0, 0)$. (If this is not the case, then this can be achieved by simply shifting the whole picture, and after finding a solution, by shifting the obtained straight lines back.)

Denote r_1 and r_2 the radii of the first and second circles, and by (v_x, v_y) the coordinates of the center of the second circle and point v different from origin. (Note: we are not considering the case in which both the circles are same).

To solve the problem, we approach it purely **algebraically**. We need to find all the lines of the form $ax + by + c = 0$ that lie at a distance r_1 from the origin of coordinates, and at a distance r_2 from a point v . In addition, we impose the condition of normalization of the straight line: the sum of the squares of the coefficients and must be equal to one (this is necessary, otherwise the same straight line will correspond to infinitely many representations of the form $ax + by + c = 0$). Total we get such a system of equations for the desired a, b, c :

$$\begin{aligned} a^2 + b^2 &= 1 \\ |a \cdot 0 + b \cdot 0 + c| &= r_1 \\ |a \cdot v_x + b \cdot v_y + c| &= r_2 \end{aligned}$$

To get rid of the modulus, note that there are only four ways to open the modulus in this system. All these methods can be considered by the general case, if we understand the opening of the modulus as the fact that the coefficient on the right-hand side may be multiplied by -1. In other words, we turn to this system:

$$\begin{aligned}
 a^2 + b^2 &= 1 \\
 c &= \pm r_1 \\
 a \cdot v_x + b \cdot v_y + c &= \pm r_2
 \end{aligned}$$

Entering the notation $d_1 = \pm r_1$ and $d_2 = \pm r_2$, we come to the conclusion that the system must have four solutions:

$$\begin{aligned}
 a^2 + b^2 &= 1 \\
 c &= d_1 \\
 a \cdot v_x + b \cdot v_y + c &= d_2
 \end{aligned}$$

The solution of this system is reduced to solving a quadratic equation. We will omit all the cumbersome calculations, and immediately give a ready answer:

$$\begin{aligned}
 a &= \frac{(d_2 - d_1)v_x \pm v_y \sqrt{v_x^2 + v_y^2 - (d_2 - d_1)^2}}{v_x^2 + v_y^2} \\
 b &= \frac{(d_2 - d_1)v_y \pm v_x \sqrt{v_x^2 + v_y^2 - (d_2 - d_1)^2}}{v_x^2 + v_y^2} \\
 c &= d_1
 \end{aligned}$$

Total we got eight solutions instead four. However, it is easy to understand where superfluous decisions arise: in fact, in the latter system, it is enough to take only one solution (for example, the first). In fact, the geometric meaning of what we take $\pm r_1$ and $\pm r_2$ is clear: we are actually sorting out which side of each circle there is a straight line. Therefore, the two methods that arise when solving the latter system are redundant: it is enough to choose one of the two solutions (only, of course, in all four cases, you must choose the same family of solutions).

The last thing that we have not yet considered is **how to shift the straight lines** in the case when the first circle was not originally located at the origin. However, everything is simple here: it follows from the linearity of the equation of a straight line that the value

$a \cdot x_0 + b \cdot y_0$ (where x_0 and y_0 are the coordinates of the original center of the first circle) must be subtracted from the coefficient c .

Implementation

We first describe all the necessary data structures and other auxiliary definitions:

```
struct pt {
    double x, y;

    pt operator- (pt p) {
        pt res = { x-p.x, y-p.y };
        return res;
    }
};

struct circle : pt {
    double r;
};

struct line {
    double a, b, c;
};

const double EPS = 1E-9;

double sqr (double a) {
    return a * a;
}
```

Then the solution itself can be written this way (where the main function for the call is the second; and the first function is an auxiliary):

```
void tangents (pt c, double r1, double r2, vector<line> & ans) {
    double r = r2 - r1;
    double z = sqr(c.x) + sqr(c.y);
    double d = z - sqr(r);
    if (d < -EPS) return;
    d = sqrt (abs (d));
    line l;
    l.a = (c.x * r + c.y * d) / z;
    l.b = (c.y * r - c.x * d) / z;
    l.c = r1;
    ans.push_back (l);
}
```

```
vector<line> tangents (circle a, circle b) {  
    vector<line> ans;  
    for (int i=-1; i<=1; i+=2)  
        for (int j=-1; j<=1; j+=2)  
            tangents (b-a, a.r*i, b.r*j, ans);  
    for (size_t i=0; i<ans.size(); ++i)  
        ans[i].c -= ans[i].a * a.x + ans[i].b * a.y;  
    return ans;  
}
```

Problems

TIMUS 1163 Chapaev

Contributors:

[Prakash Jha](#) (74.05%) [Jakob Kogler](#) (16.03%) [Oleksandr Kulkov](#) (4.58%)
[Yury Semenov](#) (3.05%) [Michael Hayter](#) (2.29%)

Translated

From: e-maxx.ru

Length of the union of segments

Given n segments on a line, each described by a pair of coordinates (a_{i1}, a_{i2}) . We have to find the length of their union.

The following algorithm was proposed by Klee in 1977. It works in $O(n \log n)$ and has been proven to be the asymptotically optimal.

Solution

We store in an array x the endpoints of all the segments sorted by their values. And additionally we store whether it is a left end or a right end of a segment. Now we iterate over the array, keeping a counter c of currently opened segments. Whenever the current element is a left end, we increase this counter, and otherwise we decrease it. To compute the answer, we take the length between the last to x values $x_i - x_{i-1}$, whenever we come to a new coordinate, and there is currently at least one segment is open.

Implementation

```
int length_union(const vector<pair<int, int>> &a) {
    int n = a.size();
    vector<pair<int, bool>> x(n*2);
    for (int i = 0; i < n; i++) {
        x[i*2] = {a[i].first, false};
        x[i*2+1] = {a[i].second, true};
    }

    sort(x.begin(), x.end());

    int result = 0;
```

```
int c = 0;
for (int i = 0; i < n * 2; i++) {
    if (i > 0 && x[i].first > x[i-1].first && c > 0)
        result += x[i].first - x[i-1].first;
    if (x[i].second)
        c--;
    else
        c++;
}
return result;
}
```

Contributors:

[Jakob Kogler](#) (43.75%) [Edward Lemur](#) (35.42%) [Oleksandr Kulkov](#) (12.5%)
[Bhuvnesh Jain](#) (6.25%) [Koustav Chowdhury](#) (2.08%)

Translated

From: e-maxx.ru

Oriented area of a triangle

Given three points p_1 , p_2 and p_3 , calculate an oriented (signed) area of a triangle formed by them. The sign of the area is determined in the following way: imagine you are standing in the plane at point p_1 and are facing p_2 . You go to p_2 and if p_3 is to your right (then we say the three vectors turn "clockwise"), the sign of the area is negative, otherwise it is positive. If the three points are collinear, the area is zero.

Using this signed area, we can both get the regular unsigned area (as the absolute value of the signed area) and determine if the points lie clockwise or counterclockwise in their specified order (which is useful, for example, in convex hull algorithms).

Calculation

We can use the fact that a determinant of a 2×2 matrix is equal to the signed area of a parallelogram spanned by column (or row) vectors of the matrix. This is analog to the definition of the cross product in 2D (see [Basic Geometry](#)). By dividing this area by two we get the area of a triangle that we are interested in. We will use p_1p_2 and p_1p_3 as the column vectors and calculate a 2×2 determinant:

$$2S = \begin{vmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{vmatrix} = (x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)$$

Implementation

```
int signed_area_parallelogram(point2d p1, point2d p2, point2d p3) {
    return cross(p2 - p1, p3 - p1);
}
```

```
double triangle_area(point2d p1, point2d p2, point2d p3) {  
    return abs(signed_area_parallelogram(p1, p2, p3)) / 2.0;  
}  
  
bool clockwise(point2d p1, point2d p2, point2d p3) {  
    return signed_area_parallelogram(p1, p2, p3) < 0;  
}  
  
bool counter_clockwise(point2d p1, point2d p2, point2d p3) {  
    return signed_area_parallelogram(p1, p2, p3) > 0;  
}
```

Translated

From: e-maxx.ru

Finding area of simple polygon in $O(N)$

Let a simple polygon (i.e. without self intersection, not necessarily convex) be given. It is required to calculate its area given its vertices.

Method 1

This is easy to do if we go through all edges and add trapezoid areas bounded by each edge and x-axis. The area needs to be taken with sign so that the extra area will be reduced. Hence, the formula is as follows:

$$A = \sum_{(p,q) \in \text{edges}} \frac{(p_x - q_x) \cdot (p_y + q_y)}{2}$$

Code:

```
double area(const vector<point>& fig) {
    double res = 0;
    for (unsigned i = 0; i < fig.size(); i++) {
        point p = i ? fig[i - 1] : fig.back();
        point q = fig[i];
        res += (p.x - q.x) * (p.y + q.y);
    }
    return fabs(res) / 2;
}
```

Method 2

We can choose a point O arbitrarily, iterate over all edges adding the oriented area of the triangle formed by the edge and point O . Again, due to the sign of area, extra area will be reduced.

This method is better as it can be generalized to more complex cases (such as when some sides are arcs instead of straight lines)

Contributors:

[Pranjal Jain](#) (79.41%) [Jakob Kogler](#) (8.82%) [Oleksandr Kulkov](#) (8.82%) [Anton Älgmyr](#) (2.94%)

Translated

From: e-maxx.ru

Check if point belongs to the convex polygon in $O(\log N)$

Consider the following problem: you are given a convex polygon with integer vertices and a lot of queries. Each query is a point, for which we should determine whether it lies inside or on the boundary of the polygon or not. Suppose the polygon is ordered counter-clockwise. We will answer each query in $O(\log n)$ online.

Algorithm

Let's pick the point with the smallest x-coordinate. If there are several of them, we pick the one with the smallest y-coordinate. Let's denote it as p_0 . Now all other points $p_1, \dots, p_n$ of the polygon are ordered by their polar angle from the chosen point (because the polygon is ordered counter-clockwise).

If the point belongs to the polygon, it belongs to some triangle p_0, p_i, p_{i+1} (maybe more than one if it lies on the boundary of triangles). Consider the triangle p_0, p_i, p_{i+1} such that p belongs to this triangle and i is maximum among all such triangles.

There is one special case. p lies on the segment (p_0, p_n) . This case we will check separately. Otherwise all points p_j with $j \leq i$ are counter-clockwise from p with respect to p_0 , and all other points are not counter-clockwise from p . This means that we can apply binary search for the point p_i , such that p_i is not counter-clockwise from p with respect to p_0 , and i is maximum among all such points. And afterwards we check if the points is actually in the determined triangle.

The sign of $(a - c) \times (b - c)$ will tell us, if the point a is clockwise or counter-clockwise from the point b with respect to the point c . If $(a - c) \times (b - c) > 0$, then the point a is to the right of the vector going from c to b , which means clockwise from

b with respect to c . And if $(a - c) \times (b - c) < 0$, then the point is to the left, or counter clockwise. And it is exactly on the line between the points b and c .

Back to the algorithm: Consider a query point p . Firstly, we must check if the point lies between p_1 and p_n . Otherwise we already know that it cannot be part of the polygon. This can be done by checking if the cross product $(p_1 - p_0) \times (p - p_0)$ is zero or has the same sign with $(p_1 - p_0) \times (p_n - p_0)$, and $(p_n - p_0) \times (p - p_0)$ is zero or has the same sign with $(p_n - p_0) \times (p_1 - p_0)$. Then we handle the special case in which p is part of the line (p_0, p_1) . And then we can binary search the last point from $p_1, \dots, p_n$ which is not counter-clockwise from p with respect to p_0 . For a single point p_i this condition can be checked by checking that $(p_i - p_0) \times (p - p_0) \leq 0$. After we found such a point p_i , we must test if p lies inside the triangle p_0, p_i, p_{i+1} . To test if it belongs to the triangle, we may simply check that

$|(p_i - p_0) \times (p_{i+1} - p_0)| = |(p_0 - p) \times (p_i - p)| + |(p_i - p) \times (p_{i+1} - p)| + |(p_{i+1} - p) \times (p_0 - p)|$. This checks if the area of the triangle p_0, p_i, p_{i+1} has to exact same size as the sum of the sizes of the triangle p_0, p_i, p , the triangle p_0, p, p_{i+1} and the triangle p_i, p_{i+1}, p . If p is outside, then the sum of those three triangle will be bigger than the size of the triangle. If it is inside, then it will be equal.

Implementation

The function `prepare` will make sure that the lexicographical smallest point (smallest x value, and in ties smallest y value) will be p_0 , and computes the vectors $p_i - p_0$. Afterwards the function `pointInConvexPolygon` computes the result of a query. We additionally remember the point p_0 and translate all queried points with it in order compute the correct distance, as vectors don't have an initial point. By translating the query points we can assume that all vectors start at the origin $(0, 0)$, and simplify the computations for distances and lengths.

```
struct pt {
    long long x, y;
    pt() {}
    pt(long long _x, long long _y) : x(_x), y(_y) {}
    pt operator+(const pt &p) const { return pt(x + p.x, y + p.y); }
    pt operator-(const pt &p) const { return pt(x - p.x, y - p.y); }
    long long cross(const pt &p) const { return x * p.y - y * p.x; }
    long long dot(const pt &p) const { return x * p.x + y * p.y; }
    long long cross(const pt &a, const pt &b) const { return (a -
```

```

*this).cross(b - *this); }
    long long dot(const pt &a, const pt &b) const { return (a -
*this).dot(b - *this); }
    long long sqrLen() const { return this->dot(*this); }
};

bool lexComp(const pt &l, const pt &r) {
    return l.x < r.x || (l.x == r.x && l.y < r.y);
}

int sgn(long long val) { return val > 0 ? 1 : (val == 0 ? 0 : -1); }

vector<pt> seq;
pt translation;
int n;

bool pointInTriangle(pt a, pt b, pt c, pt point) {
    long long s1 = abs(a.cross(b, c));
    long long s2 = abs(point.cross(a, b)) + abs(point.cross(b, c)) +
abs(point.cross(c, a));
    return s1 == s2;
}

void prepare(vector<pt> &points) {
    n = points.size();
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (lexComp(points[i], points[pos]))
            pos = i;
    }
    rotate(points.begin(), points.begin() + pos, points.end());

    n--;
    seq.resize(n);
    for (int i = 0; i < n; i++)
        seq[i] = points[i + 1] - points[0];
    translation = points[0];
}

bool pointInConvexPolygon(pt point) {
    point = point - translation;
    if (seq[0].cross(point) != 0 &&
        sgn(seq[0].cross(point)) != sgn(seq[0].cross(seq[n - 1])))
        return false;
    if (seq[n - 1].cross(point) != 0 &&
        sgn(seq[n - 1].cross(point)) != sgn(seq[n -
1].cross(seq[0])))
        return false;

    if (seq[0].cross(point) == 0)

```

```
        return seq[0].sqrLen() >= point.sqrLen();

    int l = 0, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        int pos = mid;
        if (seq[pos].cross(point) >= 0)
            l = mid;
        else
            r = mid;
    }
    int pos = l;
    return pointInTriangle(seq[pos], seq[pos + 1], pt(0, 0), point);
}
```

Problems

- [SGU253 Theodore Roosevelt](#)
- [Codeforces 55E Very simple problem](#)
- [Codeforces 166B Polygons](#)

Contributors:

[Yury Semenov](#) (79.2%) [Jakob Kogler](#) (11.2%) [Oleksandr Kulkov](#) (4.8%)
[AYUSH KUMAR TIWARI](#) (2.4%) [dallasyan](#) (1.6%) [Felipe Noronha](#) (0.8%)

Original

Minkowski sum of convex polygons

Definition

Consider two sets A and B of points on a plane. Minkowski sum $A + B$ is defined as $\{a + b \mid a \in A, b \in B\}$. Here we will consider the case when A and B consist of convex polygons P and Q with their interiors. Throughout this article we will identify polygons with ordered sequences of their vertices, so that notation like $|P|$ or P_i makes sense. It turns out that the sum of convex polygons P and Q is a convex polygon with at most $|P| + |Q|$ vertices.

Algorithm

Here we consider the polygons to be cyclically enumerated, i. e. $P_{|P|} = P_0$, $Q_{|Q|} = Q_0$ and so on.

Since the size of the sum is linear in terms of the sizes of initial polygons, we should aim at finding a linear-time algorithm. Suppose that both polygons are ordered counter-

clockwise. Consider sequences of edges $\{\overrightarrow{P_i P_{i+1}}\}$ and $\{\overrightarrow{Q_j Q_{j+1}}\}$ ordered by polar angle. We claim that the sequence of edges of $P + Q$ can be obtained by merging these two sequences preserving polar angle order and replacing consecutive co-directed vectors with their sum. Straightforward usage of this idea results in a linear-time algorithm, however, restoring the vertices of $P + Q$ from the sequence of sides requires repeated addition of vectors, which may introduce unwanted precision issues if we're working with floating-point coordinates, so we will describe a slight modification of this idea.

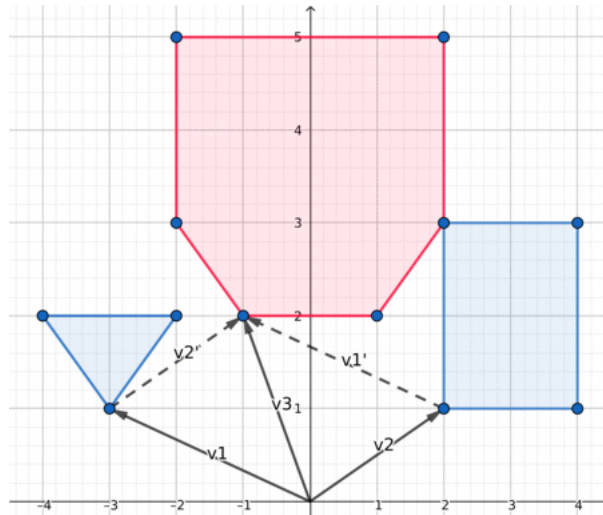
Firstly we should reorder the vertices in such a way that the first vertex of each polygon has the lowest y-coordinate (in case of several such vertices pick the one with the smallest x-coordinate). After that the sides of both polygons will become sorted by

polar angle, so there is no need to sort them manually. Now we create two pointers i (pointing to a vertex of P) and j (pointing to a vertex of Q), both initially set to 0. We repeat the following steps while $i < |P|$ or $j < |Q|$.

1. Append $P_i + Q_j$ to $P + Q$.
2. Compare polar angles of $\overrightarrow{P_i P_{i+1}}$ and $\overrightarrow{Q_j Q_{j+1}}$.
3. Increment the pointer which corresponds to the smallest angle (if the angles are equal, increment both).

Visualization

Here is a nice visualization, which may help you understand what is going on.



Distance between two polygons

One of the most common applications of Minkowski sum is computing the distance between two convex polygons (or simply checking whether they intersect). The distance between two convex polygons P and Q is defined as $\min_{a \in P, b \in Q} \|a - b\|$. One can note that the distance is always attained between two vertices or a vertex and an edge, so we can easily find the distance in $O(|P| |Q|)$. However, with clever usage of Minkowski sum we can reduce the complexity to $O(|P| + |Q|)$.

If we reflect Q through the point $(0, 0)$ obtaining polygon $-Q$, the problem boils down to finding the smallest distance between a point in $P + (-Q)$ and $(0, 0)$. We can find that distance in linear time using the following idea. If $(0, 0)$ is inside or on the boundary of polygon, the distance is 0, otherwise the distance is attained between $(0, 0)$ and some vertex or edge of the polygon. Since Minkowski sum can be computed in linear time, we obtain a linear-time algorithm for finding the distance between two convex polygons.

Implementation

Below is the implementation of Minkowski sum for polygons with integer points. Note that in this case all computations can be done in integers since instead of computing polar angles and directly comparing them we can look at the sign of cross product of two vectors.

```
struct pt{
    long long x, y;
    pt operator + (const pt & p) const {
        return pt{x + p.x, y + p.y};
    }
    pt operator - (const pt & p) const {
        return pt{x - p.x, y - p.y};
    }
    long long cross(const pt & p) const {
        return x * p.y - y * p.x;
    }
};

void reorder_polygon(vector<pt> & P){
    size_t pos = 0;
    for(size_t i = 1; i < P.size(); i++){
        if(P[i].y < P[pos].y || (P[i].y == P[pos].y && P[i].x <
P[pos].x))
            pos = i;
    }
    rotate(P.begin(), P.begin() + pos, P.end());
}

vector<pt> minkowski(vector<pt> P, vector<pt> Q){
    // the first vertex must be the lowest
    reorder_polygon(P);
    reorder_polygon(Q);
    // we must ensure cyclic indexing
```

```

P.push_back(P[0]);
P.push_back(P[1]);
Q.push_back(Q[0]);
Q.push_back(Q[1]);
// main part
vector<pt> result;
size_t i = 0, j = 0;
while(i < P.size() - 2 || j < Q.size() - 2){
    result.push_back(P[i] + Q[j]);
    auto cross = (P[i + 1] - P[i]).cross(Q[j + 1] - Q[j]);
    if(cross >= 0 && i < P.size() - 2)
        ++i;
    if(cross <= 0 && j < Q.size() - 2)
        ++j;
}
return result;
}

```

Problems

- [Codeforces 87E Mogohu-Rea Idol](#)
- [Codeforces 1195F Geometers Anonymous Club](#)
- [TIMUS 1894 Non-Flying Weather](#)

Contributors:

[Yury Semenov](#) (86.96%)
 [Oleksandr Kulkov](#) (4.35%)
 [Bergur Snorrason](#) (2.61%)
[Michael Hayter](#) (2.61%)
 [Mert Koksal](#) (1.74%)
 [Jakob Kogler](#) (0.87%)
 [samanyu](#) (0.87%)

Translated

From: e-maxx.ru

Pick's Theorem

A polygon without self-intersections is called lattice if all its vertices have integer coordinates in some 2D grid. Pick's theorem provides a way to compute the area of this polygon through the number of vertices that are lying on the boundary and the number of vertices that lie strictly inside the polygon.

Formula

Given a certain lattice polygon with non-zero area.

We denote its area by S , the number of points with integer coordinates lying strictly inside the polygon by I and the number of points lying on polygon sides by B .

Then, the **Pick's formula** states:

$$S = I + \frac{B}{2} - 1$$

In particular, if the values of I and B for a polygon are given, the area can be calculated in $O(1)$ without even knowing the vertices.

This formula was discovered and proven by Austrian mathematician Georg Alexander Pick in 1899.

Proof

The proof is carried out in many stages: from simple polygons to arbitrary ones:

- A single square: $S = 1, I = 0, B = 4$, which satisfies the formula.

- An arbitrary non-degenerate rectangle with sides parallel to coordinate axes:
Assume a and b be the length of the sides of rectangle. Then,
 $S = ab, I = (a - 1)(b - 1), B = 2(a + b)$. On substituting, we see that formula is true.
- A right angle with legs parallel to the axes: To prove this, note that any such triangle can be obtained by cutting off a rectangle by a diagonal. Denoting the number of integral points lying on diagonal by c , it can be shown that Pick's formula holds for this triangle regardless of c .
- An arbitrary triangle: Note that any such triangle can be turned into a rectangle by attaching it to sides of right-angled triangles with legs parallel to the axes (you will not need more than 3 such triangles). From here, we can get correct formula for any triangle.
- An arbitrary polygon: To prove this, triangulate it, ie, divide into triangles with integral coordinates. Further, it is possible to prove that Pick's theorem retains its validity when a polygon is added to a triangle. Thus, we have proven Pick's formula for arbitrary polygon.

Generalization to higher dimensions

Unfortunately, this simple and beautiful formula cannot be generalized to higher dimensions.

John Reeve demonstrated this by proposing a tetrahedron (**Reeve tetrahedron**) with following vertices in 1957:

$$A = (0, 0, 0), B = (1, 0, 0), C = (0, 1, 0), D = (1, 1, k),$$

where k can be any natural number. Then for any k , the tetrahedron $ABCD$ does not contain integer point inside it and has only 4 points on its borders, A, B, C, D . Thus, the volume and surface area may vary in spite of unchanged number of points within and on boundary. Therefore, Pick's theorem doesn't allow generalizations.

However, higher dimensions still has a generalization using **Ehrhart polynomials** but they are quite complex and depends not only on points inside but also on the boundary of polytype.

Extra Resources

A few simple examples and a simple proof of Pick's theorem can be found [here](#).

Contributors:

[Pranjal Jain](#) (81.82%) [Oleksandr Kulkov](#) (10.91%) [Arafat](#) (5.45%) [princezna](#) (1.82%)

Original

Lattice points inside non-lattice polygon

For lattice polygons there is Pick's formula to enumerate the lattice points inside the polygon. What about polygons with arbitrary vertices?

Let's process each of the polygon's edges individually, and after that we may sum up the amounts of lattice points under each edge considering its orientations to choose a sign (like in calculating the area of a polygon using trapezoids).

First of all we should note that if current edge has endpoints in $A = (x_1; y_1)$ and $B = (x_2; y_2)$ then it can be represented as a linear function:

$$y = y_1 + (y_2 - y_1) \cdot \frac{x - x_1}{x_2 - x_1} = \left(\frac{y_2 - y_1}{x_2 - x_1} \right) \cdot x + \left(\frac{y_1 x_2 - x_1 y_2}{x_2 - x_1} \right)$$

$$y = k \cdot x + b, \quad k = \frac{y_2 - y_1}{x_2 - x_1}, \quad b = \frac{y_1 x_2 - x_1 y_2}{x_2 - x_1}$$

Now we will perform a substitution $x = x' + \lfloor x_1 \rfloor$ so that $b' = b + k \cdot \lfloor x_1 \rfloor$. This allows us to work with $x'_1 = 0$ and $x'_2 = x_2 - \lfloor x_1 \rfloor$. Let's denote $n = \lfloor x'_2 \rfloor$.

We will not sum up points at $x = n$ and on $y = 0$ for the integrity of the algorithm.

They may be added manually afterwards. Thus we have to sum up $\sum_{x'=0}^{n-1} \lfloor k' \cdot x' + b' \rfloor$.

We also assume that $k' \geq 0$ and $b' \geq 0$. Otherwise one should substitute $x' = -t$ and add $\lceil |b'| \rceil$ to b' .

Let's discuss how we can evaluate a sum $\sum_{x=0}^{n-1} \lfloor k \cdot x + b \rfloor$. We have two cases:

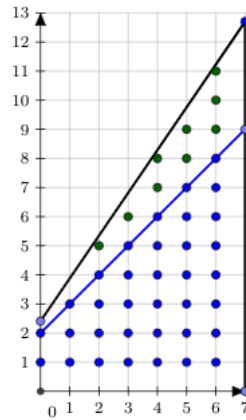
- $k \geq 1$ or $b \geq 1$.

Then we should start with summing up points below $y = \lfloor k \rfloor \cdot x + \lfloor b \rfloor$. Their amount equals to

$$\sum_{x=0}^{n-1} \lfloor k \rfloor \cdot x + \lfloor b \rfloor = \frac{(\lfloor k \rfloor)(n-1) + 2\lfloor b \rfloor)n}{2}.$$

Now we are interested only in points $(x; y)$ such that

$\lfloor k \rfloor \cdot x + \lfloor b \rfloor < y \leq k \cdot x + b$. This amount is the same as the number of points such that $0 < y \leq (k - \lfloor k \rfloor) \cdot x + (b - \lfloor b \rfloor)$. So we reduced our problem to $k' = k - \lfloor k \rfloor$, $b' = b - \lfloor b \rfloor$ and both k' and b' less than 1 now. Here is a picture, we just summed up blue points and subtracted the blue linear function from the black one to reduce problem to smaller values for k and b :

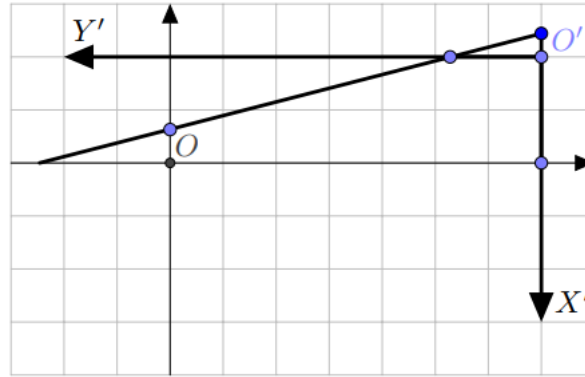


- $k < 1$ and $b < 1$.

If $\lfloor k \cdot n + b \rfloor$ equals 0, we can safely return 0. If this is not the case, we can say that there are no lattice points such that $x < 0$ and $0 < y \leq k \cdot x + b$. That means that we will have the same answer if we consider new reference system in which $O' = (n; \lfloor k \cdot n + b \rfloor)$, axis x' is directed down and axis y' is directed to the left. For this reference system we are interested in lattice points on the set

$$\{ (x; y) \mid 0 \leq x < \lfloor k \cdot n + b \rfloor, 0 < y \leq \frac{x + (k \cdot n + b) - \lfloor k \cdot n + b \rfloor}{k} \}$$

which returns us back to the case $k > 1$. You can see new reference point O' and axes X' and Y' in the picture below:



As you see, in new reference system linear function will have coefficient $\frac{1}{k}$ and its zero will be in the point $[k \cdot n + b] - (k \cdot n + b)$ which makes formula above correct.

Complexity analysis

We have to count at most $\frac{(k(n-1) + 2b)n}{2}$ points. Among them we will count $\frac{[k](n-1) + 2[b]}{2}$ on the very first step. We may consider that b is negligibly small because we can start with making it less than 1. In that case we can say that we count about $\frac{[k]}{k} \geq \frac{1}{2}$ of all points. Thus we will finish in $O(\log n)$ steps.

Implementation

Here is simple function which calculates number of integer points $(x; y)$ such for $0 \leq x < n$ and $0 < y \leq [kx + b]$:

```
int count_lattices(Fraction k, Fraction b, long long n) {
    auto fk = k.floor();
    auto fb = b.floor();
    auto cnt = 0LL;
    if (k >= 1 || b >= 1) {
        cnt += (fk * (n - 1) + 2 * fb) * n / 2;
        k -= fk;
        b -= fb;
    }
    auto t = k * n + b;
```

```
auto ft = t.floor();
if (ft >= 1) {
    cnt += count_lattices(1 / k, (t - t.floor()) / k, t.floor());
}
return cnt;
}
```

Here `Fraction` is some class handling rational numbers. On practice it seems that if all denominators and numerators are at most C by absolute value then in the recursive calls they will be at most C^2 if you keep dividing numerators and denominators by their greatest common divisor. Given this assumption we can say that one may use doubles and require accuracy of ε^2 where ε is accuracy with which k and b are given. That means that in floor one should consider numbers as integer if they differs at most by ε^2 from an integer.

Contributors:

[Alexander Kulkov](#) (78.95%) [Jakob Kogler](#) (9.47%) [Michael Hayter](#) (6.32%)
[Oleksandr Kulkov](#) (5.26%)

Translated

From: e-maxx.ru

Convex Hull construction

In this article we will discuss the problem of constructing a convex hull from a set of points.

Consider N points given on a plane, and the objective is to generate a convex hull, i.e. the smallest convex polygon that contains all the given points.

We will see the **Graham's scan** algorithm published in 1972 by Graham, and also the **Monotone chain** algorithm published in 1979 by Andrew. Both are $O(N \log N)$, and are asymptotically optimal (as it is proven that there is no algorithm asymptotically better), with the exception of a few problems where parallel or online processing is involved.

Graham's scan Algorithm

The algorithm first finds the bottom-most point P_0 . If there are multiple points with the same Y coordinate, the one with the smaller X coordinate is considered. This step takes $O(N)$ time.

Next, all the other points are sorted by polar angle in clockwise order. If the polar angle between two or more points is the same, the tie should be broken by distance from P_0 , in increasing order.

Then we iterate through each point one by one, and make sure that the current point and the two before it make a clockwise turn, otherwise the previous point is discarded, since it would make a non-convex shape. Checking for clockwise or anticlockwise nature can be done by checking the [orientation](#).

We use a stack to store the points, and once we reach the original point P_0 , the algorithm is done and we return the stack containing all the points of the convex hull in clockwise order.

If you need to include the collinear points while doing a Graham scan, you need another step after sorting. You need to get the points that have the biggest polar distance from P_0 (these should be at the end of the sorted vector) and are collinear. The points in this line should be reversed so that we can output all the collinear points, otherwise the algorithm would get the nearest point in this line and bail. This step shouldn't be included in the non-collinear version of the algorithm, otherwise you wouldn't get the smallest convex hull.

Implementation

```
struct pt {
    double x, y;
    bool operator == (pt const& t) const {
        return x == t.x && y == t.y;
    }
};

int orientation(pt a, pt b, pt c) {
    double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
    if (v < 0) return -1; // clockwise
    if (v > 0) return +1; // counter-clockwise
    return 0;
}

bool cw(pt a, pt b, pt c, bool include_collinear) {
    int o = orientation(a, b, c);
    return o < 0 || (include_collinear && o == 0);
}

bool collinear(pt a, pt b, pt c) { return orientation(a, b, c) == 0; }

void convex_hull(vector<pt>& a, bool include_collinear = false) {
    pt p0 = *min_element(a.begin(), a.end(), [](pt a, pt b) {
        return make_pair(a.y, a.x) < make_pair(b.y, b.x);
    });
    sort(a.begin(), a.end(), [&p0](const pt& a, const pt& b) {
        int o = orientation(p0, a, b);
        if (o == 0)
            return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
                < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
        return o < 0;
    });
    if (include_collinear) {
        int i = (int)a.size()-1;
        while (i >= 0 && collinear(p0, a[i], a.back())) i--;
        reverse(a.begin()+i+1, a.end());
    }
}
```

```

vector<pt> st;
for (int i = 0; i < (int)a.size(); i++) {
    while (st.size() > 1 && !cw(st[st.size()-2], st.back(), a[i],
include_collinear))
        st.pop_back();
    st.push_back(a[i]);
}

if (include_collinear == false && st.size() == 2 && st[0] == st[1])
    st.pop_back();

a = st;
}

```

Monotone chain Algorithm

The algorithm first finds the leftmost and rightmost points A and B. In the event multiple such points exist, the lowest among the left (lowest Y-coordinate) is taken as A, and the highest among the right (highest Y-coordinate) is taken as B. Clearly, A and B must both belong to the convex hull as they are the farthest away and they cannot be contained by any line formed by a pair among the given points.

Now, draw a line through AB. This divides all the other points into two sets, S1 and S2, where S1 contains all the points above the line connecting A and B, and S2 contains all the points below the line joining A and B. The points that lie on the line joining A and B may belong to either set. The points A and B belong to both sets. Now the algorithm constructs the upper set S1 and the lower set S2 and then combines them to obtain the answer.

To get the upper set, we sort all points by the x-coordinate. For each point we check if either - the current point is the last point, (which we defined as B), or if the orientation between the line between A and the current point and the line between the current point and B is clockwise. In those cases the current point belongs to the upper set S1. Checking for clockwise or anticlockwise nature can be done by checking the [orientation](#).

If the given point belongs to the upper set, we check the angle made by the line connecting the second last point and the last point in the upper convex hull, with the line connecting the last point in the upper convex hull and the current point. If the angle is not clockwise, we remove the most recent point added to the upper convex hull as the

current point will be able to contain the previous point once it is added to the convex hull.

The same logic applies for the lower set S2. If either - the current point is B, or the orientation of the lines, formed by A and the current point and the current point and B, is counterclockwise - then it belongs to S2.

If the given point belongs to the lower set, we act similarly as for a point on the upper set except we check for a counterclockwise orientation instead of a clockwise orientation. Thus, if the angle made by the line connecting the second last point and the last point in the lower convex hull, with the line connecting the last point in the lower convex hull and the current point is not counterclockwise, we remove the most recent point added to the lower convex hull as the current point will be able to contain the previous point once added to the hull.

The final convex hull is obtained from the union of the upper and lower convex hull, forming a clockwise hull, and the implementation is as follows.

If you need collinear points, you just need to check for them in the clockwise/counterclockwise routines. However, this allows for a degenerate case where all the input points are collinear in a single line, and the algorithm would output repeated points. To solve this, we check whether the upper hull contains all the points, and if it does, we just return the points in reverse, as that is what Graham's implementation would return in this case.

Implementation

```
struct pt {
    double x, y;
};

int orientation(pt a, pt b, pt c) {
    double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
    if (v < 0) return -1; // clockwise
    if (v > 0) return +1; // counter-clockwise
    return 0;
}

bool cw(pt a, pt b, pt c, bool include_collinear) {
    int o = orientation(a, b, c);
    return o < 0 || (include_collinear && o == 0);
}
```

```

bool ccw(pt a, pt b, pt c, bool include_collinear) {
    int o = orientation(a, b, c);
    return o > 0 || (include_collinear && o == 0);
}

void convex_hull(vector<pt>& a, bool include_collinear = false) {
    if (a.size() == 1)
        return;

    sort(a.begin(), a.end(), [](pt a, pt b) {
        return make_pair(a.x, a.y) < make_pair(b.x, b.y);
    });
    pt p1 = a[0], p2 = a.back();
    vector<pt> up, down;
    up.push_back(p1);
    down.push_back(p1);
    for (int i = 1; i < (int)a.size(); i++) {
        if (i == a.size() - 1 || cw(p1, a[i], p2, include_collinear)) {
            while (up.size() >= 2 && !cw(up[up.size()-2],
up[up.size()-1], a[i], include_collinear))
                up.pop_back();
            up.push_back(a[i]);
        }
        if (i == a.size() - 1 || ccw(p1, a[i], p2, include_collinear)) {
            while (down.size() >= 2 && !ccw(down[down.size()-2],
down[down.size()-1], a[i], include_collinear))
                down.pop_back();
            down.push_back(a[i]);
        }
    }

    if (include_collinear && up.size() == a.size()) {
        reverse(a.begin(), a.end());
        return;
    }
    a.clear();
    for (int i = 0; i < (int)up.size(); i++)
        a.push_back(up[i]);
    for (int i = down.size() - 2; i > 0; i--)
        a.push_back(down[i]);
}

```

Original

Convex hull trick and Li Chao tree

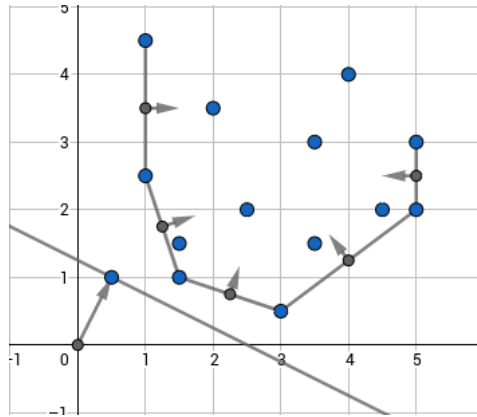
Consider the following problem. There are n cities. You want to travel from city 1 to city n by car. To do this you have to buy some gasoline. It is known that a liter of gasoline costs cost_k in the k^{th} city. Initially your fuel tank is empty and you spend one liter of gasoline per kilometer. Cities are located on the same line in ascending order with k^{th} city having coordinate x_k . Also you have to pay toll_k to enter k^{th} city. Your task is to make the trip with minimum possible cost. It's obvious that the solution can be calculated via dynamic programming:

$$\text{dp}_i = \text{toll}_i + \min_{j < i} (\text{cost}_j \cdot (x_i - x_j) + \text{dp}_j)$$

Naive approach will give you $O(n^2)$ complexity which can be improved to $O(n \log n)$ or $O(n \log[C\epsilon^{-1}])$ where C is largest possible $|x_i|$ and ϵ is precision with which x_i is considered ($\epsilon = 1$ for integers which is usually the case). To do this one should note that the problem can be reduced to adding linear functions $k \cdot x + b$ to the set and finding minimum value of the functions in some particular point x . There are two main approaches one can use here.

Convex hull trick

The idea of this approach is to maintain a lower convex hull of linear functions. Actually it would be a bit more convenient to consider them not as linear functions, but as points $(k; b)$ on the plane such that we will have to find the point which has the least dot product with a given point $(x; 1)$, that is, for this point $kx + b$ is minimized which is the same as initial problem. Such minimum will necessarily be on lower convex envelope of these points as can be seen below:



One has to keep points on the convex hull and normal vectors of the hull's edges. When you have a $(x; 1)$ query you'll have to find the normal vector closest to it in terms of angles between them, then the optimum linear function will correspond to one of its endpoints. To see that, one should note that points having a constant dot product with $(x; 1)$ lie on a line which is orthogonal to $(x; 1)$, so the optimum linear function will be the one in which tangent to convex hull which is collinear with normal to $(x; 1)$ touches the hull. This point is the one such that normals of edges lying to the left and to the right of it are headed in different sides of $(x; 1)$.

This approach is useful when queries of adding linear functions are monotone in terms of k or if we work offline, i.e. we may firstly add all linear functions and answer queries afterwards. So we cannot solve the cities/gasoline problems using this way. That would require handling online queries. When it comes to deal with online queries however, things will go tough and one will have to use some kind of set data structure to implement a proper convex hull. Online approach will however not be considered in this article due to its hardness and because second approach (which is Li Chao tree) allows to solve the problem way more simply. Worth mentioning that one can still use this approach online without complications by square-root-decomposition. That is, rebuild convex hull from scratch each $\sqrt{n}$ new lines.

To implement this approach one should begin with some geometric utility functions, here we suggest to use the C++ complex number type.

```
typedef int ftype;
typedef complex<ftype> point;
#define x real
#define y imag
```

```

ftype dot(point a, point b) {
    return (conj(a) * b).x();
}

ftype cross(point a, point b) {
    return (conj(a) * b).y();
}

```

Here we will assume that when linear functions are added, their k only increases and we want to find minimum values. We will keep points in vector `hull` and normal vectors in vector `vecs`. When we add a new point, we have to look at the angle formed between last edge in convex hull and vector from last point in convex hull to new point. This angle has to be directed counter-clockwise, that is the dot product of the last normal vector in the hull (directed inside hull) and the vector from the last point to the new one has to be non-negative. As long as this isn't true, we should erase the last point in the convex hull alongside with the corresponding edge.

```

vector<point> hull, vecs;

void add_line(ftype k, ftype b) {
    point nw = {k, b};
    while(!vecs.empty() && dot(vecs.back(), nw - hull.back()) < 0) {
        hull.pop_back();
        vecs.pop_back();
    }
    if(!hull.empty()) {
        vecs.push_back(1i * (nw - hull.back()));
    }
    hull.push_back(nw);
}

```

Now to get the minimum value in some point we will find the first normal vector in the convex hull that is directed counter-clockwise from $(x; 1)$. The left endpoint of such edge will be the answer. To check if vector a is not directed counter-clockwise of vector b , we should check if their cross product $[a, b]$ is positive.

```

int get(ftype x) {
    point query = {x, 1};
    auto it = lower_bound(vecs.begin(), vecs.end(), query, [](point a, point b) {
        return cross(a, b) > 0;
    });
}

```

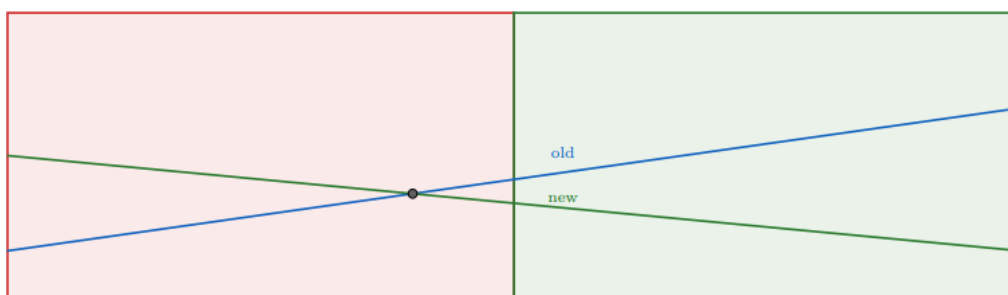
```
    return dot(query, hull[it - vecs.begin()]);
}
```

Li Chao tree

Assume you're given a set of functions such that each two can intersect at most once. Let's keep in each vertex of a segment tree some function in such way, that if we go from root to the leaf it will be guaranteed that one of the functions we met on the path will be the one giving the minimum value in that leaf. Let's see how to construct it.

Assume we're in some vertex corresponding to half-segment $[l, r)$ and the function f_{old} is kept there and we add the function f_{new} . Then the intersection point will be either in $[l; m)$ or in $[m; r)$ where $m = \lfloor \frac{l+r}{2} \rfloor$. We can efficiently find that out by comparing the values of the functions in points l and m . If the dominating function changes, then it is in $[l; m)$ otherwise it is in $[m; r)$. Now for the half of the segment with no intersection we will pick the lower function and write it in the current vertex. You can see that it will always be the one which is lower in point m . After that we recursively go to the other half of the segment with the function which was the upper one. As you can see this will keep correctness on the first half of segment and in the other one correctness will be maintained during the recursive call. Thus we can add functions and check the minimum value in the point in $O(\log[C\varepsilon^{-1}])$.

Here is the illustration of what is going on in the vertex when we add new function:



Let's go to implementation now. Once again we will use complex numbers to keep linear functions.

```
typedef long long ftype;
typedef complex<ftype> point;
#define x real
```

```

#define y imag

ftype dot(point a, point b) {
    return (conj(a) * b).x();
}

ftype f(point a, ftype x) {
    return dot(a, {x, 1});
}

```

We will keep functions in the array `line` and use binary indexing of the segment tree. If you want to use it on large numbers or doubles, you should use a dynamic segment tree. The segment tree should be initialized with default values, e.g. with lines $0x + \infty$.

```

const int maxn = 2e5;

point line[4 * maxn];

void add_line(point nw, int v = 1, int l = 0, int r = maxn) {
    int m = (l + r) / 2;
    bool lef = f(nw, l) < f(line[v], l);
    bool mid = f(nw, m) < f(line[v], m);
    if(mid) {
        swap(line[v], nw);
    }
    if(r - l == 1) {
        return;
    } else if(lef != mid) {
        add_line(nw, 2 * v, l, m);
    } else {
        add_line(nw, 2 * v + 1, m, r);
    }
}

```

Now to get the minimum in some point x we simply choose the minimum value along the path to the point.

```

ftype get(int x, int v = 1, int l = 0, int r = maxn) {
    int m = (l + r) / 2;
    if(r - l == 1) {
        return f(line[v], x);
    } else if(x < m) {
        return min(f(line[v], x), get(x, 2 * v, l, m));
    } else {
        return min(f(line[v], x), get(x, 2 * v + 1, m, r));
    }
}

```

```
}  
}
```

Problems

- [Codebreaker - TROUBLES](#) (simple application of Convex Hull Trick after a couple of observations)
- [CS Academy - Squared Ends](#)
- [Codeforces - Escape Through Leaf](#)
- [CodeChef - Polynomials](#)
- [Codeforces - Kalila and Dimna in the Logging Industry](#)
- [Codeforces - Product Sum](#)
- [Codeforces - Bear and Bowling 4](#)
- [APIO 2010 - Commando](#)

Contributors:

Alexander Kulkov (67.9%)	wiktorkuchta (12.35%)	Jakob Kogler (8.02%)
Oleksandr Kulkov (4.32%)	Michael Hayter (3.7%)	Max (1.85%)
Tanmay Sinha (0.62%)	Sabit Zahin (1.23%)	

Translated

From: e-maxx.ru

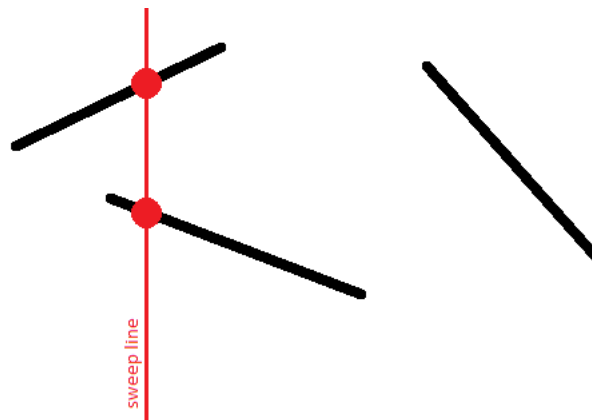
Search for a pair of intersecting segments

Given n line segments on the plane. It is required to check whether at least two of them intersect with each other. If the answer is yes, then print this pair of intersecting segments; it is enough to choose any of them among several answers.

The naive solution algorithm is to iterate over all pairs of segments in $O(n^2)$ and check for each pair whether they intersect or not. This article describes an algorithm with the runtime time $O(n \log n)$, which is based on the **sweep line algorithm**.

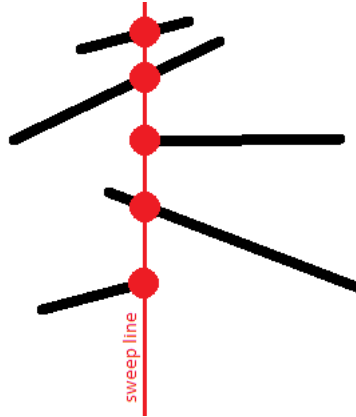
Algorithm

Let's draw a vertical line $x = -\infty$ mentally and start moving this line to the right. In the course of its movement, this line will meet with segments, and at each time a segment intersect with our line it intersects in exactly one point (we will assume that there are no vertical segments).

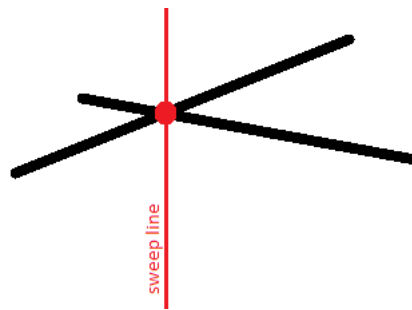


Thus, for each segment, at some point in time, its point will appear on the sweep line, then with the movement of the line, this point will move, and finally, at some point, the segment will disappear from the line.

We are interested in the **relative order of the segments** along the vertical. Namely, we will store a list of segments crossing the sweep line at a given time, where the segments will be sorted by their y-coordinate on the sweep line.



This order is interesting because intersecting segments will have the same y-coordinate at least at one time:



We formulate key statements:

- To find an intersecting pair, it is sufficient to consider **only adjacent segments** at each fixed position of the sweep line.
- It is enough to consider the sweep line not in all possible real positions $(-\infty \dots +\infty)$, but **only in those positions when new segments appear or old ones disappear**. In other words, it is enough to limit yourself only to the positions equal to the abscissas of the end points of the segments.
- When a new line segment appears, it is enough to **insert** it to the desired location in the list obtained for the previous sweep line. We should only check for the intersection of the **added segment with its immediate neighbors in the list above and below**.

- If the segment disappears, it is enough to **remove** it from the current list. After that, it is necessary **check for the intersection of the upper and lower neighbors in the list**.
- Other changes in the sequence of segments in the list, except for those described, do not exist. No other intersection checks are required.

To understand the truth of these statements, the following remarks are sufficient:

- Two disjoint segments never change their **relative order**.
In fact, if one segment was first higher than the other, and then became lower, then between these two moments there was an intersection of these two segments.
- Two non-intersecting segments also cannot have the same y-coordinates.
- From this it follows that at the moment of the segment appearance we can find the position for this segment in the queue, and we will not have to rearrange this segment in the queue any more: **its order relative to other segments in the queue will not change**.
- Two intersecting segments at the moment of their intersection point will be neighbors of each other in the queue.
- Therefore, for finding pairs of intersecting line segments is sufficient to check the intersection of all and only those pairs of segments that sometime during the movement of the sweep line at least once were neighbors to each other.
It is easy to notice that it is enough only to check the added segment with its upper and lower neighbors, as well as when removing the segment — its upper and lower neighbors (which after removal will become neighbors of each other).
- It should be noted that at a fixed position of the sweep line, we must **first add all the segments** that start at this x-coordinate, and only **then remove all the segments** that end here.
Thus, we do not miss the intersection of segments on the vertex: i.e. such cases when two segments have a common vertex.
- Note that **vertical segments** do not actually affect the correctness of the algorithm. These segments are distinguished by the fact that they appear and disappear at the same time. However, due to the previous comment, we know that all segments will be added to the queue first, and only then they will be deleted. Therefore, if the vertical segment intersects with some other segment opened at that moment (including the vertical one), it will be detected.

In what place of the queue to place vertical segments? After all, a vertical segment does not have one specific y-coordinate, it extends for an entire segment along the y-coordinate. However, it is easy to understand that any coordinate from this segment can be taken as a y-coordinate.

Thus, the entire algorithm will perform no more than $2n$ tests on the intersection of a pair of segments, and will perform $O(n)$ operations with a queue of segments ($O(1)$ operations at the time of appearance and disappearance of each segment).

The final **asymptotic behavior of the algorithm** is thus $O(n \log n)$.

Implementation

We present the full implementation of the described algorithm:

```
const double EPS = 1E-9;

struct pt {
    double x, y;
};

struct seg {
    pt p, q;
    int id;

    double get_y(double x) const {
        if (abs(p.x - q.x) < EPS)
            return p.y;
        return p.y + (q.y - p.y) * (x - p.x) / (q.x - p.x);
    }
};

bool intersect1d(double l1, double r1, double l2, double r2) {
    if (l1 > r1)
        swap(l1, r1);
    if (l2 > r2)
        swap(l2, r2);
    return max(l1, l2) <= min(r1, r2) + EPS;
}

int vec(const pt& a, const pt& b, const pt& c) {
    double s = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
    return abs(s) < EPS ? 0 : s > 0 ? +1 : -1;
}
```

```

bool intersect(const seg& a, const seg& b)
{
    return intersect1d(a.p.x, a.q.x, b.p.x, b.q.x) &&
           intersect1d(a.p.y, a.q.y, b.p.y, b.q.y) &&
           vec(a.p, a.q, b.p) * vec(a.p, a.q, b.q) <= 0 &&
           vec(b.p, b.q, a.p) * vec(b.p, b.q, a.q) <= 0;
}

bool operator<(const seg& a, const seg& b)
{
    double x = max(min(a.p.x, a.q.x), min(b.p.x, b.q.x));
    return a.get_y(x) < b.get_y(x) - EPS;
}

struct event {
    double x;
    int tp, id;

    event() {}
    event(double x, int tp, int id) : x(x), tp(tp), id(id) {}

    bool operator<(const event& e) const {
        if (abs(x - e.x) > EPS)
            return x < e.x;
        return tp > e.tp;
    }
};

set<seg> s;
vector<set<seg>::iterator> where;

set<seg>::iterator prev(set<seg>::iterator it) {
    return it == s.begin() ? s.end() : --it;
}

set<seg>::iterator next(set<seg>::iterator it) {
    return ++it;
}

pair<int, int> solve(const vector<seg>& a) {
    int n = (int)a.size();
    vector<event> e;
    for (int i = 0; i < n; ++i) {
        e.push_back(event(min(a[i].p.x, a[i].q.x), +1, i));
        e.push_back(event(max(a[i].p.x, a[i].q.x), -1, i));
    }
    sort(e.begin(), e.end());

    s.clear();

```

```

    where.resize(a.size());
    for (size_t i = 0; i < e.size(); ++i) {
        int id = e[i].id;
        if (e[i].tp == +1) {
            set<seg>::iterator nxt = s.lower_bound(a[id]), prv =
prev(nxt);
            if (nxt != s.end() && intersect(*nxt, a[id]))
                return make_pair(nxt->id, id);
            if (prv != s.end() && intersect(*prv, a[id]))
                return make_pair(prv->id, id);
            where[id] = s.insert(nxt, a[id]);
        } else {
            set<seg>::iterator nxt = next(where[id]), prv =
prev(where[id]);
            if (nxt != s.end() && prv != s.end() && intersect(*nxt,
*prv))
                return make_pair(prv->id, nxt->id);
            s.erase(where[id]);
        }
    }

    return make_pair(-1, -1);
}

```

The main function here is `solve()`, which returns the intersecting segments if exists, or $(-1, -1)$, if there are no intersections.

Checking for the intersection of two segments is carried out by the `intersect()` function, using an **algorithm based on the oriented area of the triangle**.

The queue of segments is the global variable `s`, a `set<event>`. Iterators that specify the position of each segment in the queue (for convenient removal of segments from the queue) are stored in the global array `where`.

Two auxiliary functions `prev()` and `next()` are also introduced, which return iterators to the previous and next elements (or `end()`, if one does not exist).

The constant `EPS` denotes the error of comparing two real numbers (it is mainly used when checking two segments for intersection).

Problems

- [TIMUS 1469 No Smoking!](#)

Contributors:

[Amandeep Singh](#) (89.01%) [Michael Hayter](#) (4.95%) [Oleksandr Kulkov](#) (3.3%)
[Yury Semenov](#) (1.65%) [Anton Älgmyr](#) (0.55%) [Juan Camilo Salazar](#) (0.55%)

Translated

From: e-maxx.ru

Finding faces of a planar graph

Consider a graph G with n vertices and m edges, which can be drawn on a plane in such a way that two edges intersect only at a common vertex (if it exists). Such graphs are called **planar**. Now suppose that we are given a planar graph together with its straight-line embedding, which means that for each vertex v we have a corresponding point (x, y) and all edges are drawn as line segments between these points without intersection (such embedding always exists). These line segments split the plane into several regions, which are called faces. Exactly one of the faces is unbounded. This face is called **outer**, while the other faces are called **inner**.

In this article we will deal with finding both inner and outer faces of a planar graph. We will assume that the graph is connected.

Some facts about planar graphs

In this section we present several facts about planar graphs without proof. Readers who are interested in proofs should refer to [Graph Theory by R. Diestel](#) (see also [video lectures on planarity](#) based on this book) or some other book.

Euler's theorem

Euler's theorem states that any correct embedding of a connected planar graph with n vertices, m edges and f faces satisfies:

$$n - m + f = 2$$

And more generally, every planar graph with k connected components satisfies:

$$n - m + f = 1 + k$$

Number of edges of a planar graph.

If $n \geq 3$ then the maximum number of edges of a planar graph with n vertices is $3n - 6$. This number is achieved by any connected planar graph where each face is bounded by a triangle. In terms of complexity this fact means that $m = O(n)$ for any planar graph.

Number of faces of a planar graph.

As a direct consequence of the above fact, if $n \geq 3$ then the maximum number of faces of a planar graph with n vertices is $2n - 4$.

Minimum vertex degree in a planar graph.

Every planar graph has a vertex of degree 5 or less.

The algorithm

Firstly, sort the adjacent edges for each vertex by polar angle. Now let's traverse the graph in the following way. Suppose that we entered vertex u through the edge (v, u) and (u, w) is the next edge after (v, u) in the sorted adjacency list of u . Then the next vertex will be w . It turns out that if we start this traversal at some edge (v, u) , we will traverse exactly one of the faces adjacent to (v, u) , the exact face depending on whether our first step is from u to v or from v to u .

Now the algorithm is quite obvious. We must iterate over all edges of the graph and start the traversal for each edge that wasn't visited by one of the previous traversals. This way we will find each face exactly once, and each edge will be traversed twice (once in each direction).

Finding the next edge

During the traversal we have to find the next edge in counter-clockwise order. The most obvious way to find the next edge is binary search by angle. However, given the counter-clockwise order of adjacent edges for each vertex, we can precompute the next edges and store them in a hash table. If the edges are already sorted by angle, the complexity of finding all faces in this case becomes linear.

Finding the outer face

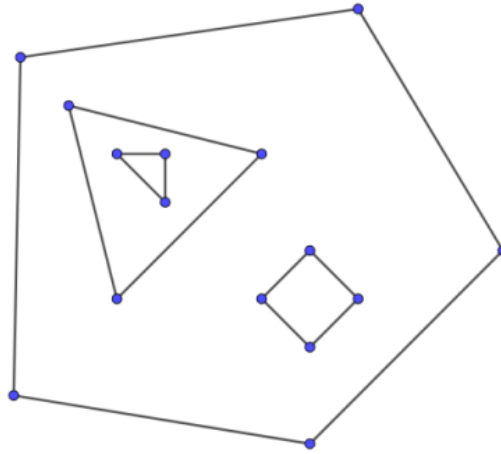
It's not hard to see that the algorithm traverses each inner face in a clockwise order and the outer face in the counter-clockwise order, so the outer face can be found by checking the order of each face.

Complexity

It's quite clear that the complexity of the algorithm is $O(m \log m)$ because of sorting, and since $m = O(n)$, it's actually $O(n \log n)$. As mentioned before, without sorting the complexity becomes $O(n)$.

What if the graph isn't connected?

At the first glance it may seem that finding faces of a disconnected graph is not much harder because we can run the same algorithm for each connected component. However, the components may be drawn in a nested way, forming **holes** (see the image below). In this case the inner face of some component becomes the outer face of some other components and has a complex disconnected border. Dealing with such cases is quite hard, one possible approach is to identify nested components with [point location](#) algorithms.



Implementation

The following implementation returns a vector of vertices for each face, outer face goes first. Inner faces are returned in counter-clockwise orders and the outer face is returned in clockwise order.

For simplicity we find the next edge by doing binary search by angle.

```
struct Point {
    int64_t x, y;

    Point(int64_t x_, int64_t y_): x(x_), y(y_) {}

    Point operator - (const Point & p) const {
        return Point(x - p.x, y - p.y);
    }

    int64_t cross (const Point & p) const {
        return x * p.y - y * p.x;
    }

    int64_t cross (const Point & p, const Point & q) const {
        return (p - *this).cross(q - *this);
    }

    int half () const {
        return int(y < 0 || (y == 0 && x < 0));
    }
};
```

```

std::vector<std::vector<size_t>> find_faces(std::vector<Point> vertices,
std::vector<std::vector<size_t>> adj) {
    size_t n = vertices.size();
    std::vector<std::vector<char>> used(n);
    for (size_t i = 0; i < n; i++) {
        used[i].resize(adj[i].size());
        used[i].assign(adj[i].size(), 0);
        auto compare = [&](size_t l, size_t r) {
            Point pl = vertices[l] - vertices[i];
            Point pr = vertices[r] - vertices[i];
            if (pl.half() != pr.half())
                return pl.half() < pr.half();
            return pl.cross(pr) > 0;
        };
        std::sort(adj[i].begin(), adj[i].end(), compare);
    }
    std::vector<std::vector<size_t>> faces;
    for (size_t i = 0; i < n; i++) {
        for (size_t edge_id = 0; edge_id < adj[i].size(); edge_id++) {
            if (used[i][edge_id]) {
                continue;
            }
            std::vector<size_t> face;
            size_t v = i;
            size_t e = edge_id;
            while (!used[v][e]) {
                used[v][e] = true;
                face.push_back(v);
                size_t u = adj[v][e];
                size_t e1 = std::lower_bound(adj[u].begin(),
adj[u].end(), v, [&](size_t l, size_t r) {
                    Point pl = vertices[l] - vertices[u];
                    Point pr = vertices[r] - vertices[u];
                    if (pl.half() != pr.half())
                        return pl.half() < pr.half();
                    return pl.cross(pr) > 0;
                }) - adj[u].begin() + 1;
                if (e1 == adj[u].size()) {
                    e1 = 0;
                }
                v = u;
                e = e1;
            }
            std::reverse(face.begin(), face.end());
            Point p1 = vertices[face[0]];
            __int128 sum = 0;
            for (int j = 0; j < face.size(); ++j) {

```

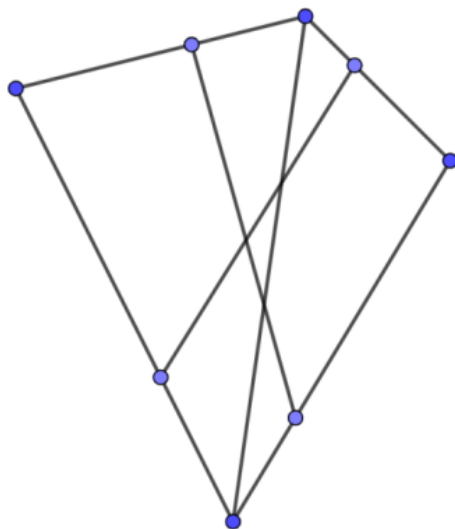
```

        Point p2 = vertices[face[j]];
        Point p3 = vertices[face[(j + 1) % face.size()]];
        sum += (p2 - p1).cross(p3 - p2);
    }
    if (sum <= 0) {
        faces.insert(faces.begin(), face);
    } else {
        faces.emplace_back(face);
    }
}
}
return faces;
}

```

Building planar graph from line segments

Sometimes you are not given a graph explicitly, but rather as a set of line segments on a plane, and the actual graph is formed by intersecting those segments, as shown in the picture below. In this case you have to build the graph manually. The easiest way to do so is as follows. Fix a segment and intersect it with all other segments. Then sort all intersection points together with the two endpoints of the segment lexicographically and add them to the graph as vertices. Also link each two adjacent vertices in lexicographical order by an edge. After doing this procedure for all edges we will obtain the graph. Of course, we should ensure that two equal intersection points will always correspond to the same vertex. The easiest way to do this is to store the points in a map by their coordinates, regarding points whose coordinates differ by a small number (say, less than 10^{-9}) as equal. This algorithm works in $O(n^2 \log n)$.



Implementation

```
using dbl = long double;

const dbl eps = 1e-9;

struct Point {
    dbl x, y;

    Point(){}
    Point(dbl x_, dbl y_): x(x_), y(y_) {}

    Point operator * (dbl d) const {
        return Point(x * d, y * d);
    }

    Point operator + (const Point & p) const {
        return Point(x + p.x, y + p.y);
    }

    Point operator - (const Point & p) const {
        return Point(x - p.x, y - p.y);
    }

    dbl cross (const Point & p) const {
        return x * p.y - y * p.x;
    }
}
```

```

dbl cross (const Point & p, const Point & q) const {
    return (p - *this).cross(q - *this);
}

dbl dot (const Point & p) const {
    return x * p.x + y * p.y;
}

dbl dot (const Point & p, const Point & q) const {
    return (p - *this).dot(q - *this);
}

bool operator < (const Point & p) const {
    if (fabs(x - p.x) < eps) {
        if (fabs(y - p.y) < eps) {
            return false;
        } else {
            return y < p.y;
        }
    } else {
        return x < p.x;
    }
}

bool operator == (const Point & p) const {
    return fabs(x - p.x) < eps && fabs(y - p.y) < eps;
}

bool operator >= (const Point & p) const {
    return !(*this < p);
}

};

struct Line{
    Point p[2];

    Line(Point l, Point r){p[0] = l; p[1] = r;}
    Point& operator [] (const int & i){return p[i];}
    const Point& operator [] (const int & i) const {return p[i];}
    Line(const Line & l){
        p[0] = l.p[0]; p[1] = l.p[1];
    }
    Point getOrth() const {
        return Point(p[1].y - p[0].y, p[0].x - p[1].x);
    }
    bool hasPointLine(const Point & t) const {
        return std::fabs(p[0].cross(p[1], t)) < eps;
    }
}

```

```

    bool hasPointSeg(const Point & t) const {
        return hasPointLine(t) && t.dot(p[0], p[1]) < eps;
    }
};

std::vector<Point> interLineLine(Line l1, Line l2) {
    if (std::fabs(l1.getOrth().cross(l2.getOrth())) < eps) {
        if (l1.hasPointLine(l2[0])) return {l1[0], l1[1]};
        else return {};
    }
    Point u = l2[1] - l2[0];
    Point v = l1[1] - l1[0];
    dbl s = u.cross(l2[0] - l1[0]) / u.cross(v);
    return {Point(l1[0] + v * s)};
}

std::vector<Point> interSegSeg(Line l1, Line l2) {
    if (l1[0] == l1[1]) {
        if (l2[0] == l2[1]) {
            if (l1[0] == l2[0])
                return {l1[0]};
            else
                return {};
        } else {
            if (l2.hasPointSeg(l1[0]))
                return {l1[0]};
            else
                return {};
        }
    }
    if (l2[0] == l2[1]) {
        if (l1.hasPointSeg(l2[0]))
            return {l2[0]};
        else
            return {};
    }
    auto li = interLineLine(l1, l2);
    if (li.empty())
        return li;
    if (li.size() == 2) {
        if (l1[0] >= l1[1])
            std::swap(l1[0], l1[1]);
        if (l2[0] >= l2[1])
            std::swap(l2[0], l2[1]);
        std::vector<Point> res(2);
        if (l1[0] < l2[0])
            res[0] = l2[0];
        else

```

```

        res[0] = l1[0];
    if (l1[1] < l2[1])
        res[1] = l1[1];
    else
        res[1] = l2[1];
    if (res[0] == res[1])
        res.pop_back();
    if (res.size() == 2u && res[1] < res[0])
        return {};
    else
        return res;
}
Point cand = li[0];
if (l1.hasPointSeg(cand) && l2.hasPointSeg(cand))
    return {cand};
else
    return {};
}

std::pair<std::vector<Point>, std::vector<std::vector<size_t>>>
build_graph(std::vector<Line> segments) {
    std::vector<Point> p;
    std::vector<std::vector<size_t>> adj;
    std::map<std::pair<int64_t, int64_t>, size_t> point_id;
    auto get_point_id = [&](Point pt) {
        auto repr = std::make_pair(
            int64_t(std::round(pt.x * 1000000000) + 1e-6),
            int64_t(std::round(pt.y * 1000000000) + 1e-6)
        );
        if (!point_id.count(repr)) {
            adj.emplace_back();
            size_t id = point_id.size();
            point_id[repr] = id;
            p.push_back(pt);
            return id;
        } else {
            return point_id[repr];
        }
    };
    for (size_t i = 0; i < segments.size(); i++) {
        std::vector<size_t> curr = {
            get_point_id(segments[i][0]),
            get_point_id(segments[i][1])
        };
        for (size_t j = 0; j < segments.size(); j++) {
            if (i == j)
                continue;
            auto inter = interSegSeg(segments[i], segments[j]);

```

```

        for (auto pt: inter) {
            curr.push_back(get_point_id(pt));
        }
    }
    std::sort(curr.begin(), curr.end(), [&](size_t l, size_t r) {
return p[l] < p[r]; });
    curr.erase(std::unique(curr.begin(), curr.end()), curr.end());
    for (size_t j = 0; j + 1 < curr.size(); j++) {
        adj[curr[j]].push_back(curr[j + 1]);
        adj[curr[j + 1]].push_back(curr[j]);
    }
}
for (size_t i = 0; i < adj.size(); i++) {
    std::sort(adj[i].begin(), adj[i].end());
    // removing edges that were added multiple times
    adj[i].erase(std::unique(adj[i].begin(), adj[i].end()),
adj[i].end());
}
return {p, adj};
}

```

Problems

- [TIMUS 1664 Pipeline Transportation](#)
- [TIMUS 1681 Brother Bear's Garden](#)

Contributors:

[Yury Semenov](#) (95.7%)
 [hazzlerr](#) (2.01%)
 [Michael Hayter](#) (1.72%)
 [Bartosz Kostka](#) (0.29%)
[Oleksandr Kulkov](#) (0.29%)

Original

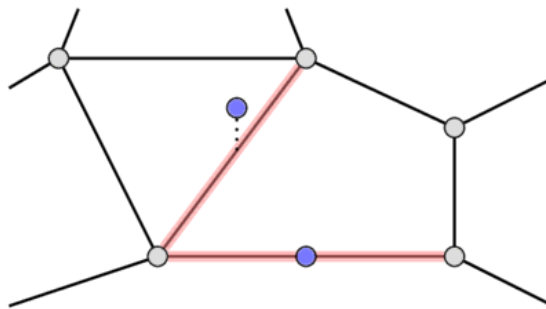
Point location in $O(\log n)$

Consider the following problem: you are given a [planar subdivision](#) without any vertices of degree one and zero, and a lot of queries. Each query is a point, for which we should determine the face of the subdivision it belongs to. We will answer each query in $O(\log n)$ offline.

This problem may arise when you need to locate some points in a Voronoi diagram or in some simple polygon.

Algorithm

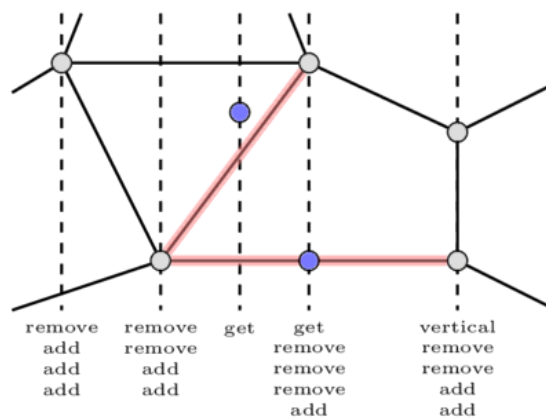
Firstly, for each query point $p(x_0, y_0)$ we want to find such an edge that if the point belongs to any edge, the point lies on the edge we found, otherwise this edge must intersect the line $x = x_0$ at some unique point (x_0, y) where $y < y_0$ and this y is maximum among all such edges. The following image shows both cases.



We will solve this problem offline using the sweep line algorithm. Let's iterate over x -coordinates of query points and edges' endpoints in increasing order and keep a set of edges s . For each x -coordinate we will add some events beforehand.

The events will be of four types: *add*, *remove*, *vertical*, *get*. For each vertical edge (both endpoints have the same x-coordinate) we will add one *vertical* event for the corresponding x-coordinate. For every other edge we will add one *add* event for the minimum of x-coordinates of the endpoints and one *remove* event for the maximum of x-coordinates of the endpoints. Finally, for each query point we will add one *get* event for its x-coordinate.

For each x-coordinate we will sort the events in order (*vertical*, *get*, *remove*, *add*). The following image shows all events in sorted order for each x-coordinate.



We will keep two sets during the sweep-line process. A set t for all non-vertical edges, and one set $vert$ especially for the vertical ones. We will clear the set $vert$ at the beginning of processing each x-coordinate.

Now let's process the events for a fixed x-coordinate.

- If we got a *vertical* event, we will simply insert the minimum y-coordinate of the corresponding edge's endpoints to $vert$.
- If we got a *remove* or *add* event, we will remove the corresponding edge from t or add it to t .
- Finally, for each *get* event we must check if the point lies on some vertical edge by performing a binary search in $vert$. If the point doesn't lie on any vertical edge, we must find the answer for this query in t . To do this, we again make a binary search. In order to handle some degenerate cases (e.g. in case of the triangle $(0, 0)$, $(0, 2)$, $(1, 1)$ when we query the point $(0, 0)$), we must answer all *get* events

again after we processed all the events for this x-coordinate and choose the best of two answers.

Now let's choose a comparator for the set t . This comparator should check if one edge doesn't lie above other for every x-coordinate they both cover. Suppose that we have two edges (a, b) and (c, d) . Then the comparator is (in pseudocode):

```
val = sgn((b - a) × (c - a)) + sgn((b - a) × (d - a))
if val ≠ 0
  then return val > 0
val = sgn((d - c) × (a - c)) + sgn((d - c) × (b - c))
return val < 0
```

Now for every query we have the corresponding edge. How to find the face? If we couldn't find the edge it means that the point is in the outer face. If the point belongs to the edge we found, the face is not unique. Otherwise, there are two candidates - the faces that are bounded by this edge. How to check which one is the answer? Note that the edge is not vertical. Then the answer is the face that is above this edge. Let's find such a face for each non-vertical edge. Consider a counter-clockwise traversal of each face. If during this traversal we increased x-coordinate while passing through the edge, then this face is the face we need to find for this edge.

Notes

Actually, with persistent trees this approach can be used to answer the queries online.

Implementation

The following code is implemented for integers, but it can be easily modified to work with doubles (by changing the compare methods and the point type). This implementation assumes that the subdivision is correctly stored inside a [DCEL](#) and the outer face is numbered -1 .

For each query a pair $(1, i)$ is returned if the point lies strictly inside the face number i , and a pair $(0, i)$ is returned if the point lies on the edge number i .

```

typedef long long ll;

bool ge(const ll& a, const ll& b) { return a >= b; }
bool le(const ll& a, const ll& b) { return a <= b; }
bool eq(const ll& a, const ll& b) { return a == b; }
bool gt(const ll& a, const ll& b) { return a > b; }
bool lt(const ll& a, const ll& b) { return a < b; }
int sgn(const ll& x) { return le(x, 0) ? eq(x, 0) ? 0 : -1 : 1; }

struct pt {
    ll x, y;
    pt() {}
    pt(ll _x, ll _y) : x(_x), y(_y) {}
    pt operator-(const pt& a) const { return pt(x - a.x, y - a.y); }
    ll dot(const pt& a) const { return x * a.x + y * a.y; }
    ll dot(const pt& a, const pt& b) const { return (a - *this).dot(b -
*this); }
    ll cross(const pt& a) const { return x * a.y - y * a.x; }
    ll cross(const pt& a, const pt& b) const { return (a -
*this).cross(b - *this); }
    bool operator==(const pt& a) const { return a.x == x && a.y == y; }
};

struct Edge {
    pt l, r;
};

bool edge_cmp(Edge* edge1, Edge* edge2)
{
    const pt a = edge1->l, b = edge1->r;
    const pt c = edge2->l, d = edge2->r;
    int val = sgn(a.cross(b, c)) + sgn(a.cross(b, d));
    if (val != 0)
        return val > 0;
    val = sgn(c.cross(d, a)) + sgn(c.cross(d, b));
    return val < 0;
}

enum EventType { DEL = 2, ADD = 3, GET = 1, VERT = 0 };

struct Event {
    EventType type;
    int pos;
    bool operator<(const Event& event) const { return type < event.type; }
};

vector<Edge*> sweepline(vector<Edge*> planar, vector<pt> queries)

```

```

{
    using pt_type = decltype(pt::x);

    // collect all x-coordinates
    auto s =
        set<pt_type, std::function<bool(const pt_type&, const
pt_type&)>>(lt);
    for (pt p : queries)
        s.insert(p.x);
    for (Edge* e : planar) {
        s.insert(e->l.x);
        s.insert(e->r.x);
    }

    // map all x-coordinates to ids
    int cid = 0;
    auto id =
        map<pt_type, int, std::function<bool(const pt_type&, const
pt_type&)>>(
        lt);
    for (auto x : s)
        id[x] = cid++;

    // create events
    auto t = set<Edge*, decltype(*edge_cmp)>(edge_cmp);
    auto vert_cmp = [] (const pair<pt_type, int>& l,
                        const pair<pt_type, int>& r) {
        if (!eq(l.first, r.first))
            return lt(l.first, r.first);
        return l.second < r.second;
    };
    auto vert = set<pair<pt_type, int>, decltype(vert_cmp)>(vert_cmp);
    vector<vector<Event>> events(cid);
    for (int i = 0; i < (int)queries.size(); i++) {
        int x = id[queries[i].x];
        events[x].push_back(Event{GET, i});
    }
    for (int i = 0; i < (int)planar.size(); i++) {
        int lx = id[planar[i]->l.x], rx = id[planar[i]->r.x];
        if (lx > rx) {
            swap(lx, rx);
            swap(planar[i]->l, planar[i]->r);
        }
        if (lx == rx) {
            events[lx].push_back(Event{VERT, i});
        } else {
            events[lx].push_back(Event{ADD, i});
            events[rx].push_back(Event{DEL, i});
        }
    }
}

```

```

    }
}

// perform sweep line algorithm
vector<Edge*> ans(queries.size(), nullptr);
for (int x = 0; x < cid; x++) {
    sort(events[x].begin(), events[x].end());
    vert.clear();
    for (Event event : events[x]) {
        if (event.type == DEL) {
            t.erase(planar[event.pos]);
        }
        if (event.type == VERT) {
            vert.insert(make_pair(
                min(planar[event.pos]->l.y, planar[event.pos]->r.y),
                event.pos));
        }
        if (event.type == ADD) {
            t.insert(planar[event.pos]);
        }
        if (event.type == GET) {
            auto jt = vert.upper_bound(
                make_pair(queries[event.pos].y, planar.size()));
            if (jt != vert.begin()) {
                --jt;
                int i = jt->second;
                if (ge(max(planar[i]->l.y, planar[i]->r.y),
                    queries[event.pos].y)) {
                    ans[event.pos] = planar[i];
                    continue;
                }
            }
            Edge* e = new Edge;
            e->l = e->r = queries[event.pos];
            auto it = t.upper_bound(e);
            if (it != t.begin())
                ans[event.pos] = *(--it);
            delete e;
        }
    }
}

for (Event event : events[x]) {
    if (event.type != GET)
        continue;
    if (ans[event.pos] != nullptr &&
        eq(ans[event.pos]->l.x, ans[event.pos]->r.x))
        continue;
}

```

```

        Edge* e = new Edge;
        e->l = e->r = queries[event.pos];
        auto it = t.upper_bound(e);
        delete e;
        if (it == t.begin())
            e = nullptr;
        else
            e = *(--it);
        if (ans[event.pos] == nullptr) {
            ans[event.pos] = e;
            continue;
        }
        if (e == nullptr)
            continue;
        if (e == ans[event.pos])
            continue;
        if (id[ans[event.pos]->r.x] == x) {
            if (id[e->l.x] == x) {
                if (gt(e->l.y, ans[event.pos]->r.y))
                    ans[event.pos] = e;
            }
        } else {
            ans[event.pos] = e;
        }
    }
}

return ans;
}

struct DCEL {
    struct Edge {
        pt origin;
        Edge* nxt = nullptr;
        Edge* twin = nullptr;
        int face;
    };
    vector<Edge*> body;
};

vector<pair<int, int>> point_location(DCEL planar, vector<pt> queries)
{
    vector<pair<int, int>> ans(queries.size());
    vector<Edge*> planar2;
    map<intptr_t, int> pos;
    map<intptr_t, int> added_on;
    int n = planar.body.size();
    for (int i = 0; i < n; i++) {
        if (planar.body[i]->face > planar.body[i]->twin->face)

```

```

        continue;
    Edge* e = new Edge;
    e->l = planar.body[i]->origin;
    e->r = planar.body[i]->twin->origin;
    added_on[(intptr_t)e] = i;
    pos[(intptr_t)e] =
        lt(planar.body[i]->origin.x, planar.body[i]->twin->origin.x)
        ? planar.body[i]->face
        : planar.body[i]->twin->face;
    planar2.push_back(e);
}
auto res = sweepline(planar2, queries);
for (int i = 0; i < (int)queries.size(); i++) {
    if (res[i] == nullptr) {
        ans[i] = make_pair(1, -1);
        continue;
    }
    pt p = queries[i];
    pt l = res[i]->l, r = res[i]->r;
    if (eq(p.cross(l, r), 0) && le(p.dot(l, r), 0)) {
        ans[i] = make_pair(0, added_on[(intptr_t)res[i]]);
        continue;
    }
    ans[i] = make_pair(1, pos[(intptr_t)res[i]]);
}
for (auto e : planar2)
    delete e;
return ans;
}

```

Problems

- [TIMUS 1848 Fly Hunt](#)
- [UVA 12310 Point Location](#)

Contributors:

[Yury Semenov](#) (94.33%) [Jakob Kogler](#) (2.33%) [Michael Hayter](#) (2.0%)
[Oleksandr Kulkov](#) (0.67%) [JustAnAverageGuy](#) (0.33%) [wiktorkuchta](#) (0.33%)

Translated

From: e-maxx.ru

Finding the nearest pair of points

Problem statement

Given n points on the plane. Each point p_i is defined by its coordinates (x_i, y_i) . It is required to find among them two such points, such that the distance between them is minimal:

$$\min_{\substack{i,j=0 \dots n-1, \\ i \neq j}} \rho(p_i, p_j).$$

We take the usual Euclidean distances:

$$\rho(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

The trivial algorithm - iterating over all pairs and calculating the distance for each — works in $O(n^2)$.

The algorithm running in time $O(n \log n)$ is described below. This algorithm was proposed by Shamos and Hoey in 1975. (Source: Ch. 5 Notes of *Algorithm Design* by Kleinberg & Tardos, also see [here](#)) Preparata and Shamos also showed that this algorithm is optimal in the decision tree model.

Algorithm

We construct an algorithm according to the general scheme of **divide-and-conquer** algorithms: the algorithm is designed as a recursive function, to which we pass a set of points; this recursive function splits this set in half, calls itself recursively on each half, and then performs some operations to combine the answers. The operation of

combining consist of detecting the cases when one point of the optimal solution fell into one half, and the other point into the other (in this case, recursive calls from each of the halves cannot detect this pair separately). The main difficulty, as always in case of divide and conquer algorithms, lies in the effective implementation of the merging stage. If a set of n points is passed to the recursive function, then the merge stage should work no more than $O(n)$, then the asymptotics of the whole algorithm $T(n)$ will be found from the equation:

$$T(n) = 2T(n/2) + O(n).$$

The solution to this equation, as is known, is $T(n) = O(n \log n)$.

So, we proceed on to the construction of the algorithm. In order to come to an effective implementation of the merge stage in the future, we will divide the set of points into two subsets, according to their x -coordinates: In fact, we draw some vertical line dividing the set of points into two subsets of approximately the same size. It is convenient to make such a partition as follows: We sort the points in the standard way as pairs of numbers, ie.:

$$p_i < p_j \iff (x_i < x_j) \vee ((x_i = x_j) \wedge (y_i < y_j))$$

Then take the middle point after sorting $p_m (m = \lfloor n/2 \rfloor)$, and all the points before it and the p_m itself are assigned to the first half, and all the points after it - to the second half:

$$A_1 = \{p_i \mid i = 0 \dots m\}$$

$$A_2 = \{p_i \mid i = m + 1 \dots n - 1\}.$$

Now, calling recursively on each of the sets A_1 and A_2 , we will find the answers h_1 and h_2 for each of the halves. And take the best of them: $h = \min(h_1, h_2)$.

Now we need to make a **merge stage**, i.e. we try to find such pairs of points, for which the distance between which is less than h and one point is lying in A_1 and the other in

A_2 . It is obvious that it is sufficient to consider only those points that are separated from the vertical line by a distance less than h , i.e. the set B of the points considered at this stage is equal to:

$$B = \{p_i \mid |x_i - x_m| < h\}.$$

For each point in the set B , we try to find the points that are closer to it than h . For example, it is sufficient to consider only those points whose y -coordinate differs by no more than h . Moreover, it makes no sense to consider those points whose y -coordinate is greater than the y -coordinate of the current point. Thus, for each point p_i we define the set of considered points $C(p_i)$ as follows:

$$C(p_i) = \{p_j \mid p_j \in B, y_i - h < y_j \leq y_i\}.$$

If we sort the points of the set B by y -coordinate, it will be very easy to find $C(p_i)$: these are several points in a row ahead to the point p_i .

So, in the new notation, the **merging stage** looks like this: build a set B , sort the points in it by y -coordinate, then for each point $p_i \in B$ consider all points $p_j \in C(p_i)$, and for each pair (p_i, p_j) calculate the distance and compare with the current best distance.

At first glance, this is still a non-optimal algorithm: it seems that the sizes of sets $C(p_i)$ will be of order n , and the required asymptotics will not work. However, surprisingly, it can be proved that the size of each of the sets $C(p_i)$ is a quantity $O(1)$, i.e. it does not exceed some small constant regardless of the points themselves. Proof of this fact is given in the next section.

Finally, we pay attention to the sorting, which the above algorithm contains: first, sorting by pairs (x, y) , and then second, sorting the elements of the set B by y . In fact, both of these sorts inside the recursive function can be eliminated (otherwise we would not reach the $O(n)$ estimate for the **merging stage**, and the general asymptotics of the algorithm would be $O(n \log^2 n)$). It is easy to get rid of the first sort – it is enough to perform this sort before starting the recursion: after all, the elements themselves do not change inside the recursion, so there is no need to sort again. With the second sorting a

little more difficult to perform, performing it previously will not work. But, remembering the merge sort, which also works on the principle of divide-and-conquer, we can simply embed this sort in our recursion. Let recursion, taking some set of points (as we remember, ordered by pairs (x, y)), return the same set, but sorted by the y -coordinate. To do this, simply merge (in $O(n)$) the two results returned by recursive calls. This will result in a set sorted by y -coordinate.

Evaluation of the asymptotics

To show that the above algorithm is actually executed in $O(n \log n)$, we need to prove the following fact: $|C(p_i)| = O(1)$.

So, let us consider some point p_i ; recall that the set $C(p_i)$ is a set of points whose y -coordinate lies in the segment $[y_i - h; y_i]$, and, moreover, along the x coordinate, the point p_i itself, and all the points of the set $C(p_i)$ lie in the band width $2h$. In other words, the points we are considering p_i and $C(p_i)$ lie in a rectangle of size $2h \times h$.

Our task is to estimate the maximum number of points that can lie in this rectangle $2h \times h$; thus, we estimate the maximum size of the set $C(p_i)$. At the same time, when evaluating, we must not forget that there may be repeated points.

Remember that h was obtained from the results of two recursive calls — on sets A_1 and A_2 , and A_1 contains points to the left of the partition line and partially on it, A_2 contains the remaining points of the partition line and points to the right of it. For any pair of points from A_1 , as well as from A_2 , the distance can not be less than h — otherwise it would mean incorrect operation of the recursive function.

To estimate the maximum number of points in the rectangle $2h \times h$ we divide it into two squares $h \times h$, the first square include all points $C(p_i) \cap A_1$, and the second contains all the others, i.e. $C(p_i) \cap A_2$. It follows from the above considerations that in each of these squares the distance between any two points is at least h .

We show that there are at most four points in each square. For example, this can be done as follows: divide the square into 4 sub-squares with sides $h/2$. Then there can be no more than one point in each of these sub-squares (since even the diagonal is

equal to $h/\sqrt{2}$, which is less than h). Therefore, there can be no more than 4 points in the whole square.

So, we have proved that in a rectangle $2h \times h$ can not be more than $4 \cdot 2 = 8$ points, and, therefore, the size of the set $C(p_i)$ cannot exceed 7, as required.

Implementation

We introduce a data structure to store a point (its coordinates and a number) and comparison operators required for two types of sorting:

```
struct pt {
    int x, y, id;
};

struct cmp_x {
    bool operator()(const pt & a, const pt & b) const {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    }
};

struct cmp_y {
    bool operator()(const pt & a, const pt & b) const {
        return a.y < b.y;
    }
};

int n;
vector<pt> a;
```

For a convenient implementation of recursion, we introduce an auxiliary function `upd_ans()`, which will calculate the distance between two points and check whether it is better than the current answer:

```
double mindist;
pair<int, int> best_pair;

void upd_ans(const pt & a, const pt & b) {
    double dist = sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));
    if (dist < mindist) {
        mindist = dist;
        best_pair = {a.id, b.id};
    }
}
```

```

    }
}

```

Finally, the implementation of the recursion itself. It is assumed that before calling it, the array `a[]` is already sorted by x-coordinate. In recursion we pass just two pointers `l, r`, which indicate that it should look for the answer for `a[l ... r)`. If the distance between `r` and `l` is too small, the recursion must be stopped, and perform a trivial algorithm to find the nearest pair and then sort the subarray by y-coordinate.

To merge two sets of points received from recursive calls into one (ordered by y-coordinate), we use the standard STL `merge()` function, and create an auxiliary buffer `t[]` (one for all recursive calls). (Using `inplace_merge()` is impractical because it generally does not work in linear time.)

Finally, the set `B` is stored in the same array `t`.

```

vector<pt> t;

void rec(int l, int r) {
    if (r - l <= 3) {
        for (int i = l; i < r; ++i) {
            for (int j = i + 1; j < r; ++j) {
                upd_ans(a[i], a[j]);
            }
        }
        sort(a.begin() + l, a.begin() + r, cmp_y());
        return;
    }

    int m = (l + r) >> 1;
    int midx = a[m].x;
    rec(l, m);
    rec(m, r);

    merge(a.begin() + l, a.begin() + m, a.begin() + m, a.begin() + r,
          t.begin(), cmp_y());
    copy(t.begin(), t.begin() + r - l, a.begin() + l);

    int tsz = 0;
    for (int i = l; i < r; ++i) {
        if (abs(a[i].x - midx) < mindist) {
            for (int j = tsz - 1; j >= 0 && a[i].y - t[j].y < mindist; --j)
                upd_ans(a[i], t[j]);
            t[tsz++] = a[i];
        }
    }
}

```

```

    }
  }
}

```

By the way, if all the coordinates are integer, then at the time of the recursion you can not move to fractional values, and store in `mindist` the square of the minimum distance.

In the main program, recursion should be called as follows:

```

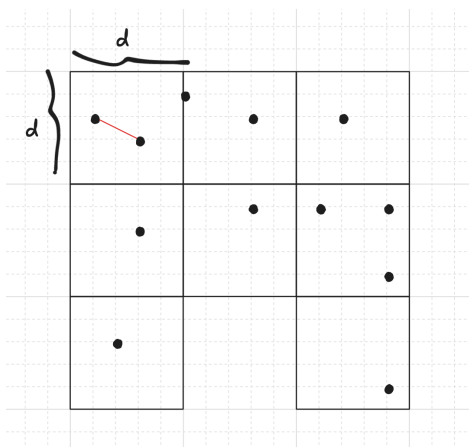
t.resize(n);
sort(a.begin(), a.end(), cmp_x());
mindist = 1E20;
rec(0, n);

```

Linear time randomized algorithms

A randomized algorithm with linear expected time

An alternative method, originally proposed by Rabin in 1976, arises from a very simple idea to heuristically improve the runtime: We can divide the plane into a grid of $d \times d$ squares, then it is only required to test distances between same-block or adjacent-block points (unless all squares are disconnected from each other, but we will avoid this by design), since any other pair has a larger distance than the two points in the same square.



We will consider only the squares containing at least one point. Denote by $n_1, n_2, \dots, n_k$ the number of points in each of the k remaining squares. Assuming at

least two points are in the same or in adjacent squares, and that there are no duplicated points, the time complexity is $\Theta(\sum_{i=1}^k n_i^2)$. We can look for duplicated points in expected linear time using a hash table, and in the affirmative case, the answer is this pair.

Proof

For the i -th square containing n_i points, the number of pairs inside is $\Theta(n_i^2)$. If the i -th square is adjacent to the j -th square, then we also perform $n_i n_j \leq \max(n_i, n_j)^2 \leq n_i^2 + n_j^2$ distance comparisons. Notice that each square has at most 8 adjacent squares, so we can bound the sum of all comparisons by $\Theta(\sum_{i=1}^k n_i^2)$.

Now we need to decide on how to set d so that it minimizes $\Theta(\sum_{i=1}^k n_i^2)$.

Choosing d

We need d to be an approximation of the minimum distance d . Richard Lipton proposed to sample n distances randomly and choose d to be the smallest of these distances as an approximation for d . We now prove that the expected running time of the algorithm is linear.

 Proof


Imagine the disposition of points in squares with a particular choice of d , say x . Consider d a random variable, resulting from our sampling of distances. Let's define $C(x) := \sum_{i=1}^{k(x)} n_i(x)^2$ as the cost estimation for a particular disposition when we choose $d = x$. Now, let's define $\lambda(x)$ such that $C(x) = \lambda(x) n$. What is the probability that such choice x survives the sampling of n independent distances? If a single pair among the sampled ones has distance smaller than x , this arrangement will be replaced by the smaller d . Inside a square, about $1/16$ of the pairs would raise a smaller distance (imagine four subsquares in every square; using the pigeonhole principle, at least one subsquare has $n_i/4$ points), so we have about $\sum_{i=1}^k \binom{n_i/4}{2} \approx \sum_{i=1}^k \frac{1}{16} \binom{n_i}{2}$ pairs which yield a smaller final d . This is, approximately, $\frac{1}{32} \sum_{i=1}^k n_i^2 = \frac{1}{32} \lambda(x) n$. On the other hand, there are about $\frac{1}{2} n^2$ pairs that can be sampled. We have that the probability of sampling a pair with distance smaller than x is at least (approximately)

$$\frac{\lambda(x) n / 32}{n^2 / 2} = \frac{\lambda(x) / 16}{n}$$

so the probability of at least one such pair being chosen during the n rounds (and therefore finding a smaller d) is

$$1 - \left(1 - \frac{\lambda(x) / 16}{n}\right)^n \geq 1 - e^{-\lambda(x) / 16}$$

(we have used that $(1 + x)^n \leq e^{xn}$ for any real number x , check [Bernoulli inequalities](#)).

Notice this goes to 1 exponentially as $\lambda(x)$ increases. This hints that λ will be small for a poorly chosen d .

We have shown that $\Pr(d \leq x) \geq 1 - e^{-\lambda(x) / 16}$, or equivalently, $\Pr(d \geq x) \leq e^{-\lambda(x) / 16}$. We need to know $\Pr(\lambda(d) \geq \text{something})$ to be able to estimate its expected value. We notice that $\lambda(d) \geq \lambda(x) \iff d \geq x$. This is because making the squares smaller only reduces the number of points in each square

(splits the points into other squares), and this keeps reducing the sum of squares. Therefore,

$$\Pr(\lambda(d) \geq \lambda(x)) = \Pr(d \geq x) \leq e^{-\lambda(x)/16} \implies \Pr(\lambda(d) \geq t) \leq e^{-t/16} \implies$$



(we have used that $E[X] = \int_0^{+\infty} \Pr(X \geq x) dx$, check [Stackexchange proof](#)).

Finally, $[C(d)] = [\lambda(d) n] \leq 16n$, and the expected running time is $O(n)$, with a reasonable constant factor.

Implementation of the algorithm

The advantage of this algorithm is that it is straightforward to implement, but still has good performance in practise. We first sample n distances and set d as the minimum of the distances. Then we insert points into the "blocks" by using a hash table from 2D coordinates to a vector of points. Finally, just compute distances between same-block pairs and adjacent-block pairs. Hash table operations have $O(1)$ expected time cost, and therefore our algorithm retains the $O(n)$ expected time cost with an increased constant.

Check out [this submission](#) to Library Checker.

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using ld = long double;

struct pt {
    ll x, y;
    pt() {}
    pt(ll x_, ll y_) : x(x_), y(y_) {}
    void read() {
        cin >> x >> y;
    }
};

bool operator==(const pt& a, const pt& b) {
    return a.x == b.x and a.y == b.y;
```

```

}

struct CustomHashPoint {
    size_t operator()(const pt& p) const {
        static const uint64_t C =
chrono::steady_clock::now().time_since_epoch().count();
        return C ^ ((p.x << 32) ^ p.y);
    }
};

ll dist2(pt a, pt b) {
    ll dx = a.x - b.x;
    ll dy = a.y - b.y;
    return dx*dx + dy*dy;
}

pair<int,int> closest_pair_of_points(vector<pt> P) {
    int n = int(P.size());
    assert(n >= 2);

    // if there is a duplicated point, we have the solution
    unordered_map<pt,int,CustomHashPoint> previous;
    for (int i = 0; i < int(P.size()); ++i) {
        auto it = previous.find(P[i]);
        if (it != previous.end()) {
            return {it->second, i};
        }
        previous[P[i]] = i;
    }

    unordered_map<pt,vector<int>,CustomHashPoint> grid;
    grid.reserve(n);

    mt19937 rd(chrono::system_clock::now().time_since_epoch().count());
    uniform_int_distribution<int> dis(0, n-1);

    ll d2 = dist2(P[0], P[1]);
    pair<int,int> closest = {0, 1};

    auto candidate_closest = [&](int i, int j) -> void {
        ll ab2 = dist2(P[i], P[j]);
        if (ab2 < d2) {
            d2 = ab2;
            closest = {i, j};
        }
    };
};

```

```

for (int i = 0; i < n; ++i) {
    int j = dis(rd);
    int k = dis(rd);
    while (j == k) k = dis(rd);
    candidate_closest(j, k);
}

ll d = ll( sqrt(ld(d2)) + 1 );

for (int i = 0; i < n; ++i) {
    grid[{P[i].x/d, P[i].y/d}].push_back(i);
}

// same block
for (const auto& it : grid) {
    int k = int(it.second.size());
    for (int i = 0; i < k; ++i) {
        for (int j = i+1; j < k; ++j) {
            candidate_closest(it.second[i], it.second[j]);
        }
    }
}

// adjacent blocks
for (const auto& it : grid) {
    auto coord = it.first;
    for (int dx = 0; dx <= 1; ++dx) {
        for (int dy = -1; dy <= 1; ++dy) {
            if (dx == 0 and dy == 0) continue;
            pt neighbour = pt(
                coord.x + dx,
                coord.y + dy
            );
            for (int i : it.second) {
                if (not grid.count(neighbour)) continue;
                for (int j : grid.at(neighbour)) {
                    candidate_closest(i, j);
                }
            }
        }
    }
}

return closest;
}

```

An alternative randomized linear expected time algorithm

Now we introduce a different randomized algorithm which is less practical but very easy to show that it runs in expected linear time.

- Permute the n points randomly
- Take $\delta := \text{dist}(p_1, p_2)$
- Partition the plane in squares of side $\delta/2$
- For $i = 1, 2, \dots, n$:
 - Take the square corresponding to p_i
 - Iterate over the 25 squares within two steps to our square in the grid of squares partitioning the plane
 - If some p_j in those squares has $\text{dist}(p_j, p_i) < \delta$, then
 - Recompute the partition and squares with $\delta := \text{dist}(p_j, p_i)$
 - Store points $p_1, \dots, p_i$ in the corresponding squares
 - else, store p_i in the corresponding square
- output δ

The correctness follows from the fact that at any moment we already have some pair with distance δ , so we try to find only new pairs with distance smaller than δ . Since each square has side $\delta/2$, a candidate pair can be at most at a distance of 2 squares, so for a given point we check candidates in the surrounding 25 squares. Any point in a square further away will always give a distance larger than δ .

While this algorithm may look slow, because of recomputing everything multiple times, we can show the total expected cost is linear.

Proof

Let X_i the random variable that is 1 when point p_i causes a change of δ and a recomputation of the data structures, and 0 if not. It is easy to show that the cost is $O(n + \sum_{i=1}^n iX_i)$, since on the i -th step we are considering only the first i points. However, turns out that $\Pr(X_i = 1) \leq \frac{2}{i}$. This is because on the i -th step, δ is the distance of the closest pair in $\{p_1, \dots, p_i\}$, and $\Pr(X_i = 1)$ is the probability of p_i belonging to the closest pair, which only happens in $2(i-1)$ pairs out of the $i(i-1)$ possible pairs (assuming all distances are different), so the probability is at most $\frac{2(i-1)}{i(i-1)} = \frac{2}{i}$, since we previously shuffled the points uniformly.

We can therefore see that the expected cost is

$$O\left(n + \sum_{i=1}^n i \Pr(X_i = 1)\right) \leq O\left(n + \sum_{i=1}^n i \frac{2}{i}\right) = O(3n) = O(n)$$

Generalization: finding a triangle with minimal perimeter

The algorithm described above is interestingly generalized to this problem: among a given set of points, choose three different points so that the sum of pairwise distances between them is the smallest.

In fact, to solve this problem, the algorithm remains the same: we divide the field into two halves of the vertical line, call the solution recursively on both halves, choose the minimum minper from the found perimeters, build a strip with the thickness of $\text{minper}/2$, and iterate through all triangles that can improve the answer. (Note that the triangle with perimeter $\leq \text{minper}$ has the longest side $\leq \text{minper}/2$.)

Translated

From: e-maxx.ru

Delaunay triangulation and Voronoi diagram

Consider a set $\{p_i\}$ of points on the plane. A **Voronoi diagram** $V(\{p_i\})$ of $\{p_i\}$ is a partition of the plane into n regions V_i , where

$V_i = \{p \in \mathbb{R}^2; \rho(p, p_i) = \min_k \rho(p, p_k)\}$. The cells of the Voronoi diagram are polygons (possibly infinite). A **Delaunay triangulation** $D(\{p_i\})$ of $\{p_i\}$ is a triangulation where every point p_i is outside or on the boundary of the circumcircle of each triangle $T \in D(\{p_i\})$.

There is a nasty degenerated case when the Voronoi diagram isn't connected and Delaunay triangulation doesn't exist. This case is when all points are collinear.

Properties

The Delaunay triangulation maximizes the minimum angle among all possible triangulations.

The Minimum Euclidean spanning tree of a point set is a subset of edges of its' Delaunay triangulation.

Duality

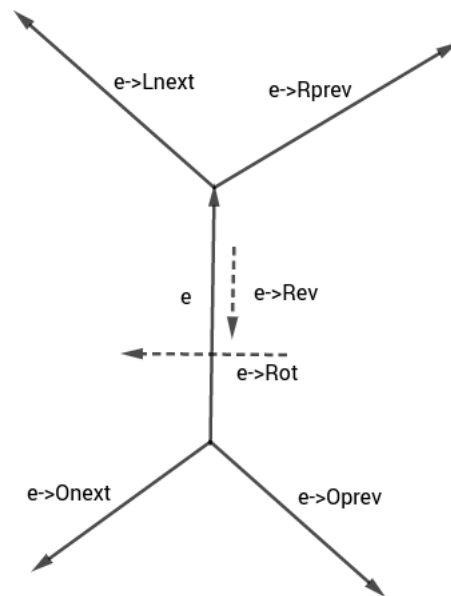
Suppose that $\{p_i\}$ is not collinear and among $\{p_i\}$ no four points lie on one circle. Then $V(\{p_i\})$ and $D(\{p_i\})$ are dual, so if we obtain one of them, we may obtain the other in $O(n)$. What to do if it's not the case? The collinear case may be processed easily. Otherwise, V and D' are dual, where D' is obtained from D by removing all the edges such that two triangles on this edge share the circumcircle.

Building Delaunay and Voronoi

Because of the duality, we only need a fast algorithm to compute only one of V and D . We will describe how to build $D(\{p_i\})$ in $O(n \log n)$. The triangulation will be built via divide-and-conquer algorithm due to Guibas and Stolfi.

Quad-edge data structure

During the algorithm D will be stored inside the quad-edge data structure. This structure is described in the picture:



In the algorithm we will use the following functions on edges:

1. `make_edge(a, b)`

This function creates an isolated edge from point `a` to point `b` together with its reverse edge and both dual edges.

2. `splice(a, b)`

This is a key function of the algorithm. It swaps `a->Onext` with `b->Onext` and `a->Onext->Rot->Onext` with `b->Onext->Rot->Onext`.

3. `delete_edge(e)`

This function deletes `e` from the triangulation. To delete `e`, we may simply call `splice(e, e->Oprev)` and `splice(e->Rev, e->Rev->Oprev)`.

4. `connect(a, b)`

This function creates a new edge `e` from `a->Dest` to `b->Org` in such a way that `a`, `b`, `e` all have the same left face. To do this, we call `e = make_edge(a->Dest, b->Org)`, `splice(e, a->Lnext)` and `splice(e->Rev, b)`.

Algorithm

The algorithm will compute the triangulation and return two quad-edges: the counterclockwise convex hull edge out of the leftmost vertex and the clockwise convex hull edge out of the rightmost vertex.

Let's sort all points by x , and if $x_1 = x_2$ then by y . Let's solve the problem for some segment (l, r) (initially $(l, r) = (0, n - 1)$). If $r - l + 1 = 2$, we will add an edge $(p[l], p[r])$ and return. If $r - l + 1 = 3$, we will firstly add the edges $(p[l], p[l + 1])$ and $(p[l + 1], p[r])$. We must also connect them using `splice(a->Rev, b)`. Now we must close the triangle. Our next action will depend on the orientation of $p[l], p[l + 1], p[r]$. If they are collinear, we can't make a triangle, so we simply return `(a, b->Rev)`. Otherwise, we create a new edge `c` by calling `connect(b, a)`. If the points are oriented counter-clockwise, we return `(a, b->Rev)`. Otherwise we return `(c->Rev, c)`.

Now suppose that $r - l + 1 \geq 4$. Firstly, let's solve $L = (l, \frac{l+r}{2})$ and $R = (\frac{l+r}{2} + 1, r)$ recursively. Now we have to merge these triangulations into one triangulation. Note that our points are sorted, so while merging we will add edges from L to R (so-called *cross edges*) and remove some edges from L to L and from R to R . What is the structure of the cross edges? All these edges must cross a line parallel to the y -axis and placed at the splitting x value. This establishes a linear ordering of the cross edges, so we can talk about successive cross edges, the bottom-most cross edge, etc. The algorithm will add the cross edges in ascending order. Note that any two adjacent cross edges will have a common endpoint, and the third side of the triangle they define goes from L to L or from R to R . Let's call the current cross edge the base. The successor of the base will either go from the left endpoint of the base to one of the R -neighbors of the right endpoint or vice versa. Consider the circumcircle of base and the previous cross edge. Suppose this circle is transformed into other circles having base as a chord but lying further into the Oy direction. Our circle will go up for a while, but unless base is an upper tangent of L and R we will encounter a point belonging

either to L or to R giving rise to a new triangle without any points in the circumcircle. The new L-R edge of this triangle is the next cross edge added. To do this efficiently, we compute two edges `lcand` and `rcand` so that `lcand` points to the first L point encountered in this process, and `rcand` points to the first R point. Then we choose the one that would be encountered first. Initially base points to the lower tangent of L and R.

Implementation

Note that the implementation of the `in_circle` function is GCC-specific.

```
typedef long long ll;

bool ge(const ll& a, const ll& b) { return a >= b; }
bool le(const ll& a, const ll& b) { return a <= b; }
bool eq(const ll& a, const ll& b) { return a == b; }
bool gt(const ll& a, const ll& b) { return a > b; }
bool lt(const ll& a, const ll& b) { return a < b; }
int sgn(const ll& a) { return a >= 0 ? a ? 1 : 0 : -1; }

struct pt {
    ll x, y;
    pt() {}
    pt(ll _x, ll _y) : x(_x), y(_y) {}
    pt operator-(const pt& p) const {
        return pt(x - p.x, y - p.y);
    }
    ll cross(const pt& p) const {
        return x * p.y - y * p.x;
    }
    ll cross(const pt& a, const pt& b) const {
        return (a - *this).cross(b - *this);
    }
    ll dot(const pt& p) const {
        return x * p.x + y * p.y;
    }
    ll dot(const pt& a, const pt& b) const {
        return (a - *this).dot(b - *this);
    }
    ll sqrLength() const {
        return this->dot(*this);
    }
    bool operator==(const pt& p) const {
        return eq(x, p.x) && eq(y, p.y);
    }
};
```

```

const pt inf_pt = pt(1e18, 1e18);

struct QuadEdge {
    pt origin;
    QuadEdge* rot = nullptr;
    QuadEdge* onext = nullptr;
    bool used = false;
    QuadEdge* rev() const {
        return rot->rot;
    }
    QuadEdge* lnext() const {
        return rot->rev()->onext->rot;
    }
    QuadEdge* oprev() const {
        return rot->onext->rot;
    }
    pt dest() const {
        return rev()->origin;
    }
};

QuadEdge* make_edge(pt from, pt to) {
    QuadEdge* e1 = new QuadEdge;
    QuadEdge* e2 = new QuadEdge;
    QuadEdge* e3 = new QuadEdge;
    QuadEdge* e4 = new QuadEdge;
    e1->origin = from;
    e2->origin = to;
    e3->origin = e4->origin = inf_pt;
    e1->rot = e3;
    e2->rot = e4;
    e3->rot = e2;
    e4->rot = e1;
    e1->onext = e1;
    e2->onext = e2;
    e3->onext = e4;
    e4->onext = e3;
    return e1;
}

void splice(QuadEdge* a, QuadEdge* b) {
    swap(a->onext->rot->onext, b->onext->rot->onext);
    swap(a->onext, b->onext);
}

void delete_edge(QuadEdge* e) {
    splice(e, e->oprev());
    splice(e->rev(), e->rev()->oprev());
    delete e->rev()->rot;
    delete e->rev();
}

```

```

    delete e->rot;
    delete e;
}

QuadEdge* connect(QuadEdge* a, QuadEdge* b) {
    QuadEdge* e = make_edge(a->dest(), b->origin);
    splice(e, a->lnext());
    splice(e->rev(), b);
    return e;
}

bool left_of(pt p, QuadEdge* e) {
    return gt(p.cross(e->origin, e->dest()), 0);
}

bool right_of(pt p, QuadEdge* e) {
    return lt(p.cross(e->origin, e->dest()), 0);
}

template <class T>
T det3(T a1, T a2, T a3, T b1, T b2, T b3, T c1, T c2, T c3) {
    return a1 * (b2 * c3 - c2 * b3) - a2 * (b1 * c3 - c1 * b3) +
        a3 * (b1 * c2 - c1 * b2);
}

bool in_circle(pt a, pt b, pt c, pt d) {
    // If there is __int128, calculate directly.
    // Otherwise, calculate angles.
    #if defined(__LP64__) || defined(_WIN64)
        __int128 det = -det3<__int128>(b.x, b.y, b.sqrLength(), c.x, c.y,
            c.sqrLength(), d.x, d.y,
            d.sqrLength());
        det += det3<__int128>(a.x, a.y, a.sqrLength(), c.x, c.y,
            c.sqrLength(), d.x,
            d.y, d.sqrLength());
        det -= det3<__int128>(a.x, a.y, a.sqrLength(), b.x, b.y,
            b.sqrLength(), d.x,
            d.y, d.sqrLength());
        det += det3<__int128>(a.x, a.y, a.sqrLength(), b.x, b.y,
            b.sqrLength(), c.x,
            c.y, c.sqrLength());
        return det > 0;
    #else
        auto ang = [](pt l, pt mid, pt r) {
            ll x = mid.dot(l, r);
            ll y = mid.cross(l, r);
            long double res = atan2((long double)x, (long double)y);
            return res;
        };
        long double kek = ang(a, b, c) + ang(c, d, a) - ang(b, c, d) -

```

```

ang(d, a, b);
    if (kek > 1e-8)
        return true;
    else
        return false;
#endif
}

pair<QuadEdge*, QuadEdge*> build_tr(int l, int r, vector<pt>& p) {
    if (r - l + 1 == 2) {
        QuadEdge* res = make_edge(p[l], p[r]);
        return make_pair(res, res->rev());
    }
    if (r - l + 1 == 3) {
        QuadEdge *a = make_edge(p[l], p[l + 1]), *b = make_edge(p[l +
1], p[r]);
        splice(a->rev(), b);
        int sg = sgn(p[l].cross(p[l + 1], p[r]));
        if (sg == 0)
            return make_pair(a, b->rev());
        QuadEdge* c = connect(b, a);
        if (sg == 1)
            return make_pair(a, b->rev());
        else
            return make_pair(c->rev(), c);
    }
    int mid = (l + r) / 2;
    QuadEdge *ldo, *ldi, *rdo, *rdi;
    tie(ldo, ldi) = build_tr(l, mid, p);
    tie(rdi, rdo) = build_tr(mid + 1, r, p);
    while (true) {
        if (left_of(rdi->origin, ldi)) {
            ldi = ldi->lnext();
            continue;
        }
        if (right_of(ldi->origin, rdi)) {
            rdi = rdi->rev()->onext;
            continue;
        }
        break;
    }
    QuadEdge* basel = connect(rdi->rev(), ldi);
    auto valid = [&basel](QuadEdge* e) { return right_of(e->dest(),
basel); };
    if (ldi->origin == ldo->origin)
        ldo = basel->rev();
    if (rdi->origin == rdo->origin)
        rdo = basel;
    while (true) {
        QuadEdge* lcand = basel->rev()->onext;

```

```

        if (valid(lcand)) {
            while (in_circle(basel->dest(), basel->origin, lcand->dest(),
                            lcand->onext->dest())) {
                QuadEdge* t = lcand->onext;
                delete_edge(lcand);
                lcand = t;
            }
        }
        QuadEdge* rcand = basel->oprev();
        if (valid(rcand)) {
            while (in_circle(basel->dest(), basel->origin, rcand->dest(),
                            rcand->oprev->dest())) {
                QuadEdge* t = rcand->oprev();
                delete_edge(rcand);
                rcand = t;
            }
        }
        if (!valid(lcand) && !valid(rcand))
            break;
        if (!valid(lcand) ||
            (valid(rcand) && in_circle(lcand->dest(), lcand->origin,
                                      rcand->origin, rcand->dest())))
            basel = connect(rcand, basel->rev());
        else
            basel = connect(basel->rev(), lcand->rev());
    }
    return make_pair(ldo, rdo);
}

vector<tuple<pt, pt, pt>> delaunay(vector<pt> p) {
    sort(p.begin(), p.end(), [](const pt& a, const pt& b) {
        return lt(a.x, b.x) || (eq(a.x, b.x) && lt(a.y, b.y));
    });
    auto res = build_tr(0, (int)p.size() - 1, p);
    QuadEdge* e = res.first;
    vector<QuadEdge*> edges = {e};
    while (lt(e->onext->dest().cross(e->dest(), e->origin), 0))
        e = e->onext;
    auto add = [&p, &e, &edges]() {
        QuadEdge* curr = e;
        do {
            curr->used = true;
            p.push_back(curr->origin);
            edges.push_back(curr->rev());
            curr = curr->lnext();
        } while (curr != e);
    };
    add();
}

```

```
p.clear();
int kek = 0;
while (kek < (int)edges.size()) {
    if (!(e = edges[kek++])->used)
        add();
}
vector<tuple<pt, pt, pt>> ans;
for (int i = 0; i < (int)p.size(); i += 3) {
    ans.push_back(make_tuple(p[i], p[i + 1], p[i + 2]));
}
return ans;
}
```

Problems

- [TIMUS 1504 Good Manners](#)
- [TIMUS 1520 Empire Strikes Back](#)
- [SGU 383 Caravans](#)

Contributors:

[Yury Semenov](#) (94.55%) [Oleksandr Kulkov](#) (1.92%) [Bergur Snorrason](#) (1.6%)
[Jakob Kogler](#) (0.96%) [Michael Hayter](#) (0.96%)

Translated

From: e-maxx.ru

Vertical decomposition

Overview

Vertical decomposition is a powerful technique used in various geometry problems. The general idea is to cut the plane into several vertical stripes with some "good" properties and solve the problem for these stripes independently. We will illustrate the idea on some examples.

Area of the union of triangles

Suppose that there are n triangles on a plane and we are to find the area of their union. The problem would be easy if the triangles didn't intersect, so let's get rid of these intersections by dividing the plane into vertical stripes by drawing vertical lines through all vertices and all points of intersection of sides of different triangles. There may be $O(n^2)$ such lines so we obtained $O(n^2)$ stripes. Now consider some vertical stripe. Each non-vertical segment either crosses it from left to right or doesn't cross at all. Also, no two segments intersect strictly inside the stripe. It means that the part of the union of triangles that lies inside this stripe is composed of disjoint trapezoids with bases lying on the sides of the stripe. This property allows us to compute the area inside each stripe with a following scanline algorithm. Each segment crossing the stripe is either upper or lower, depending on whether the interior of the corresponding triangle is above or below the segment. We can visualize each upper segment as an opening bracket and each lower segment as a closing bracket and decompose the stripe into trapezoids by decomposing the bracket sequence into smaller correct bracket sequences. This algorithm requires $O(n^3 \log n)$ time and $O(n^2)$ memory.

Optimization 1

Firstly we will reduce the runtime to $O(n^2 \log n)$. Instead of generating trapezoids for each stripe let's fix some triangle side (segment $s = (s_0, s_1)$) and find the set of stripes where this segment is a side of some trapezoid. Note that in this case we only have to find the stripes where the balance of brackets below (or above, in case of a lower segment) s is zero. It means that instead of running vertical scanline for each stripe we can run a horizontal scanline for all parts of other segments which affect the balance of brackets with respect to s . For simplicity we will show how to do this for an upper segment, the algorithm for lower segments is similar. Consider some other non-vertical segment $t = (t_0, t_1)$ and find the intersection $[x_1, x_2]$ of projections of s and t on Ox . If this intersection is empty or consists of one point, t can be discarded since s and t do not intersect the interior of the same stripe. Otherwise consider the intersection I of s and t . There are three cases.

1. $I = \emptyset$

In this case t is either above or below s on $[x_1, x_2]$. If t is above, it doesn't affect whether s is a side of some trapezoid or not. If t is below s , we should add 1 or -1 to the balance of bracket sequences for all stripes in $[x_1, x_2]$, depending on whether t is upper or lower.

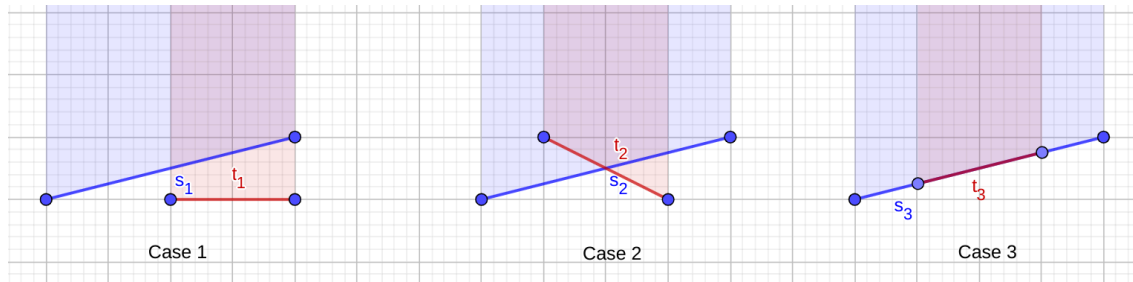
2. I consists of a single point p

This case can be reduced to the previous one by splitting $[x_1, x_2]$ into $[x_1, p_x]$ and $[p_x, x_2]$.

3. I is some segment l

This case means that the parts of s and t for $x \in [x_1, x_2]$ coincide. If t is lower, s is clearly not a side of a trapezoid. Otherwise, it could happen that both s and t can be considered as a side of some trapezoid. In order to resolve this ambiguity, we can decide that only the segment with the lowest index should be considered as a side (here we suppose that triangle sides are enumerated in some way). So, if $\text{index}(s) < \text{index}(t)$, we should ignore this case, otherwise we should mark that s can never be a side on $[x_1, x_2]$ (for example, by adding a corresponding event with balance -2).

Here is a graphic representation of the three cases.



Finally we should remark on processing all the additions of 1 or -1 on all stripes in $[x_1, x_2]$. For each addition of w on $[x_1, x_2]$ we can create events (x_1, w) , $(x_2, -w)$ and process all these events with a sweep line.

Optimization 2

Note that if we apply the previous optimization, we no longer have to find all stripes explicitly. This reduces the memory consumption to $O(n)$.

Intersection of convex polygons

Another usage of vertical decomposition is to compute the intersection of two convex polygons in linear time. Suppose the plane is split into vertical stripes by vertical lines passing through each vertex of each polygon. Then if we consider one of the input polygons and some stripe, their intersection is either a trapezoid, a triangle or a point. Therefore we can simply intersect these shapes for each vertical stripe and merge these intersections into a single polygon.

Implementation

Below is the code that calculates area of the union of a set of triangles in $O(n^2 \log n)$ time and $O(n)$ memory.

```
typedef double dbl;

const dbl eps = 1e-9;

inline bool eq(dbl x, dbl y){
    return fabs(x - y) < eps;
}
```

```

inline bool lt(dbl x, dbl y){
    return x < y - eps;
}

inline bool gt(dbl x, dbl y){
    return x > y + eps;
}

inline bool le(dbl x, dbl y){
    return x < y + eps;
}

inline bool ge(dbl x, dbl y){
    return x > y - eps;
}

struct pt{
    dbl x, y;
    inline pt operator - (const pt & p) const{
        return pt{x - p.x, y - p.y};
    }
    inline pt operator + (const pt & p) const{
        return pt{x + p.x, y + p.y};
    }
    inline pt operator * (dbl a) const{
        return pt{x * a, y * a};
    }
    inline dbl cross(const pt & p) const{
        return x * p.y - y * p.x;
    }
    inline dbl dot(const pt & p) const{
        return x * p.x + y * p.y;
    }
    inline bool operator == (const pt & p) const{
        return eq(x, p.x) && eq(y, p.y);
    }
};

struct Line{
    pt p[2];
    Line(){}
    Line(pt a, pt b):p{a, b}{}
    pt vec() const{
        return p[1] - p[0];
    }
    pt& operator [] (size_t i){
        return p[i];
    }
};

```

```

inline bool lexComp(const pt & l, const pt & r){
    if(fabs(l.x - r.x) > eps){
        return l.x < r.x;
    }
    else return l.y < r.y;
}

vector<pt> interSegSeg(Line l1, Line l2){
    if(eq(l1.vec().cross(l2.vec()), 0)){
        if(!eq(l1.vec().cross(l2[0] - l1[0]), 0))
            return {};
        if(!lexComp(l1[0], l1[1]))
            swap(l1[0], l1[1]);
        if(!lexComp(l2[0], l2[1]))
            swap(l2[0], l2[1]);
        pt l = lexComp(l1[0], l2[0]) ? l2[0] : l1[0];
        pt r = lexComp(l1[1], l2[1]) ? l1[1] : l2[1];
        if(l == r)
            return {l};
        else return lexComp(l, r) ? vector<pt>{l, r} : vector<pt>();
    }
    else{
        dbl s = (l2[0] - l1[0]).cross(l2.vec()) /
l1.vec().cross(l2.vec());
        pt inter = l1[0] + l1.vec() * s;
        if(ge(s, 0) && le(s, 1) && le((l2[0] - inter).dot(l2[1] -
inter), 0))
            return {inter};
        else
            return {};
    }
}

inline char get_segtype(Line segment, pt other_point){
    if(eq(segment[0].x, segment[1].x))
        return 0;
    if(!lexComp(segment[0], segment[1]))
        swap(segment[0], segment[1]);
    return (segment[1] - segment[0]).cross(other_point - segment[0]) > 0
? 1 : -1;
}

dbl union_area(vector<tuple<pt, pt, pt> > triangles){
    vector<Line> segments(3 * triangles.size());
    vector<char> segtype(segments.size());
    for(size_t i = 0; i < triangles.size(); i++){
        pt a, b, c;
        tie(a, b, c) = triangles[i];
        segments[3 * i] = lexComp(a, b) ? Line(a, b) : Line(b, a);
        segtype[3 * i] = get_segtype(segments[3 * i], c);
    }
}

```

```

        segments[3 * i + 1] = lexComp(b, c) ? Line(b, c) : Line(c, b);
        segtype[3 * i + 1] = get_segtype(segments[3 * i + 1], a);
        segments[3 * i + 2] = lexComp(c, a) ? Line(c, a) : Line(a, c);
        segtype[3 * i + 2] = get_segtype(segments[3 * i + 2], b);
    }
    vector<dbl> k(segments.size()), b(segments.size());
    for(size_t i = 0; i < segments.size(); i++){
        if(segtype[i]){
            k[i] = (segments[i][1].y - segments[i][0].y) / (segments[i][1].x - segments[i][0].x);
            b[i] = segments[i][0].y - k[i] * segments[i][0].x;
        }
    }
    dbl ans = 0;
    for(size_t i = 0; i < segments.size(); i++){
        if(!segtype[i])
            continue;
        dbl l = segments[i][0].x, r = segments[i][1].x;
        vector<pair<dbl, int>> evts;
        for(size_t j = 0; j < segments.size(); j++){
            if(!segtype[j] || i == j)
                continue;
            dbl l1 = segments[j][0].x, r1 = segments[j][1].x;
            if(ge(l1, r) || ge(l, r1))
                continue;
            dbl common_l = max(l, l1), common_r = min(r, r1);
            auto pts = interSegSeg(segments[i], segments[j]);
            if(pts.empty()){
                dbl y11 = k[j] * common_l + b[j];
                dbl y1 = k[i] * common_l + b[i];
                if(lt(y11, y1) == (segtype[i] == 1)){
                    int evt_type = -segtype[i] * segtype[j];
                    evts.emplace_back(common_l, evt_type);
                    evts.emplace_back(common_r, -evt_type);
                }
            }
            else if(pts.size() == 1u){
                dbl y1 = k[i] * common_l + b[i], y11 = k[j] * common_l +
b[j];

                int evt_type = -segtype[i] * segtype[j];
                if(lt(y11, y1) == (segtype[i] == 1)){
                    evts.emplace_back(common_l, evt_type);
                    evts.emplace_back(pts[0].x, -evt_type);
                }
                y1 = k[i] * common_r + b[i], y11 = k[j] * common_r +
b[j];

                if(lt(y11, y1) == (segtype[i] == 1)){
                    evts.emplace_back(pts[0].x, evt_type);
                    evts.emplace_back(common_r, -evt_type);
                }
            }
        }
    }
}

```

```

    }
    else{
        if(segtype[j] != segtype[i] || j > i){
            evts.emplace_back(common_l, -2);
            evts.emplace_back(common_r, 2);
        }
    }
}
evts.emplace_back(l, 0);
sort(evts.begin(), evts.end());
size_t j = 0;
int balance = 0;
while(j < evts.size()){
    size_t ptr = j;
    while(ptr < evts.size() && eq(evts[j].first,
evts[ptr].first)){
        balance += evts[ptr].second;
        ++ptr;
    }
    if(!balance && !eq(evts[j].first, r)){
        dbl next_x = ptr == evts.size() ? r : evts[ptr].first;
        ans -= segtype[i] * (k[i] * (next_x + evts[j].first) + 2
* b[i]) * (next_x - evts[j].first);
    }
    j = ptr;
}
}
return ans/2;
}

```

Problems

- [Codeforces 62C Inquisition](#)
- [Codeforces 107E Darts](#)

Contributors:

[Yury Semenov](#) (95.49%)
 [Oleksandr Kulkov](#) (2.46%)
 [Michael Hayter](#) (1.23%)
[Jakob Kogler](#) (0.82%)

Original

Half-plane intersection

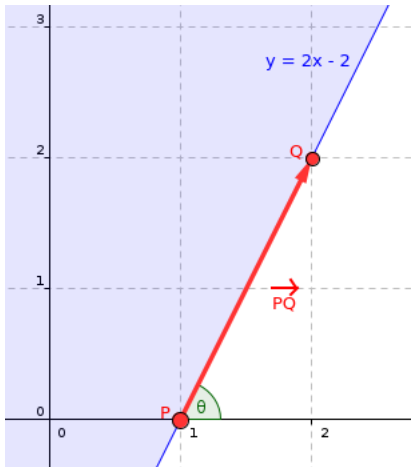
In this article we will discuss the problem of computing the intersection of a set of half-planes. Such an intersection can be conveniently represented as a convex region/polygon, where every point inside of it is also inside all of the half-planes, and it is this polygon that we're trying to find or construct. We give some initial intuition for the problem, describe a $O(N \log N)$ approach known as the Sort-and-Incremental algorithm and give some sample applications of this technique.

It is strongly recommended for the reader to be familiar with basic geometrical primitives and operations (points, vectors, intersection of lines). Additionally, knowledge about [Convex Hulls](#) or the [Convex Hull Trick](#) may help to better understand the concepts in this article, but they are not a prerequisite by any means.

Initial clarifications and definitions

For the entire article, we will make some assumptions (unless specified otherwise):

1. We define N to be the quantity of half-planes in the given set.
2. We will represent lines and half-planes by one point and one vector (any point that lies on the given line, and the direction vector of the line). In the case of half-planes, we assume that every half-plane allows the region to the left side of its direction vector. Additionally, we define the angle of a half-plane to be the polar angle of its direction vector. See image below for example.
3. We will assume that the resulting intersection is always either bounded or empty. If we need to handle the unbounded case, we can simply add 4 half-planes that define a large-enough bounding box.
4. We will assume, for simplicity, that there are no parallel half-planes in the given set. Towards the end of the article we will discuss how to deal with such cases.



The half-plane $y \geq 2x - 2$ can be represented as the point $P = (1, 0)$ with direction vector $PQ = Q - P = (1, 2)$

Brute force approach - $O(N^3)$

One of the most straightforward and obvious solutions would be to compute the intersection point of the lines of all pairs of half-planes and, for each point, check if it is inside all of the other half-planes. Since there are $O(N^2)$ intersection points, and for each of them we have to check $O(N)$ half-planes, the total time complexity is $O(N^3)$. The actual region of the intersection can then be reconstructed using, for example, a Convex Hull algorithm on the set of intersection points that were included in all the half-planes.

It is fairly easy to see why this works: the vertices of the resulting convex polygon are all intersection points of the half-plane lines, and each of those vertices is obviously part of all the half-planes. The main advantage of this method is that it's easy to understand, remember and code on-the-fly if you just need to check if the intersection is empty or not. However, it is awfully slow and unfit for most problems, so we need something faster.

Incremental approach - $O(N^2)$

Another fairly straightforward approach is to incrementally construct the intersection of the half-planes, one at a time. This method is basically equivalent to cutting a convex

polygon by a line N times, and removing the redundant half-planes at every step. To do this, we can represent the convex polygon as a list of line segments, and to cut it with a half-plane we simply find the intersection points of the segments with the half-plane line (there will only be two intersection points if the line properly intersects the polygon), and replace all the line segments in-between with the new segment corresponding to the half-plane. Since such procedure can be implemented in linear time, we can simply start with a big bounding box and cut it down with each one of the half-planes, obtaining a total time complexity of $O(N^2)$.

This method is a big step in the right direction, but it does feel wasteful to have to iterate over $O(N)$ half-planes at every step. We will see next that, by making some clever observations, the ideas behind this incremental approach can be recycled to create a $O(N \log N)$ algorithm.

Sort-and-Incremental algorithm - $O(N \log N)$

The first properly-documented source of this algorithm we could find was Zeyuan Zhu's thesis for Chinese Team Selecting Contest titled [New Algorithm for Half-plane Intersection and its Practical Value](#), from the year 2006. The approach we'll describe next is based on this same algorithm, but instead of computing two separate intersections for the lower and upper halves of the intersections, we'll construct it all at once in one pass with a deque (double-ended queue).

The algorithm itself, as the name may spoil, takes advantage of the fact that the resulting region from the intersection of half-planes is convex, and thus it will consist of some segments of half-planes in order sorted by their angles. This leads to a crucial observation: if we incrementally intersect the half-planes in their order sorted by angle (as they would appear in the final, resulting shape of the intersection) and store them in a double-ended queue, then we will only ever need to remove half-planes from the front and the back of the deque.

To better visualize this fact, suppose we're performing the incremental approach described previously on a set of half-planes that is sorted by angle (in this case, we'll assume they're sorted from $-\pi$ to π), and suppose that we're about to start some arbitrary k 'th step. This means we have already constructed the intersection of the first $k - 1$ half-planes. Now, because the half-planes are sorted by angle, whatever the k 'th

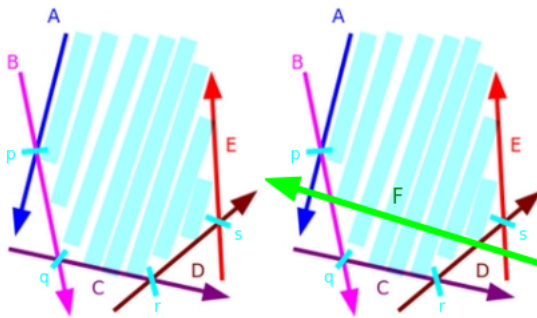
half-plane is, we can be sure that it will form a convex turn with the $(K - 1)$ 'th half-plane. For that reason, a few things may happen:

1. Some (possibly none) of the half-planes in the back of the intersection may become *redundant*. In this case, we need to pop these now-useless half-planes from the back of the deque.
2. Some (possibly none) of the half-planes at the front may become *redundant*. Analogous to case 1, we just pop them from the front of the deque.
3. The intersection may become empty (after handling cases 1 and/or 2). In this case, we just report the intersection is empty and terminate the algorithm.

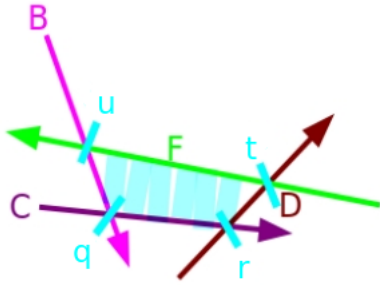
We say a half-plane is "redundant" if it does not contribute anything to the intersection. Such a half-plane could be removed and the resulting intersection would not change at all.

Here's a small example with an illustration:

Let $H = \{A, B, C, D, E\}$ be the set of half-planes currently present in the intersection. Additionally, let $P = \{p, q, r, s\}$ be the set of intersection points of adjacent half-planes in H . Now, suppose we wish to intersect it with the half-plane F , as seen in the illustration below:



Notice the half-plane F makes A and E redundant in the intersection. So we remove both A and E from the front and back of the intersection, respectively, and add F at the end. And we finally obtain the new intersection $H = \{B, C, D, F\}$ with $P = \{q, r, t, u\}$.



With all of this in mind, we have almost everything we need to actually implement the algorithm, but we still need to talk about some special cases. At the beginning of the article we said we would add a bounding box to take care of the cases where the intersection could be unbounded, so the only tricky case we actually need to handle is parallel half-planes. We can have two sub-cases: two half-planes can be parallel with the same direction or with opposite direction. The reason this case needs to be handled separately is because we will need to compute intersection points of half-plane lines to be able to check if a half-plane is redundant or not, and two parallel lines have no intersection point, so we need a special way to deal with them.

For the case of parallel half-planes of opposite orientation: Notice that, because we're adding the bounding box to deal with the unbounded case, this also deals with the case where we have two adjacent parallel half-planes with opposite directions after sorting, since there will have to be at least one of the bounding-box half-planes in between these two (remember they are sorted by angle).

- However, it is possible that, after removing some half-planes from the back of the deque, two parallel half-planes of opposite direction end up together. This case only happens, specifically, when these two half-planes form an empty intersection, as this last half-plane will cause everything to be removed from the deque. To avoid this problem, we have to manually check for parallel half-planes, and if they have opposite direction, we just instantly stop the algorithm and return an empty intersection.

Thus the only case we actually need to handle is having multiple half-planes with the same angle, and it turns out this case is fairly easy to handle: we only have keep the leftmost half-plane and erase the rest, since they will be completely redundant anyways. To sum up, the full algorithm will roughly look as follows:

1. We begin by sorting the set of half-planes by angle, which takes $O(N \log N)$ time.

2. We will iterate over the set of half-planes, and for each one, we will perform the incremental procedure, popping from the front and the back of the double-ended queue as necessary. This will take linear time in total, as every half-plane can only be added or removed once.
3. At the end, the convex polygon resulting from the intersection can be simply obtained by computing the intersection points of adjacent half-planes in the deque at the end of the procedure. This will take linear time as well. It is also possible to store such points during step 2 and skip this step entirely, but we believe it is slightly easier (in terms of implementation) to compute them on-the-fly.

In total, we have achieved a time complexity of $O(N \log N)$. Since sorting is clearly the bottleneck, the algorithm can be made to run in linear time in the special case where we are given half-planes sorted in advance by their angles (an example of such a case would be obtaining the half-planes that define a convex polygon).

Direct implementation

Here is a sample, direct implementation of the algorithm, with comments explaining most parts:

Simple point/vector and half-plane structs:

```
// Redefine epsilon and infinity as necessary. Be mindful of precision
errors.
const long double eps = 1e-9, inf = 1e9;

// Basic point/vector struct.
struct Point {

    long double x, y;
    explicit Point(long double x = 0, long double y = 0) : x(x), y(y) {}

    // Addition, subtraction, multiply by constant, dot product, cross
    product.

    friend Point operator + (const Point& p, const Point& q) {
        return Point(p.x + q.x, p.y + q.y);
    }

    friend Point operator - (const Point& p, const Point& q) {
        return Point(p.x - q.x, p.y - q.y);
    }
}
```

```

    friend Point operator * (const Point& p, const long double& k) {
        return Point(p.x * k, p.y * k);
    }

    friend long double dot(const Point& p, const Point& q) {
        return p.x * q.x + p.y * q.y;
    }

    friend long double cross(const Point& p, const Point& q) {
        return p.x * q.y - p.y * q.x;
    }
};

// Basic half-plane struct.
struct Halfplane {

    // 'p' is a passing point of the line and 'pq' is the direction
    // vector of the line.
    Point p, pq;
    long double angle;

    Halfplane() {}
    Halfplane(const Point& a, const Point& b) : p(a), pq(b - a) {
        angle = atan2l(pq.y, pq.x);
    }

    // Check if point 'r' is outside this half-plane.
    // Every half-plane allows the region to the LEFT of its line.
    bool out(const Point& r) {
        return cross(pq, r - p) < -eps;
    }

    // Comparator for sorting.
    bool operator < (const Halfplane& e) const {
        return angle < e.angle;
    }

    // Intersection point of the lines of two half-planes. It is assumed
    // they're never parallel.
    friend Point inter(const Halfplane& s, const Halfplane& t) {
        long double alpha = cross((t.p - s.p), t.pq) / cross(s.pq,
t.pq);
        return s.p + (s.pq * alpha);
    }
};

```

Algorithm:

```

// Actual algorithm
vector<Point> hp_intersect(vector<Halfplane>& H) {

    Point box[4] = { // Bounding box in CCW order
        Point(1e18, 1e18),
        Point(-1e18, 1e18),
        Point(-1e18, -1e18),
        Point(1e18, -1e18)
    };

    for(int i = 0; i < 4; i++) { // Add bounding box half-planes.
        Halfplane aux(box[i], box[(i+1) % 4]);
        H.push_back(aux);
    }

    // Sort by angle and start algorithm
    sort(H.begin(), H.end());
    deque<Halfplane> dq;
    int len = 0;
    for(int i = 0; i < int(H.size()); i++) {

        // Remove from the back of the deque while last half-plane is
        // redundant
        while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2]))) {
            dq.pop_back();
            --len;
        }

        // Remove from the front of the deque while first half-plane is
        // redundant
        while (len > 1 && H[i].out(inter(dq[0], dq[1]))) {
            dq.pop_front();
            --len;
        }

        // Special case check: Parallel half-planes
        if (len > 0 && fabs1(cross(H[i].pq, dq[len-1].pq)) < eps) {
            // Opposite parallel half-planes that ended up checked
            // against each other.
            if (dot(H[i].pq, dq[len-1].pq) < 0.0)
                return vector<Point>();

            // Same direction half-plane: keep only the leftmost half-
            // plane.
            if (H[i].out(dq[len-1].p)) {
                dq.pop_back();
                --len;
            }
            else continue;
        }
    }
}

```

```

    }

    // Add new half-plane
    dq.push_back(H[i]);
    ++len;
}

// Final cleanup: Check half-planes at the front against the back
and vice-versa
while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2]))) {
    dq.pop_back();
    --len;
}

while (len > 2 && dq[len-1].out(inter(dq[0], dq[1]))) {
    dq.pop_front();
    --len;
}

// Report empty intersection if necessary
if (len < 3) return vector<Point>();

// Reconstruct the convex polygon from the remaining half-planes.
vector<Point> ret(len);
for(int i = 0; i+1 < len; i++) {
    ret[i] = inter(dq[i], dq[i+1]);
}
ret.back() = inter(dq[len-1], dq[0]);
return ret;
}

```

Implementation discussion

A special thing to note is that, in case there multiple half-planes that intersect at the same point, then this algorithm could return repeated adjacent points in the final polygon. However, this should not have any impact on judging correctly whether the intersection is empty or not, and it does not affect the polygon area at all either. You may want to remove these duplicates depending on what tasks you need to do after. You can do this very easily with `std::unique`. We want to keep the repeat points during the execution of the algorithm so that the intersections with area equal to zero can be computed correctly (for example, intersections that consist of a single point, line or line-segment). I encourage the reader to test some small hand-made cases where the intersection results in a single point or line.

One more thing that should be talked about is what to do if we are given half-planes in the form of a linear constraint (for example, $ax + by + c \leq 0$). In such case, there are two options. You can either implement the algorithm with the corresponding modifications to work with such representation (essentially create your own half-plane struct, should be fairly straightforward if you're familiar with the convex hull trick), or you can transform the lines into the representation we used in this article by taking any 2 points of each line. In general, it is recommended to work with the representation that you're given in the problem to avoid additional precision issues.

Problems, tasks and applications

Many problems that can be solved with half-plane intersection can also be solved without it, but with (usually) more complicated or uncommon approaches. Generally, half-plane intersection can appear when dealing with problems related to polygons (mostly convex), visibility in the plane and two-dimensional linear programming. Here are some sample tasks that can be solved with this technique:

Convex polygon intersection

One of the classical applications of half-plane intersection: Given N polygons, compute the region that is included inside all of the polygons.

Since the intersection of a set of half-planes is a convex polygon, we can also represent a convex polygon as a set of half-planes (every edge of the polygon is a segment of a half-plane). Generate these half-planes for every polygon and compute the intersection of the whole set. The total time complexity is $O(S \log S)$, where S is the total number of sides of all the polygons. The problem can also theoretically be solved in $O(S \log N)$ by merging the N sets of half-planes using a heap and then running the algorithm without the sorting step, but such solution has much worse constant factor than straightforward sorting and only provides minor speed gains for very small N .

Visibility in the plane

Problems that require something among the lines of "determine if some line segments are visible from some point(s) in the plane" can usually be formulated as half-plane intersection problems. Take, for example, the following task: Given some simple

polygon (not necessarily convex), determine if there's any point inside the polygon such that the whole boundary of the polygon can be observed from that point. This is also known as finding the [kernel of a polygon](#) and can be solved by simple half-plane intersection, taking each edge of the polygon as a half-plane and then computing its intersection.

Here's a related, more interesting problem that was presented by Artem Vasilyev in one of his [Brazilian ICPC Summer School lectures](#): Given a set p of points $p_1, p_2 \dots p_n$ in the plane, determine if there's any point q you can stand at such that you can see all the points of p from left to right in increasing order of their index.

Such problem can be solved by noticing that being able to see some point p_i to the left of p_j is the same as being able to see the right side of the line segment from p_i to p_j (or equivalently, being able to see the left side of the segment from p_j to p_i). With that in mind, we can simply create a half-plane for every line segment $p_i p_{i+1}$ (or $p_{i+1} p_i$ depending on the orientation you choose) and check if the intersection of the whole set is empty or not.

Half-plane intersection with binary search

Another common application is utilizing half-plane intersection as a tool to validate the predicate of a binary search procedure. Here's an example of such a problem, also presented by Artem Vasilyev in the same lecture that was previously mentioned: Given a **convex** polygon P , find the biggest circumference that can be inscribed inside of it.

Instead of looking for some sort of closed-form solution, annoying formulas or obscure algorithmic solutions, let's instead try to binary search on the answer. Notice that, for some fixed r , a circle with radius r can be inscribed inside P only if there exists some point inside P that has distance greater or equal than r to all the points of the boundary of P . This condition can be validated by "shrinking" the polygon inwards by a distance of r and checking that the polygon remains non-degenerate (or is a point/segment itself). Such procedure can be simulated by taking the half-planes of the polygon sides in counter-clockwise order, translating each of them by a distance of r in the direction of the region they allow (that is, orthogonal to the direction vector of the half-plane), and checking if the intersection is not empty.

Clearly, if we can inscribe a circle of radius r , we can also inscribe any other circle of radius smaller than r . So we can perform a binary search on the radius r and validate

every step using half-plane intersection. Also, note that the half-planes of a convex polygon are already sorted by angle, so the sorting step can be skipped in the algorithm. Thus we obtain a total time complexity of $O(NK)$, where N is the number of polygon vertices and K is the number of iterations of the binary search (the actual value will depend on the range of possible answers and the desired precision).

Two-dimensional linear programming

One more application of half-plane intersection is linear programming in two variables. All linear constraints for two variables can be expressed in the form of

$Ax + By + C \leq 0$ (inequality comparator may vary). Clearly, these are just half-planes, so checking if a feasible solution exists for a set of linear constraints can be done with half-plane intersection. Additionally, for a given set of linear constraints, it is possible to compute the region of feasible solutions (i.e. the intersection of the half-planes) and then answer multiple queries of maximizing/minimizing some linear function $f(x, y)$ subject to the constraints in $O(\log N)$ per query using binary search (very similar to the convex hull trick).

It is worth mentioning that there also exists a fairly simple randomized algorithm that can check whether a set of linear constraints has a feasible solution or not, and maximize/minimize some linear function subject to the given constraints. This randomized algorithm was also explained nicely by Artem Vasilyev in the lecture mentioned earlier. Here are some additional resources on it, should the reader be interested: [CG - Lecture 4, parts 4 and 5](#) and [Petr Mitrichev's blog \(which includes the solution to the hardest problem in the practice problems list below\)](#).

Classic problems, direct application

- [Codechef - Animesh decides to settle down](#)
- [POJ - How I mathematician Wonder What You Are!](#)
- [POJ - Rotating Scoreboard](#)
- [POJ - Video Surveillance](#)
- [POJ - Art Gallery](#)
- [POJ - Uyuw's Concert](#)

Harder problems

- [POJ - Most Distant Point from the Sea](#) - Medium
- [Baekjoon - Jeju's Island](#) - Same as above but seemingly stronger test cases
- [POJ - Feng Shui](#) - Medium
- [POJ - Triathlon](#) - Medium/hard
- [DMOJ - Arrow](#) - Medium/hard
- [POJ - Jungle Outpost](#) - Hard
- [Codeforces - Jungle Outpost \(alternative link, problem J\)](#) - Hard
- [Yandex - Asymmetry Value \(need virtual contest to see, problem F\)](#) - Very Hard

Additional problems

- 40th Petrozavodsk Programming Camp, Winter 2021 - Day 1: Jagiellonian U Contest, Grand Prix of Krakow - Problem B: (Almost) Fair Cake-Cutting. At the time of writing the article, this problem was private and only accessible by participants of the Programming Camp.

References, bibliography and other sources

Main sources

- [New Algorithm for Half-plane Intersection and its Practical Value](#). Original paper of the algorithm.
- [Artem Vasilyev's Brazilian ICPC Summer School 2020 lecture](#). Amazing lecture on half-plane intersection. Also covers other geometry topics.

Good blogs (Chinese)

- [Fundamentals of Computational Geometry - Intersection of Half-planes](#).
- [Detailed introduction to the half-plane intersection algorithm](#).
- [Summary of Half-plane intersection problems](#).
- [Sorting incremental method of half-plane intersection](#).

Randomized algorithm

- [Linear Programming and Half-Plane intersection - Parts 4 and 5.](#)
- [Petr Mitrichev's Blog: A half-plane week.](#)

Contributors:

[oscarburga](#) (95.15%) [Jakob Kogler](#) (2.27%) [Oleksandr Kulkov](#) (1.62%)
[Bergur Snorrason](#) (0.65%) [LUTLJS](#) (0.32%)

Original

Half-plane intersection

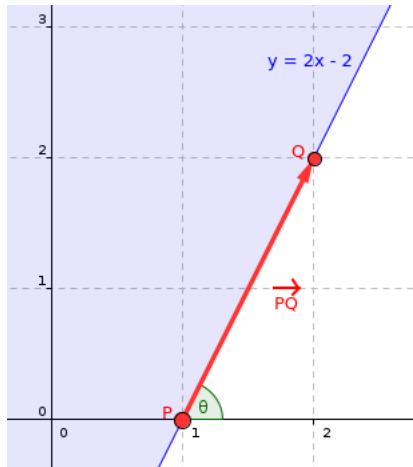
In this article we will discuss the problem of computing the intersection of a set of half-planes. Such an intersection can be conveniently represented as a convex region/polygon, where every point inside of it is also inside all of the half-planes, and it is this polygon that we're trying to find or construct. We give some initial intuition for the problem, describe a $O(N \log N)$ approach known as the Sort-and-Incremental algorithm and give some sample applications of this technique.

It is strongly recommended for the reader to be familiar with basic geometrical primitives and operations (points, vectors, intersection of lines). Additionally, knowledge about [Convex Hulls](#) or the [Convex Hull Trick](#) may help to better understand the concepts in this article, but they are not a prerequisite by any means.

Initial clarifications and definitions

For the entire article, we will make some assumptions (unless specified otherwise):

1. We define N to be the quantity of half-planes in the given set.
2. We will represent lines and half-planes by one point and one vector (any point that lies on the given line, and the direction vector of the line). In the case of half-planes, we assume that every half-plane allows the region to the left side of its direction vector. Additionally, we define the angle of a half-plane to be the polar angle of its direction vector. See image below for example.
3. We will assume that the resulting intersection is always either bounded or empty. If we need to handle the unbounded case, we can simply add 4 half-planes that define a large-enough bounding box.
4. We will assume, for simplicity, that there are no parallel half-planes in the given set. Towards the end of the article we will discuss how to deal with such cases.



The half-plane $y \geq 2x - 2$ can be represented as the point $P = (1, 0)$ with direction vector $PQ = Q - P = (1, 2)$

Brute force approach - $O(N^3)$

One of the most straightforward and obvious solutions would be to compute the intersection point of the lines of all pairs of half-planes and, for each point, check if it is inside all of the other half-planes. Since there are $O(N^2)$ intersection points, and for each of them we have to check $O(N)$ half-planes, the total time complexity is $O(N^3)$. The actual region of the intersection can then be reconstructed using, for example, a Convex Hull algorithm on the set of intersection points that were included in all the half-planes.

It is fairly easy to see why this works: the vertices of the resulting convex polygon are all intersection points of the half-plane lines, and each of those vertices is obviously part of all the half-planes. The main advantage of this method is that it's easy to understand, remember and code on-the-fly if you just need to check if the intersection is empty or not. However, it is awfully slow and unfit for most problems, so we need something faster.

Incremental approach - $O(N^2)$

Another fairly straightforward approach is to incrementally construct the intersection of the half-planes, one at a time. This method is basically equivalent to cutting a convex

polygon by a line N times, and removing the redundant half-planes at every step. To do this, we can represent the convex polygon as a list of line segments, and to cut it with a half-plane we simply find the intersection points of the segments with the half-plane line (there will only be two intersection points if the line properly intersects the polygon), and replace all the line segments in-between with the new segment corresponding to the half-plane. Since such procedure can be implemented in linear time, we can simply start with a big bounding box and cut it down with each one of the half-planes, obtaining a total time complexity of $O(N^2)$.

This method is a big step in the right direction, but it does feel wasteful to have to iterate over $O(N)$ half-planes at every step. We will see next that, by making some clever observations, the ideas behind this incremental approach can be recycled to create a $O(N \log N)$ algorithm.

Sort-and-Incremental algorithm - $O(N \log N)$

The first properly-documented source of this algorithm we could find was Zeyuan Zhu's thesis for Chinese Team Selecting Contest titled [New Algorithm for Half-plane Intersection and its Practical Value](#), from the year 2006. The approach we'll describe next is based on this same algorithm, but instead of computing two separate intersections for the lower and upper halves of the intersections, we'll construct it all at once in one pass with a deque (double-ended queue).

The algorithm itself, as the name may spoil, takes advantage of the fact that the resulting region from the intersection of half-planes is convex, and thus it will consist of some segments of half-planes in order sorted by their angles. This leads to a crucial observation: if we incrementally intersect the half-planes in their order sorted by angle (as they would appear in the final, resulting shape of the intersection) and store them in a double-ended queue, then we will only ever need to remove half-planes from the front and the back of the deque.

To better visualize this fact, suppose we're performing the incremental approach described previously on a set of half-planes that is sorted by angle (in this case, we'll assume they're sorted from $-\pi$ to π), and suppose that we're about to start some arbitrary k 'th step. This means we have already constructed the intersection of the first $k - 1$ half-planes. Now, because the half-planes are sorted by angle, whatever the k 'th

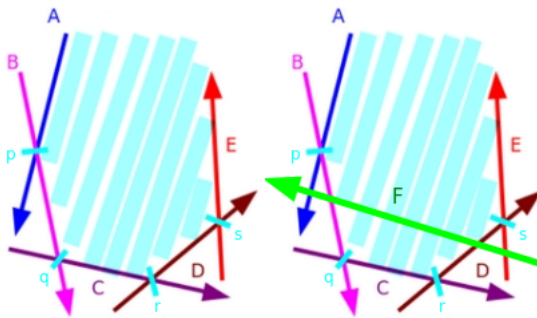
half-plane is, we can be sure that it will form a convex turn with the $(K - 1)$ 'th half-plane. For that reason, a few things may happen:

1. Some (possibly none) of the half-planes in the back of the intersection may become *redundant*. In this case, we need to pop these now-useless half-planes from the back of the deque.
2. Some (possibly none) of the half-planes at the front may become *redundant*. Analogous to case 1, we just pop them from the front of the deque.
3. The intersection may become empty (after handling cases 1 and/or 2). In this case, we just report the intersection is empty and terminate the algorithm.

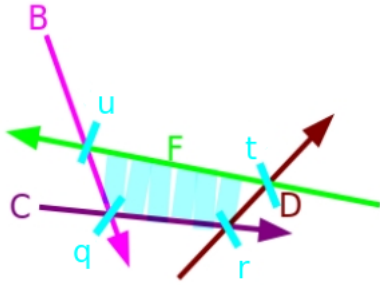
We say a half-plane is "redundant" if it does not contribute anything to the intersection. Such a half-plane could be removed and the resulting intersection would not change at all.

Here's a small example with an illustration:

Let $H = \{A, B, C, D, E\}$ be the set of half-planes currently present in the intersection. Additionally, let $P = \{p, q, r, s\}$ be the set of intersection points of adjacent half-planes in H . Now, suppose we wish to intersect it with the half-plane F , as seen in the illustration below:



Notice the half-plane F makes A and E redundant in the intersection. So we remove both A and E from the front and back of the intersection, respectively, and add F at the end. And we finally obtain the new intersection $H = \{B, C, D, F\}$ with $P = \{q, r, t, u\}$.



With all of this in mind, we have almost everything we need to actually implement the algorithm, but we still need to talk about some special cases. At the beginning of the article we said we would add a bounding box to take care of the cases where the intersection could be unbounded, so the only tricky case we actually need to handle is parallel half-planes. We can have two sub-cases: two half-planes can be parallel with the same direction or with opposite direction. The reason this case needs to be handled separately is because we will need to compute intersection points of half-plane lines to be able to check if a half-plane is redundant or not, and two parallel lines have no intersection point, so we need a special way to deal with them.

For the case of parallel half-planes of opposite orientation: Notice that, because we're adding the bounding box to deal with the unbounded case, this also deals with the case where we have two adjacent parallel half-planes with opposite directions after sorting, since there will have to be at least one of the bounding-box half-planes in between these two (remember they are sorted by angle).

- However, it is possible that, after removing some half-planes from the back of the deque, two parallel half-planes of opposite direction end up together. This case only happens, specifically, when these two half-planes form an empty intersection, as this last half-plane will cause everything to be removed from the deque. To avoid this problem, we have to manually check for parallel half-planes, and if they have opposite direction, we just instantly stop the algorithm and return an empty intersection.

Thus the only case we actually need to handle is having multiple half-planes with the same angle, and it turns out this case is fairly easy to handle: we only have keep the leftmost half-plane and erase the rest, since they will be completely redundant anyways. To sum up, the full algorithm will roughly look as follows:

1. We begin by sorting the set of half-planes by angle, which takes $O(N \log N)$ time.

2. We will iterate over the set of half-planes, and for each one, we will perform the incremental procedure, popping from the front and the back of the double-ended queue as necessary. This will take linear time in total, as every half-plane can only be added or removed once.
3. At the end, the convex polygon resulting from the intersection can be simply obtained by computing the intersection points of adjacent half-planes in the deque at the end of the procedure. This will take linear time as well. It is also possible to store such points during step 2 and skip this step entirely, but we believe it is slightly easier (in terms of implementation) to compute them on-the-fly.

In total, we have achieved a time complexity of $O(N \log N)$. Since sorting is clearly the bottleneck, the algorithm can be made to run in linear time in the special case where we are given half-planes sorted in advance by their angles (an example of such a case would be obtaining the half-planes that define a convex polygon).

Direct implementation

Here is a sample, direct implementation of the algorithm, with comments explaining most parts:

Simple point/vector and half-plane structs:

```
const long double eps = 1e-8;
const long double inf = 1e6; // Modify as needed based on input scale.

// Basic point/vector struct.
struct Point {
    long double x, y;
    explicit Point(long double x = 0, long double y = 0) : x(x), y(y) {}

    // Addition, subtraction, multiply by constant, dot product, cross
    product.
    friend Point operator + (const Point& p, const Point& q) {
        return Point(p.x + q.x, p.y + q.y);
    }

    friend Point operator - (const Point& p, const Point& q) {
        return Point(p.x - q.x, p.y - q.y);
    }

    friend Point operator * (const Point& p, const long double& k) {
        return Point(p.x * k, p.y * k);
    }
}
```

```

    }

    friend long double dot(const Point& p, const Point& q) {
        return p.x * q.x + p.y * q.y;
    }

    friend long double cross(const Point& p, const Point& q) {
        return p.x * q.y - p.y * q.x;
    }
};

// Basic half-plane struct.
struct Halfplane {
    Point p, pq; // 'p' is a passing point of the line, 'pq' is the
    direction vector of the line.
    long double angle;

    Halfplane() {}
    Halfplane(const Point& a, const Point& b) : p(a), pq(b - a) {
        angle = atan2l(pq.y, pq.x);
    }

    // Check if point 'r' is outside this half-plane.
    // Every half-plane allows the region to the LEFT of its line.
    bool out(const Point& r) {
        return cross(pq, r - p) < -eps;
    }

    // Comparator for sorting.
    bool operator < (const Halfplane& e) const {
        return angle < e.angle;
    }

    // Intersection point of the lines of two half-planes. It is assumed
    they're never parallel.
    friend Point inter(const Halfplane& s, const Halfplane& t) {
        long double alpha = cross((t.p - s.p), t.pq) / cross(s.pq,
t.pq);
        return s.p + (s.pq * alpha);
    }
};

// Function to calculate a dynamic bounding box based on input points.
Point calculate_bounding_box(const std::vector<Point>& points) {
    long double maxX = -inf, maxY = -inf;
    long double minX = inf, minY = inf;

    // Traverse all points to find the bounding box
    for (const Point& pt : points) {
        if (pt.x > maxX) maxX = pt.x;

```

```

        if (pt.x < minX) minX = pt.x;
        if (pt.y > maxY) maxY = pt.y;
        if (pt.y < minY) minY = pt.y;
    }

    // Use the most extreme points to define the bounding box
    Point bbox((maxX - minX) * 2, (maxY - minY) * 2); // Making the box
    larger for stability
    return bbox;
}

```

Algorithm:

```

// Actual Algorithm
vector<Point> hp_intersect(vector<Halfplane>& H) {

    Point box[4] = { // Bounding box in CCW order
        Point(Inf, Inf),
        Point(-Inf, Inf),
        Point(-Inf, -Inf),
        Point(Inf, -Inf)
    };

    for(int i = 0; i < 4; i++) { // Add bounding box half-planes.
        Halfplane aux(box[i], box[(i+1) % 4]);
        H.push_back(aux);
    }

    // Sort by angle and start algorithm
    sort(H.begin(), H.end());
    deque<Halfplane> dq;
    int len = 0;

    for(int i = 0; i < int(H.size()); i++) {

        // Remove from the back of the deque while the last half-plane
        is redundant
        while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2]))) {
            dq.pop_back();
            --len;
        }

        // Remove from the front of the deque while the first half-plane
        is redundant
        while (len > 1 && H[i].out(inter(dq[0], dq[1]))) {
            dq.pop_front();
            --len;
        }
    }
}

```

```

// Special case: Check for parallel half-planes
if (len > 0 && fabs1(cross(H[i].pq, dq[len-1].pq)) < eps) {
    // Opposite parallel half-planes
    if (dot(H[i].pq, dq[len-1].pq) < 0.0)
        return vector<Point>();

    // Keep only the leftmost half-plane for same direction
    if (H[i].out(dq[len-1].p)) {
        dq.pop_back();
        --len;
    }
    else continue;
}

// Add the new half-plane
dq.push_back(H[i]);
++len;
}

// Final cleanup: Check half-planes at the front and back
while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2]))) {
    dq.pop_back();
    --len;
}

while (len > 2 && dq[len-1].out(inter(dq[0], dq[1]))) {
    dq.pop_front();
    --len;
}

// If less than 3 planes, report empty intersection
if (len < 3) return vector<Point>();

// Reconstruct the convex polygon from the remaining half-planes
vector<Point> ret(len);
for (int i = 0; i+1 < len; i++) {
    ret[i] = inter(dq[i], dq[i+1]);
}
ret.back() = inter(dq[len-1], dq[0]);
return ret;
}

```

Implementation discussion

A special thing to note is that, in case there multiple half-planes that intersect at the same point, then this algorithm could return repeated adjacent points in the final polygon. However, this should not have any impact on judging correctly whether the

intersection is empty or not, and it does not affect the polygon area at all either. You may want to remove these duplicates depending on what tasks you need to do after. You can do this very easily with `std::unique`. We want to keep the repeat points during the execution of the algorithm so that the intersections with area equal to zero can be computed correctly (for example, intersections that consist of a single point, line or line-segment). I encourage the reader to test some small hand-made cases where the intersection results in a single point or line.

One more thing that should be talked about is what to do if we are given half-planes in the form of a linear constraint (for example, $ax + by + c \leq 0$). In such case, there are two options. You can either implement the algorithm with the corresponding modifications to work with such representation (essentially create your own half-plane struct, should be fairly straightforward if you're familiar with the convex hull trick), or you can transform the lines into the representation we used in this article by taking any 2 points of each line. In general, it is recommended to work with the representation that you're given in the problem to avoid additional precision issues.

Problems, tasks and applications

Many problems that can be solved with half-plane intersection can also be solved without it, but with (usually) more complicated or uncommon approaches. Generally, half-plane intersection can appear when dealing with problems related to polygons (mostly convex), visibility in the plane and two-dimensional linear programming. Here are some sample tasks that can be solved with this technique:

Convex polygon intersection

One of the classical applications of half-plane intersection: Given N polygons, compute the region that is included inside all of the polygons.

Since the intersection of a set of half-planes is a convex polygon, we can also represent a convex polygon as a set of half-planes (every edge of the polygon is a segment of a half-plane). Generate these half-planes for every polygon and compute the intersection of the whole set. The total time complexity is $O(S \log S)$, where S is the total number of sides of all the polygons. The problem can also theoretically be solved in $O(S \log N)$ by merging the N sets of half-planes using a heap and then running the

algorithm without the sorting step, but such solution has much worse constant factor than straightforward sorting and only provides minor speed gains for very small N .

Visibility in the plane

Problems that require something among the lines of "determine if some line segments are visible from some point(s) in the plane" can usually be formulated as half-plane intersection problems. Take, for example, the following task: Given some simple polygon (not necessarily convex), determine if there's any point inside the polygon such that the whole boundary of the polygon can be observed from that point. This is also known as finding the [kernel of a polygon](#) and can be solved by simple half-plane intersection, taking each edge of the polygon as a half-plane and then computing its intersection.

Here's a related, more interesting problem that was presented by Artem Vasilyev in one of his [Brazilian ICPC Summer School lectures](#): Given a set p of points $p_1, p_2 \dots p_n$ in the plane, determine if there's any point q you can stand at such that you can see all the points of p from left to right in increasing order of their index.

Such problem can be solved by noticing that being able to see some point p_i to the left of p_j is the same as being able to see the right side of the line segment from p_i to p_j (or equivalently, being able to see the left side of the segment from p_j to p_i). With that in mind, we can simply create a half-plane for every line segment $p_i p_{i+1}$ (or $p_{i+1} p_i$ depending on the orientation you choose) and check if the intersection of the whole set is empty or not.

Half-plane intersection with binary search

Another common application is utilizing half-plane intersection as a tool to validate the predicate of a binary search procedure. Here's an example of such a problem, also presented by Artem Vasilyev in the same lecture that was previously mentioned: Given a **convex** polygon P , find the biggest circumference that can be inscribed inside of it.

Instead of looking for some sort of closed-form solution, annoying formulas or obscure algorithmic solutions, let's instead try to binary search on the answer. Notice that, for some fixed r , a circle with radius r can be inscribed inside P only if there exists some point inside P that has distance greater or equal than r to all the points of the boundary of P . This condition can be validated by "shrinking" the polygon inwards by a

distance of r and checking that the polygon remains non-degenerate (or is a point/segment itself). Such procedure can be simulated by taking the half-planes of the polygon sides in counter-clockwise order, translating each of them by a distance of r in the direction of the region they allow (that is, orthogonal to the direction vector of the half-plane), and checking if the intersection is not empty.

Clearly, if we can inscribe a circle of radius r , we can also inscribe any other circle of radius smaller than r . So we can perform a binary search on the radius r and validate every step using half-plane intersection. Also, note that the half-planes of a convex polygon are already sorted by angle, so the sorting step can be skipped in the algorithm. Thus we obtain a total time complexity of $O(NK)$, where N is the number of polygon vertices and K is the number of iterations of the binary search (the actual value will depend on the range of possible answers and the desired precision).

Two-dimensional linear programming

One more application of half-plane intersection is linear programming in two variables. All linear constraints for two variables can be expressed in the form of $Ax + By + C \leq 0$ (inequality comparator may vary). Clearly, these are just half-planes, so checking if a feasible solution exists for a set of linear constraints can be done with half-plane intersection. Additionally, for a given set of linear constraints, it is possible to compute the region of feasible solutions (i.e. the intersection of the half-planes) and then answer multiple queries of maximizing/minimizing some linear function $f(x, y)$ subject to the constraints in $O(\log N)$ per query using binary search (very similar to the convex hull trick).

It is worth mentioning that there also exists a fairly simple randomized algorithm that can check whether a set of linear constraints has a feasible solution or not, and maximize/minimize some linear function subject to the given constraints. This randomized algorithm was also explained nicely by Artem Vasilyev in the lecture mentioned earlier. Here are some additional resources on it, should the reader be interested: [CG - Lecture 4, parts 4 and 5](#) and [Petr Mitrichev's blog \(which includes the solution to the hardest problem in the practice problems list below\)](#).

Classic problems, direct application

- [Codechef - Animesh decides to settle down](#)

- [POJ - How I mathematician Wonder What You Are!](#)
- [POJ - Rotating Scoreboard](#)
- [POJ - Video Surveillance](#)
- [POJ - Art Gallery](#)
- [POJ - Uyuw's Concert](#)

Harder problems

- [POJ - Most Distant Point from the Sea - Medium](#)
- [Baekjoon - Jeju's Island - Same as above but seemingly stronger test cases](#)
- [POJ - Feng Shui - Medium](#)
- [POJ - Triathlon - Medium/hard](#)
- [DMOJ - Arrow - Medium/hard](#)
- [POJ - Jungle Outpost - Hard](#)
- [Codeforces - Jungle Outpost \(alternative link, problem J\) - Hard](#)
- [Yandex - Asymmetry Value \(need virtual contest to see, problem F\) - Very Hard](#)

Additional problems

- 40th Petrozavodsk Programming Camp, Winter 2021 - Day 1: Jagiellonian U Contest, Grand Prix of Krakow - Problem B: (Almost) Fair Cake-Cutting. At the time of writing the article, this problem was private and only accessible by participants of the Programming Camp.

References, bibliography and other sources

Main sources

- [New Algorithm for Half-plane Intersection and its Practical Value](#). Original paper of the algorithm.
- [Artem Vasilyev's Brazilian ICPC Summer School 2020 lecture](#). Amazing lecture on half-plane intersection. Also covers other geometry topics.

Good blogs (Chinese)

- [Fundamentals of Computational Geometry - Intersection of Half-planes.](#)
- [Detailed introduction to the half-plane intersection algorithm.](#)
- [Summary of Half-plane intersection problems.](#)
- [Sorting incremental method of half-plane intersection.](#)

Randomized algorithm

- [Linear Programming and Half-Plane intersection - Parts 4 and 5.](#)
- [Petr Mitrichev's Blog: A half-plane week.](#)

Contributors:

[Your name](#) (100.0%)

Original

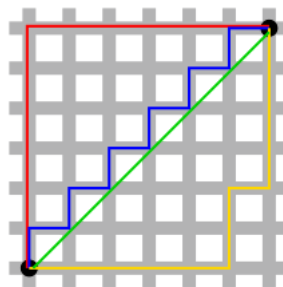
Manhattan Distance

Definition

For points p and q on a plane, we can define the distance between them as the sum of the differences between their x and y coordinates:

$$d(p, q) = |x_p - x_q| + |y_p - y_q|$$

Defined this way, the distance corresponds to the so-called [Manhattan \(taxicab\) geometry](#), in which the points are considered intersections in a well designed city, like Manhattan, where you can only move on the streets horizontally or vertically, as shown in the image below:



This images show some of the smallest paths from one black point to the other, all of them with length 12.

There are some interesting tricks and algorithms that can be done with this distance, and we will show some of them here.

Farthest pair of points in Manhattan distance

Given n points P , we want to find the pair of points p, q that are farther apart, that is, maximize $|x_p - x_q| + |y_p - y_q|$.

Let's think first in one dimension, so $y = 0$. The main observation is that we can brute force if $|x_p - x_q|$ is equal to $x_p - x_q$ or $-x_p + x_q$, because if we "miss the sign" of the absolute value, we will get only a smaller value, so it can't affect the answer. More formally, it holds that:

$$|x_p - x_q| = \max(x_p - x_q, -x_p + x_q)$$

So, for example, we can try to have p such that x_p has the plus sign, and then q must have the negative sign. This way we want to find:

$$\max_{p, q \in P} (x_p + (-x_q)) = \max_{p \in P} (x_p) + \max_{q \in P} (-x_q).$$

Notice that we can extend this idea further for 2 (or more!) dimensions. For d dimensions, we must brute force 2^d possible values of the signs. For example, if we are in 2 dimensions and brute force that p has both the plus signs we want to find:

$$\max_{p, q \in P} [(x_p + (-x_q)) + (y_p + (-y_q))] = \max_{p \in P} (x_p + y_p) + \max_{q \in P} (-x_q - y_q).$$

As we made p and q independent, it is now easy to find the p and q that maximize the expression.

The code below generalizes this to d dimensions and runs in $O(n \cdot 2^d \cdot d)$.

```
long long ans = 0;
for (int msk = 0; msk < (1 << d); msk++) {
    long long mx = LLONG_MIN, mn = LLONG_MAX;
    for (int i = 0; i < n; i++) {
        long long cur = 0;
        for (int j = 0; j < d; j++) {
            if (msk & (1 << j)) cur += p[i][j];
            else cur -= p[i][j];
        }
        mx = max(mx, cur);
        mn = min(mn, cur);
    }
}
```

```

    }
    ans = max(ans, mx - mn);
}

```

Rotating the points and Chebyshev distance

It's well known that, for all $m, n \in \mathbb{R}$,

$$|m| + |n| = \max(|m + n|, |m - n|).$$

To prove this, we just need to analyze the signs of m and n . And it's left as an exercise.

We may apply this equation to the Manhattan distance formula to find out that

$$d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2| = \max(|(x_1 + y_1) - (x_2 + y_2)|, |(y_1 - x_1) - (y_2 - x_2)|).$$

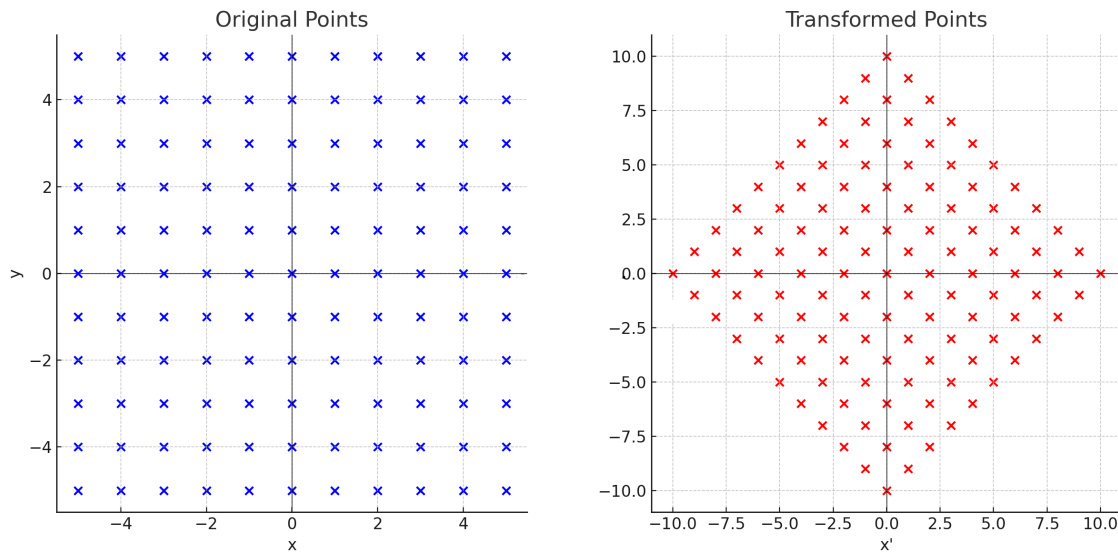
The last expression in the previous equation is the [Chebyshev distance](#) of the points $(x_1 + y_1, y_1 - x_1)$ and $(x_2 + y_2, y_2 - x_2)$. This means that, after applying the transformation

$$\alpha : (x, y) \rightarrow (x + y, y - x),$$

the Manhattan distance between the points p and q turns into the Chebyshev distance between $\alpha(p)$ and $\alpha(q)$.

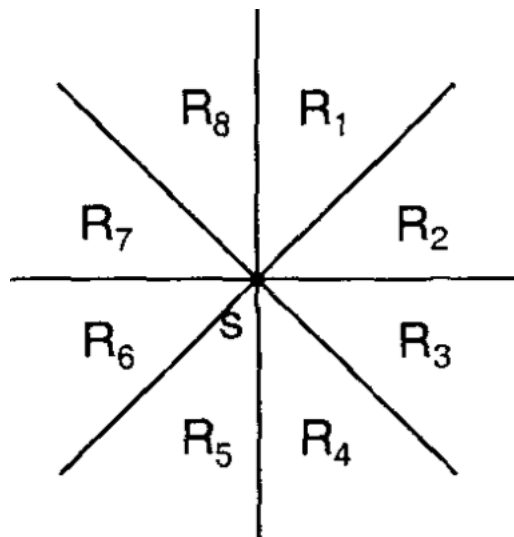
Also, we may realize that α is a [spiral similarity](#) (rotation of the plane followed by a dilation about a center O) with center $(0, 0)$, rotation angle of 45° in clockwise direction and dilation by $\sqrt{2}$.

Here's an image to help visualizing the transformation:



Manhattan Minimum Spanning Tree

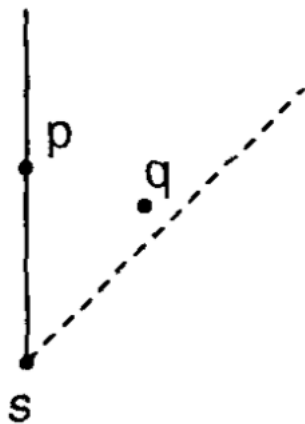
The Manhattan MST problem consists of, given some points in the plane, find the edges that connect all the points and have a minimum total sum of weights. The weight of an edge that connects two points is their Manhattan distance. For simplicity, we assume that all points have different locations. Here we show a way of finding the MST in $O(n \log n)$ by finding for each point its nearest neighbor in each octant, as represented by the image below. This will give us $O(n)$ candidate edges, which, as we show below, will guarantee that they contain the MST. The final step is then using some standard MST, for example, [Kruskal algorithm using disjoint set union](#).



The 8 octants relative to a point S

The algorithm shown here was first presented in a paper from [H. Zhou, N. Shenoy, and W. Nichollos \(2002\)](#). There is also another known algorithm that uses a Divide and conquer approach by [J. Stolfi](#), which is also very interesting and only differs in the way they find the nearest neighbor in each octant. They both have the same complexity, but the one presented here is easier to implement and has a lower constant factor.

First, let's understand why it is enough to consider only the nearest neighbor in each octant. The idea is to show that for a point s and any two other points p and q in the same octant, $d(p, q) < \max(d(s, p), d(s, q))$. This is important, because it shows that if there was a MST where s is connected to both p and q , we could erase one of these edges and add the edge (p, q) , which would decrease the total cost. To prove this, we assume without loss of generality that p and q are in the octant R_1 , which is defined by: $x_s \leq x$ and $x_s - y_s > x - y$, and then do some casework. The image below gives some intuition on why this is true.



Intuitively, the limitation of the octant makes it impossible that p and q are both closer to s than to each other

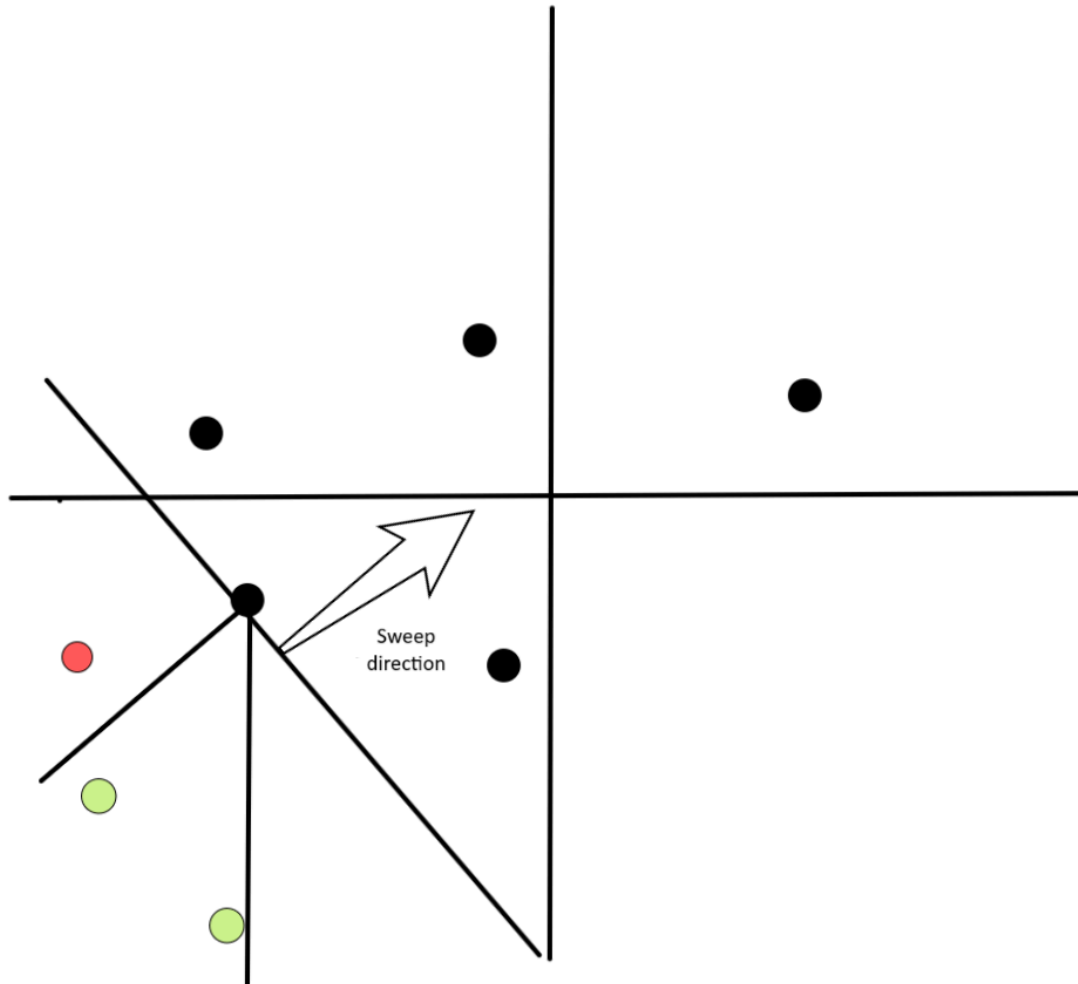
Therefore, the main question is how to find the nearest neighbor in each octant for every single of the n points.

Nearest Neighbor in each Octant in $O(n \log n)$

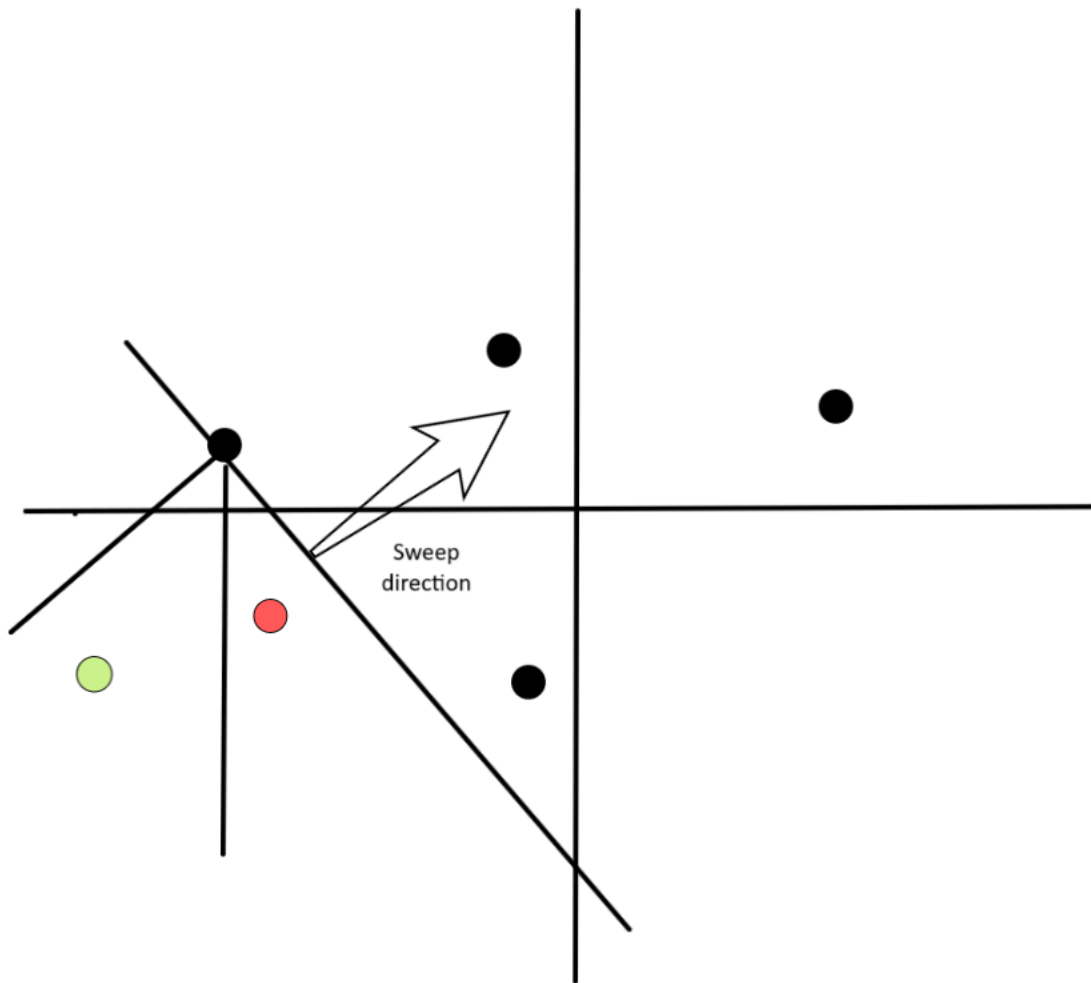
For simplicity we focus on the NNE octant (R_1 in the image above). All other directions can be found with the same algorithm by rotating the input.

We will use a sweep-line approach. We process the points from south-west to north-east, that is, by non-decreasing $x + y$. We also keep a set of points which don't have

their nearest neighbor yet, which we call "active set". We add the images below to help visualize the algorithm.



In black with an arrow you can see the direction of the line-sweep. All the points below this lines are in the active set, and the points above are still not processed. In green we see the points which are in the octant of the processed point. In red the points that are not in the searched octant.



In this image we see the active set after processing the point p . Note that the green points of the previous image had p in its north-north-east octant and are not in the active set anymore, because they already found their nearest neighbor.

When we add a new point p , for every point s that has it in its octant we can safely assign p as the nearest neighbor. This is true because their distance is $d(p, s) = |x_p - x_s| + |y_p - y_s| = (x_p + y_p) - (x_s + y_s)$, because p is in the north-north-east octant. As all the next points will not have a smaller value of $x + y$ because of the sorting step, p is guaranteed to have the smaller distance. We can then remove all such points from the active set, and finally add p to the active set.

The next question is how to efficiently find which points s have p in the north-north-east octant. That is, which points s satisfy:

- $x_s \leq x_p$
- $x_p - y_p < x_s - y_s$

Because no points in the active set are in the R_1 region of another, we also have that for two points q_1 and q_2 in the active set, $x_{q_1} \neq x_{q_2}$ and their ordering implies

$$x_{q_1} < x_{q_2} \implies x_{q_1} - y_{q_1} \leq x_{q_2} - y_{q_2}.$$

You can try to visualize this on the images above by thinking of the ordering of $x - y$ as a "sweep-line" that goes from the north-west to the south-east, so perpendicular to the one that is drawn.

This means that if we keep the active set ordered by x the candidates s are consecutively placed. We can then find the largest $x_s \leq x_p$ and process the points in decreasing order of x until the second condition $x_p - y_p < x_s - y_s$ breaks (we can actually allow that $x_p - y_p = x_s - y_s$ and that deals with the case of points with equal coordinates). Notice that because we remove from the set right after processing, this will have an amortized complexity of $O(n \log(n))$. Now that we have the nearest point in the north-east direction, we rotate the points and repeat. It is possible to show that actually we also find this way the nearest point in the south-west direction, so we can repeat only 4 times, instead of 8.

In summary we:

- Sort the points by $x + y$ in non-decreasing order;
- For every point, we iterate over the active set starting with the point with the largest x such that $x \leq x_p$, and we break the loop if $x_p - y_p \geq x_s - y_s$. For every valid point s we add the edge $(s, p, d(s, p))$ in our list;
- We add the point p to the active set;
- Rotate the points and repeat until we iterate over all the octants.
- Apply Kruskal algorithm in the list of edges to get the MST.

Below you can find a implementation, based on the one from [KACTL](#).

```
struct point {
    long long x, y;
};

// Returns a list of edges in the format (weight, u, v).
```

```
// Passing this list to Kruskal algorithm will give the Manhattan MST.
vector<tuple<long long, int, int>> manhattan_mst_edges(vector<point> ps)
{
    vector<int> ids(ps.size());
    iota(ids.begin(), ids.end(), 0);
    vector<tuple<long long, int, int>> edges;
    for (int rot = 0; rot < 4; rot++) { // for every rotation
        sort(ids.begin(), ids.end(), [&](int i, int j){
            return (ps[i].x + ps[i].y) < (ps[j].x + ps[j].y);
        });
        map<int, int, greater<int>> active; // (xs, id)
        for (auto i : ids) {
            for (auto it = active.lower_bound(ps[i].x); it !=
active.end();
                active.erase(it++)) {
                int j = it->second;
                if (ps[i].x - ps[i].y > ps[j].x - ps[j].y) break;
                assert(ps[i].x >= ps[j].x && ps[i].y >= ps[j].y);
                edges.push_back({(ps[i].x - ps[j].x) + (ps[i].y -
ps[j].y), i, j});
            }
            active[ps[i].x] = i;
        }
        for (auto &p : ps) { // rotate
            if (rot & 1) p.x *= -1;
            else swap(p.x, p.y);
        }
    }
    return edges;
}
```

Problems

- [AtCoder Beginner Contest 178E - Dist Max](#)
- [CodeForces 1093G - Multidimensional Queries](#)
- [CodeForces 944F - Game with Tokens](#)
- [AtCoder Code Festival 2017D - Four Coloring](#)
- [The 2023 ICPC Asia EC Regionals Online Contest \(I\) - J. Minimum Manhattan Distance](#)
- [Petrozavodsk Winter Training Camp 2016 Contest 4 - B. Airports](#)

Contributors:

[NaimSS](#) (71.95%) [Michael Hayter](#) (11.64%) [Gabriel Ribeiro Paiva](#) (10.05%) [Izan Beltran](#) (3.7%)
[Shoot](#) (1.59%) [Oleksandr Kulkov](#) (1.06%)

Original

Minimum Enclosing Circle

Consider the following problem:

Library Checker - Minimum Enclosing Circle

You're given $n \leq 10^5$ points $p_i = (x_i, y_i)$.

For each p_i , find whether it lies on the circumference of the minimum enclosing circle of $\{p_1, \dots, p_n\}$.

Here, by the minimum enclosing circle (MEC) we mean a circle with minimum possible radius that contains all the n points, inside the circle or on its boundary. This problem has a simple randomized solution that, on first glance, looks like it would run in $O(n^3)$, but actually works in $O(n)$ expected time.

To better understand the reasoning below, we should immediately note that the solution to the problem is unique:

? Why is the MEC unique?



Consider the following setup: Let r be the radius of the MEC. We draw a circle of radius r around each of the points $p_1, \dots, p_n$. Geometrically, the centers of circles that have radius r and cover all the points $p_1, \dots, p_n$ form the intersection of all n circles.

Now, if the intersection is just a single point, this already proves that it is unique. Otherwise, the intersection is a shape of non-zero area, so we can reduce r by a tiny bit, and still have non-empty intersection, which contradicts the assumption that r was the minimum possible radius of the enclosing circle.

With a similar logic, we can also show the uniqueness of the MEC if we additionally demand that it passes through a given specific point p_i or two points p_i and p_j (it is also unique because its radius uniquely defines it).

Alternatively, we can also assume that there are two MECs, and then notice that their intersection (which contains the points $p_1, \dots, p_n$ already) must have a smaller diameter than initial circles, and thus can be covered with a smaller circle.

Welzl's algorithm

For brevity, let's denote $\text{mec}(p_1, \dots, p_n)$ to be the MEC of $\{p_1, \dots, p_n\}$, and let $P_i = \{p_1, \dots, p_i\}$.

The algorithm, initially [proposed](#) by Welzl in 1991, goes as follows:

1. Apply a random permutation to the input sequence of points.
2. Maintain the current candidate to be the MEC C , starting with $C = \text{mec}(p_1, p_2)$.
3. Iterate over $i = 3..n$ and check if $p_i \in C$.
 - a. If $p_i \in C$ it means that C is the MEC of P_i .
 - b. Otherwise, assign $C = \text{mec}(p_i, p_1)$ and iterate over $j = 2..i$ and check if $p_j \in C$.
 - i. If $p_j \in C$, then C is the MEC of P_j among circles that pass through p_i .
 - ii. Otherwise, assign $C = \text{mec}(p_i, p_j)$ and iterate over $k = 1..j$ and check if $p_k \in C$.

- A. If $p_k \in C$, then C is the MEC of P_k among circles that pass through p_i and p_j .
- B. Otherwise, $C = \text{mec}(p_i, p_j, p_k)$ is the MEC of P_k among circles that pass through p_i and p_j .

We can see that each level of nestedness here has an invariant to maintain (that C is the MEC among circles that also pass through additionally given 0, 1 or 2 points), and whenever the inner loop closes, its invariant becomes equivalent to the invariant of the current iteration of its parent loop. This, in turn, ensures the *correctness* of the algorithm as a whole.

Omitting some technical details, for now, the whole algorithm can be implemented in C++ as follows:

```
struct point {...};

// Is represented by 2 or 3 points on its circumference
struct mec {...};

bool inside(mec const& C, point p) {
    return ...;
}

// Choose some good generator of randomness for the shuffle
mt19937_64 gen(...);
mec enclosing_circle(vector<point> &p) {
    ranges::shuffle(p, gen);
    auto C = mec{p[0], p[1]};
    for(int i = 0; i < n; i++) {
        if(!inside(C, p[i])) {
            C = mec{p[i], p[0]};
            for(int j = 0; j < i; j++) {
                if(!inside(C, p[j])) {
                    C = mec{p[i], p[j]};
                    for(int k = 0; k < j; k++) {
                        if(!inside(C, p[k])) {
                            C = mec{p[i], p[j], p[k]};
                        }
                    }
                }
            }
        }
    }
}
```

```
    return C;
}
```

Now, it is to be expected that checking that a point p_i is inside the MEC of 2 or 3 points can be done in $O(1)$ (we will discuss this later on). But even then, the algorithm above looks as if it would take $O(n^3)$ in the worst case just because of all the nested loops. So, how come we claimed the linear expected runtime? Let's figure out!

Complexity analysis

For the inner-most loop (over k), clearly its expected runtime is $O(j)$ operations. What about the loop over j ?

It only triggers the next loop if p_j is on the boundary of the MEC of P_j that also passes through point i , *and removing p_j would further shrink the circle*. Of all points in P_j there can only be at most 2 points with such property, because if there are more than 2 points from P_j on the boundary, it means that after removing any of them, there will still be at least 3 points on the boundary, sufficient to uniquely define the circle.

In other words, after initial random shuffle, there is at most $\frac{2}{j}$ probability that we get one of the at most two unlucky points as p_j . Summing it up over all j from 1 to i , we get the expected runtime of

$$\sum_{j=1}^i \frac{2}{j} \cdot O(j) = O(i).$$

In exactly same fashion we can now also prove that the outermost loop has expected runtime of $O(n)$.

Checking that a point is in the MEC of 2 or 3 points

Let's now figure out the implementation detail of `point` and `mec`. In this problem, it turns out to be particularly useful to use `std::complex` as a class for points:

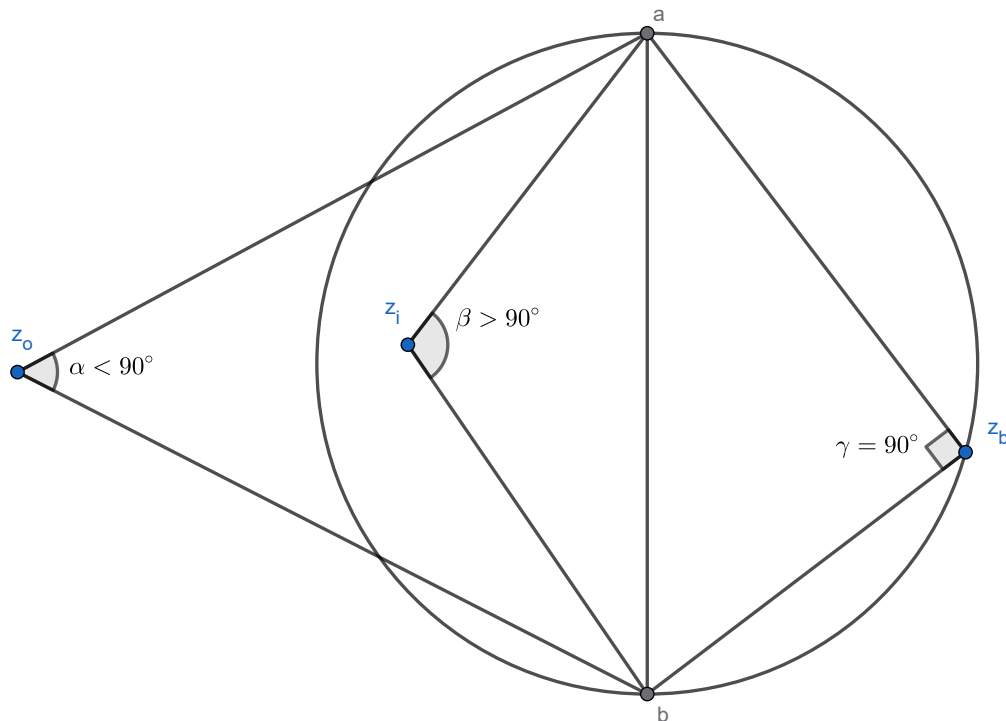
```
using ftype = int64_t;
using point = complex<ftype>;
```

As a reminder, a complex number is a number of type $x + yi$, where $i^2 = -1$ and $x, y \in \mathbb{R}$. In C++, such complex number is represented by a 2-dimensional point (x, y) . Complex numbers already implement basic component-wise linear operations (addition, multiplication by a real number), but also their multiplication and division carry certain geometric meaning.

Without going in too much detail, we will note the most important property for this particular task: Multiplying two complex numbers adds up their polar angles (counted from Ox counter-clockwise), and taking a conjugate (i.e. changing $z = x + yi$ into $\bar{z} = x - yi$) multiplies the polar angle with -1 . This allows us to formulate some very simple criteria for whether a point z is inside the MEC of 2 or 3 specific points.

MEC of 2 points

For 2 points a and b , their MEC is simply the circle centered at $\frac{a+b}{2}$ with the radius $\frac{|a-b|}{2}$, in other words the circle that has ab as a diameter. To check if z is inside this circle we simply need to check that the angle between za and zb is not acute.



Inner angles are obtuse, external angles are acute and angles on the circumference are right

Equivalently, we need to check that

$$I_0 = (b - z)\overline{(a - z)}$$

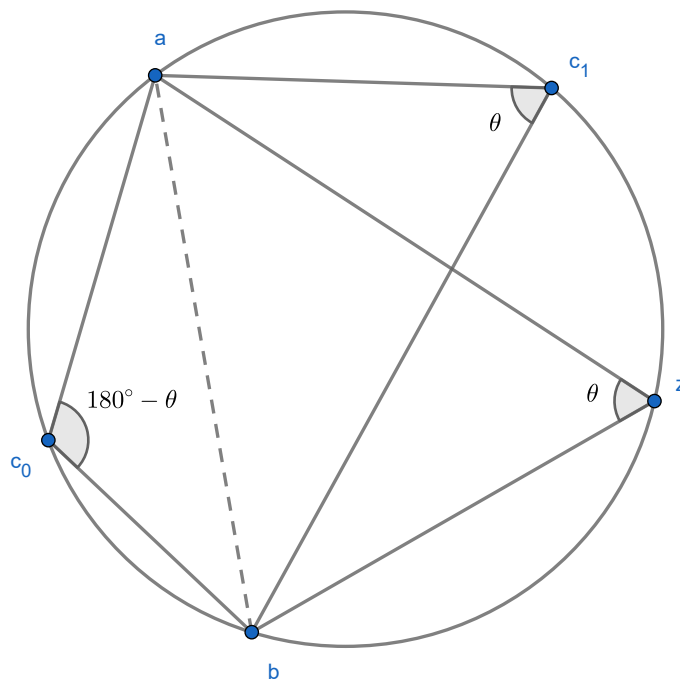
doesn't have a positive real coordinate (corresponding to points that have a polar angle between -90° and 90°).

MEC of 3 points

Adding z to the triangle abc will make it a quadrilateral. Consider the following expression:

$$\angle azb + \angle bca$$

In a [cyclic quadrilateral](#), if c and z are from the same side of ab , then the angles are equal, and will add up to 0° when summed up signed (i.e. positive if counter-clockwise and negative if clockwise). Correspondingly, if c and z are on the opposite sides, the angles will add up to 180° .



Adjacent inscribed angles are same, opposing angles complement to 180 degrees

In terms of complex numbers, we can note that $\angle azb$ is the polar angle of $(b - z)\overline{(a - z)}$ and $\angle bca$ is the polar angle of $(a - c)\overline{(b - c)}$. Thus, we can conclude that $\angle azb + \angle bca$ is the polar angle of

$$I_1 = (b - z)\overline{(a - z)}(a - c)\overline{(b - c)}$$

If the angle is 0° or 180° , it means that the imaginary part of I_1 is 0, otherwise we can deduce whether z is inside or outside of the enclosing circle of abc by checking the sign of the imaginary part of I_1 . Positive imaginary part corresponds to positive angles, and negative imaginary part corresponds to negative angles.

But which one of them means that z is inside or outside of the circle? As we already noticed, having z inside the circle generally increases the magnitude of $\angle azb$, while having it outside the circle decreases it. As such, we have the following 4 cases:

1. $\angle bca > 0^\circ$, c on the same side of ab as z . Then, $\angle azb < 0^\circ$, and $\angle azb + \angle bca < 0^\circ$ for points inside the circle.
2. $\angle bca < 0^\circ$, c on the same side of ab as z . Then, $\angle azb > 0^\circ$, and $\angle azb + \angle bca > 0^\circ$ for points inside the circle.
3. $\angle bca > 0^\circ$, c on the opposite side of ab to z . Then, $\angle azb > 0^\circ$ and $\angle azb + \angle bca > 180^\circ$ for points inside the circle.
4. $\angle bca < 0^\circ$, c on the opposite side of ab to z . Then, $\angle azb < 0^\circ$ and $\angle azb + \angle bca < 180^\circ$ for points inside the circle.

In other words, if $\angle bca$ is positive, points inside the circle will have $\angle azb + \angle bca < 0^\circ$, otherwise they will have $\angle azb + \angle bca > 0^\circ$, assuming that we normalize the angles between -180° and 180° . This, in turn, can be checked by the signs of imaginary parts of $I_2 = (a - c)\overline{(b - c)}$ and $I_1 = I_0 I_2$.

Note: As we multiply four complex numbers to get I_1 , the intermediate coefficients can be as large as $O(A^4)$, where A is the largest coordinate magnitude in the input. On the bright side, if the input is integer, both checks above can be done fully in integers.

Implementation

Now, to actually implement the check, we should first decide how to represent the MEC. As our criteria work with the points directly, a natural and efficient way to do this is to say that MEC is directly represented as a pair or triple of points that defines it:

```
using mec = variant<
    array<point, 2>,
    array<point, 3>
>;
```

Now, we can use `std::visit` to efficiently deal with both cases in accordance with criteria above:

```
/* I < 0 if z inside C,
   I > 0 if z outside C,
   I = 0 if z on the circumference of C */
ftype indicator(mec const& C, point z) {
    return visit([&](auto &&C) {
        point a = C[0], b = C[1];
        point I0 = (b - z) * conj(a - z);
        if constexpr (size(C) == 2) {
            return real(I0);
        } else {
            point c = C[2];
            point I2 = (a - c) * conj(b - c);
            point I1 = I0 * I2;
            return imag(I2) < 0 ? -imag(I1) : imag(I1);
        }
    }, C);
}

bool inside(mec const& C, point p) {
    return indicator(C, p) <= 0;
}
```

Now, we can finally ensure that everything works by submitting the problem to the Library Checker: [#308668](#).